

Texas Instruments

TMS320DM320 System Specification

Version 0.40

Preliminary

Revision History

Revision N0.	Date	Modification List
Version 0.1	11/25/2002	First version based on DM270
Version 0.2	12/19/2002	Updated Preview, CCDC, ARM, VLYNQ, USB,
Version 0.2	12/19/2002	Updated Timers, OSD, VENC, Image Buffers, iMX, ARM Boot
Version 0.3	1/26/2003	SDRAM (100 MHz, Size), PLL/CLK Drawing, Imx corrections, 3A windows, MS Pro support, Mode0/1 support in CCDC
Version 0.35	3/3/2003	CCDC drawing, Preview noise filter equation, AF window register value, AIM size in external cpu mode drawing, Separate Y and CR CB filters in VENC, added DATA FORMATTER to CCDC
Version 0.40	7/15/03	Updated all sections to latest module specs H3a v04, INT v04e, MTC v02e, tmrc v01e, usb v01, ARM926 v1.0, CCDC v0.3, CLKC v0.9e, EHIF v04, GIO v3.4e, I2C v0.1e, INTC v0.3e, mmCIF v0.2, OSD v0.2, PRV v0.61, SIF v0.6e, TIMER v0.1, VENC v0.3, Vlynq v0.0, Coprocessor v0.8, DCT v1.0, iMX v1.2, VLCD v1.3

Preliminary

Contents

1	FEATURES AND PERFORMANCE TARGET	10
1.1	Key features:.....	10
2	BLOCK DIAGRAM	12
3	SYSTEM DATA FLOWS	15
3.1	Preview Mode	15
3.2	Still Picture Capture Mode	15
3.3	Video Capture Mode	16
3.4	Still Picture and Video Playback	17
4	IMAGING PERIPHERALS.....	17
4.1	CCD Controller	17
4.1.1	Features.....	18
4.1.2	Signal Interface	19
4.1.3	The Typical CCDC Interface	19
4.1.4	CCD Controller Processing Modules.....	20
4.1.5	Digital Clamp	20
4.1.6	Black Level Compensation.....	21
4.1.7	Data Formatter	21
4.1.8	Faulty Pixel Correction.....	21
4.1.9	Gain and Offset	22
4.1.10	Output Formatter	23
4.1.11	CCDC Interrupts.....	24
4.1.12	Timing Generator	24
4.1.13	VGA movie data formatter.....	25
4.1.14	YCbCr signal Processing	26
4.1.15	Input/Output port configuration.....	26
4.2	Preview Engine.....	27
4.2.1	Features.....	29
4.2.2	CFA Arrangements	30
4.2.3	Horizontal noise filter.....	30
4.2.4	White Balance	31
4.2.5	CFA Interpolation	31
4.2.6	Black Adjustment Sub-Module.....	33
4.2.7	RGB2RGB Blending Module.....	33
4.2.8	Gamma Correction Module	33
4.2.9	RGB2YCbCr Conversion Matrix	33
4.2.10	Histogram	34
4.2.11	4:2:2 Conversion and Output Interface Modules	35
4.2.12	4:2:2 SDRAM Input mode	35
4.3	AF/AE/AWB Statistics Engine	36
4.3.1	Features.....	36
4.3.2	AF Engine.....	37
4.3.3	AE/AWB Engine.....	40
4.4	On Screen Display (OSD) and Graphics Acceleration	43
4.4.1	Features.....	43
4.4.2	OSD data storage	44
4.4.3	Setting Up an OSD Window.....	44

4.4.4	Location Register	44
4.4.5	Color Look Up Table	45
4.4.6	Blending and Transparency	45
4.4.7	Hardware Cursor	45
4.4.8	Window Magnification	46
4.5	NTCS/PAL Encoder	47
4.5.1	Composite analog video features	48
4.5.2	Digital video features	48
4.5.3	Signal Level	49
4.5.4	Signal calculation	50
4.5.5	Color signal cutoff frequency select	50
4.5.6	Video attribute insertion	50
4.5.7	Digital RGB/YUV Interface	51
5	DSP SUBSYSTEM.....	52
5.1	DSP core.....	52
5.1.1	Features.....	52
5.1.2	DSP peripherals	53
5.1.3	Multi-channel buffered serial ports	53
5.1.4	DSP subsystem memory.....	53
5.1.5	Program data and IO spaces	54
5.1.6	DSP Memory map	54
5.1.7	I/O memory space.....	55
5.2	Image Buffer	56
5.2.1	Access Switching	57
5.2.2	The Image Buffer's DMA Controller	58
5.3	Imaging Extension (iMX).....	59
5.3.1	iMX Structure.....	59
5.3.2	DSP/iMX Shared Memory Interface	60
5.4	DCT Engine	62
5.5	Sequencer.....	62
5.6	VLCD Engine.....	62
6	ARM SUBSYSTEM.....	65
6.1	Features.....	65
6.2	ARM926EJ-S RISC Processor Core	66
6.3	Cache (I-cache and D-cache) system	66
6.4	Memory Map.....	66
6.4.1	ARM Internal RAM	68
6.4.2	Peripheral Registers	69
6.4.3	DSP Memory	69
6.4.4	External Memory	70
6.5	Interrupt Controller.....	70
6.5.1	INTC Methodology	71
6.5.2	Hardware Configurable Interrupt Entry Table	72
6.6	Timers, Watch-Dog Timer.....	73
6.7	PLL / Clock Controller	74
6.8	ARM Boot-up Sequence.....	76
6.8.1	CS0 Boot	76
6.8.2	Serial boot-up mode.....	76
6.8.3	External CPU boot-up mode	77
6.8.4	Internal ROM boot-up mode.....	77

7	MEMORY TRAFFIC CONTROLLER.....	79
7.1	Memory map.....	79
7.2	Blocks Diagram	79
7.2.1	Access to Memory Traffic Controller(MTC) Areas	80
7.2.2	Boundary Addresses and Sizes of the DAEM Regions	81
7.3	External Memory Interface.....	82
7.3.1	Hardware Connection.....	82
7.3.2	Timing control	82
7.3.3	External Memory Region 0: The FLASH Memory Interface	84
7.3.4	External Memory Region 1: The CompactFlash Interface	85
7.3.5	External Memory Region 2: The SmartMedia Interface	86
7.3.6	External Memory Regions 3 and 4	87
7.4	SDRAM controller.....	88
7.4.1	Features:.....	89
7.4.2	Hardware Connection.....	90
7.4.3	Memory Structure Setting.....	90
7.4.4	SDRAM command	90
7.4.5	SDRAM Initialization	91
7.5	MTC DMA Controller.....	91
7.5.1	Sources and destinations devices	92
7.5.2	Sources/Destinations Addresses and Transfer Size	92
7.6	SDRAM & External Memory I/F Pin Sharing.....	93
8	COMMUNICATION INTERFACE.....	97
8.1	USB 2.0 Full Speed OTG	97
8.1.1	Features.....	97
8.1.2	USB Functional Description	98
8.1.3	USB Device Configuration.....	98
8.1.4	USB Power Management.....	99
8.1.5	DMA Data Transfer	99
8.2	VLYNQ.....	99
8.3	SD/MMC Interface.....	100
8.4	Memory Stick Interface	100
8.5	Serial Port	101
8.6	UART	101
8.7	I ² C Interface.....	102
8.8	GIO.....	102
8.8.1	Parallel IO mode	102
8.8.2	External Interrupt Settings	102
8.8.3	GIO Functional Settings	102
8.9	Card Detect Function	103
9	EXTERNAL CPU INTERFACE	104
9.1	Features	104
9.2	Memory Space	105
9.3	Bus Multiplex Access Mode.....	106
9.4	Access Time	106
9.5	SDRAM access	107
10	ARM-DSP COMMUNICATION.....	108
10.1	Introduction	108
10.2	HPI-16 mode.....	108

10.3	Memory Access	109
10.4	HPI Interrupt.....	109
10.4.1	Host Device Using DSPINT0 to Interrupt the DSP	109
10.4.2	DSP Using Bit 0 of 0xFFFF to Interrupt the Host Device	109
10.5	Boot loader.....	109
10.5.1	ARM boot sequence	109
10.5.2	DSP Boot Sequence.....	110
11	POWER SAVING MODES	112
11.1	DSP Idle modes.....	112
11.1.1	IDLE1 Mode	112
11.1.2	IDLE2 Mode	112
11.2	ARM Power Saving Modes.....	112
11.2.1	Sleep Mode:	112
11.2.2	Sleep by sleep mode register setting.....	113
11.2.3	Power-down mode.....	113
11.2.4	Wake up Method.....	113
11.2.5	PLL Bypass Mode.....	113
11.3	DAC Power	114
12	PIN ASSIGNMENT	115

Preliminary

Figures

Figure 1 DSC engine (DM320) system architecture	12
Figure 2 Bus Diagram	14
Figure 3 Live View mode	15
Figure 4 Still Picture Capture mode	16
Figure 5 Motion Picture Capture mode	16
Figure 6 Play Back mode	17
Figure 7 CCD controller Block Diagram	18
Figure 8 CCD signal processing	19
Figure 9 Digital Clamp Block Diagram	21
Figure 10 Median Filter (Mode 0)	22
Figure 11 Output Formatter Block Diagram	23
Figure 12 Frame Image format conversion	24
Figure 13 Frame Image Format	25
Figure 14 YCbCr signal processing	26
Figure 15 Video Port Configuration	27
Figure 16 Video System Configuration	27
Figure 17 Preview Engine block diagram	29
Figure 18 Conventional 2x2 CFA formats	30
Figure 19 Foveon sensor format	30
Figure 20 White Balancing in the Preview Engine	31
Figure 21 CFA Interpolation Module (Normal Mode)	32
Figure 22 CFA Interpolation Module (Resize Only Mode)	32
Figure 23 Pseudo-color Suppression Functional Mapping	34
Figure 24 Block diagram of the Luminance non-linear enhancer	34
Figure 25 Block Diagram of 3A Hardware Engine	37
Figure 26 Pixel Configuration	38
Figure 27 IIR Filter	39
Figure 28 Window configuration	41
Figure 29 OSD block diagram	44
Figure 30 OSD Window Example	44
Figure 31 Hardware Cursor in OSD	46
Figure 32 NTSC/PAL Encoder Block Diagram	47
Figure 33 Input Video Data	50
Figure 34 : Coprocessor/DSP Subsystem Block Diagram	54
Figure 35 DSP Memory Map	55
Figure 36 : Shared Memory Block	57
Figure 37 Two dimensional data transfer	58
Figure 38 Structure of eight parallel MACs	60
Figure 39 Co Processor Memory Configuration	60
Figure 40 VLCD Block Diagram	63
Figure 41 ARM Subsystem	66
Figure 42 : DM320 ARM926 Memory Map	68
Figure 43 INTC Methodology	72
Figure 44 Interrupt Entry Table	73
Figure 45 Clock Controller Block	75
Figure 46 Serial data read timing chart	76
Figure 47 Serial data write timing chart	77
Figure 48 : DM320 AIM Memory Map	78
Figure 49 Memory Traffic Controller blocks diagram	80
Figure 50 External Memory Bus Timings	84
Figure 51 Flash Memory Interface Data Bus	85
Figure 52 CompactFlash Interface	86

Figure 53 SmartMedia Interface	87
Figure 54 External memory bus timing with hardware wait-states	88
Figure 55 SDRAM Controller Block	89
Figure 56 EMIF DMA Controller	92
Figure 57 Pin sharing	93
Figure 58 Memory connection on DM320	95
Figure 59 Block Diagram of USB	98
Figure 60 Device Configuration	98
Figure 61 Role of the MMC Controller	100
Figure 62 Host Interface mode	105
Figure 63 ARM-DSP Communication	108
Figure 64 Host Interface mode	110

Tables

Table 1 Data Input Formats	19
Table 2 CCDC Signal Interface	20
Table 3 SDRAM Data Format	23
Table 4 SDRAM Data Format(YCbCr)	26
Table 5 YCbCr-4:2:2 Format	35
Table 6 Comparison of the two input mode	35
Table 7 SDRAM Data Format	36
Table 8 AF Engine Configuration Registers	37
Table 9 Configuration Registers	40
Table 10 Blending and transparency in OSD	45
Table 11 Magnification Ratio	46
Table 12 : Digital Output Mode pin	51
Table 13 : DSP I/O Address for Coprocessors	55
Table 14 Memory Structure	57
Table 15 byte-to word conversion	58
Table 16 Memory Structure	67
Table 17 Peripheral module memory map	69
Table 18 DM320 Interrupts	70
Table 19 ARM boot-up configuration	77
Table 20 ARM memory address map	79
Table 21 Initial values of offset	81
Table 22 Chip select signal priority order	81
Table 23 External Memory Bus Pins	82
Table 24 Configurable Timing Parameters for CS0/CS3/CS4	82
Table 25 Configurable Timing Parameters for CS1	83
Table 26 Configurable Timing Parameters for CS2	83
Table 27 FLASH_CE Bus Width	85
Table 28 Relationship Between Access Size and Flash Memory Data Bus Width	85
Table 29 CompactFlash Interface Memory Map	86
Table 30 SmartMedia Interface Memory Map	87
Table 31 SDRAM Bus Pins	90
Table 32 Correspondence between memory structure and address	90
Table 33 DMA transfer source, transfer destination devices	92
Table 34 Shared Memory Configuration	93
Table 35 External HOST CPU Configuration	94
Table 36 Memory Bus Connection	95
Table 37 GIO port selection output	102
Table 38 Memory Space	105
Table 39 ARM Memory Map	109
Table 40 External HOST Memory Map	109

Preliminary

1 Features and Performance Target

The DM320 is a highly integrated, programmable platform for the digital still camera and mobile system. Designed to offer camera manufacturers the ability to produce affordable DSC products with high picture quality, the DM320 combines programmable image processing capability with a highly integrated imaging peripheral set. The chip's programmability comes from a DSP-based imaging co-processor that enables camera manufacturers to implement their own proprietary image processing algorithms in software. The DM320 also enables seamless interface to most additional external devices required for a complete digital camera digital implementation. The interface is flexible enough to support various types of CCD and CMOS sensors, signal conditioning circuits, power management, SDRAM, shutter, Iris and auto-focus motor controls. The DM320 will allow camera manufacturers to meet customer demands by fulfilling both image quality and low cost expectations required in the volume point and shoot digital still camera segment.

1.1 Key features:

Key features:

- Supports capture of 9M pixels at <1 second SDRAM-to-SDRAM processing time
- Supports Motion JPEG compression/decompression video at VGA resolution up to 30 frames per sec
- Supports MPEG4 I+P frame video VGA resolution up to 30 frames per sec

DSP/CPU:

- On-chip 160 MHz ARM926EJ-S 32 bit RISC microprocessor with DSP extension and Java bytecode execution, and with I-cache (16 Kbytes), D-cache (8 Kbytes), on-chip program/data memory (16 Kbytes), and 8KB on-chip ROM.
- 100MHz C54x DSP processor with 128Kbytes on-chip memory to assist image/video compression and decompression.
- On-chip 180MHz programmable 8-MAC SIMD image processing engine (IMX) for programmable image processing, with 16-way parallel motion estimation accelerator
- On-chip image processing buffers (24Kbytes)
- Variable Length Coding and Decoding coprocessor (VLCD) for image/video compression and decompression.
- Hardware DCT module for DCT and IDCT operations

System:

- 308-pin micro star BGA(μ *BGA) 16x16mm
- Supports up to 100-MHz SDRAM, up to 256 Mbytes with 16/32-bit interface
- Interface to Low Voltage SDRAM / Flash (2.5V)
- Seamless interface to Smart Media, SD, MMC, and Memory Stick media storage cards
- Supports 8/16-bit NAND flash memory
- Supports general-purpose 16-bit parallel address/data interface
 - Support linear address space for ease of programming
 - Provides chip enable and wait signals
 - Programmable setup, strobe and hold times for up to 4 devices
 - Support NOR flash, SRAM, and CompactFlash Card
- Supports 48MHz and 27MHz crystal input to generate all clocks

Imager:

- Supports up to 14-bit AFE (Analog Front End)
- Support for up to 4 field CCD frame transfer (Software/Hardware)

- Support for 8/16 CCIR601/656 video format

Display:

- Preview engine for real-time live-view and digital zoom
- Able to produce 16-bit digital YCbCr output
- Versatile OSD controller
 - Two video windows (YCbCr 4:2:2)
 - One OSD bitmap window
 - One attribute window
 - One bit-map cursor windows (32x32)
 - Horizontal (X2) and vertical zoom (X4) on video and OSD
 - 1-bit, 2-bit, 4-bit or 8-bit OSD data with on-chip ROM Color LUT
 - Supports Video and OSD pixel-level blending (8 steps)
 - Supports pixel-level blinking
 - Real-time aspect ratio conversion to support square pixel input
- Digital interface for color LCD
 - 8 bit RGB Digital interface to Epson, Displaytech and Unipac LCD controllers
 - ITU-R601 interface to Sanyo and Kodak LCD's
 - On-chip control logic for direct digital panel interface to Epson, Casio, Sony, and Hitachi LCD panels

Peripherals:

- Four Timers + One WDT
- Two UARTs (TXD, RXD each set)
- Two I2S/AC97 serial audio codec interface – glueless interface to low cost audio codecs
- Two sets of SPI serial interface (host or slave initiation)
- I2C Interface (single master mode)
- Up to 41 General-purpose IO pins available (33 MUX'd + 8 dedicated)
- USB 2.0 OTG (Full Speed)
- VLYNQ high speed peripheral interface
- 2 channel programmable PWM (pulse width modulation) outputs
- On-chip bootloader ROM to optionally boot from flash, NAND flash, UART, or serial port.

Preliminary

2 Block Diagram

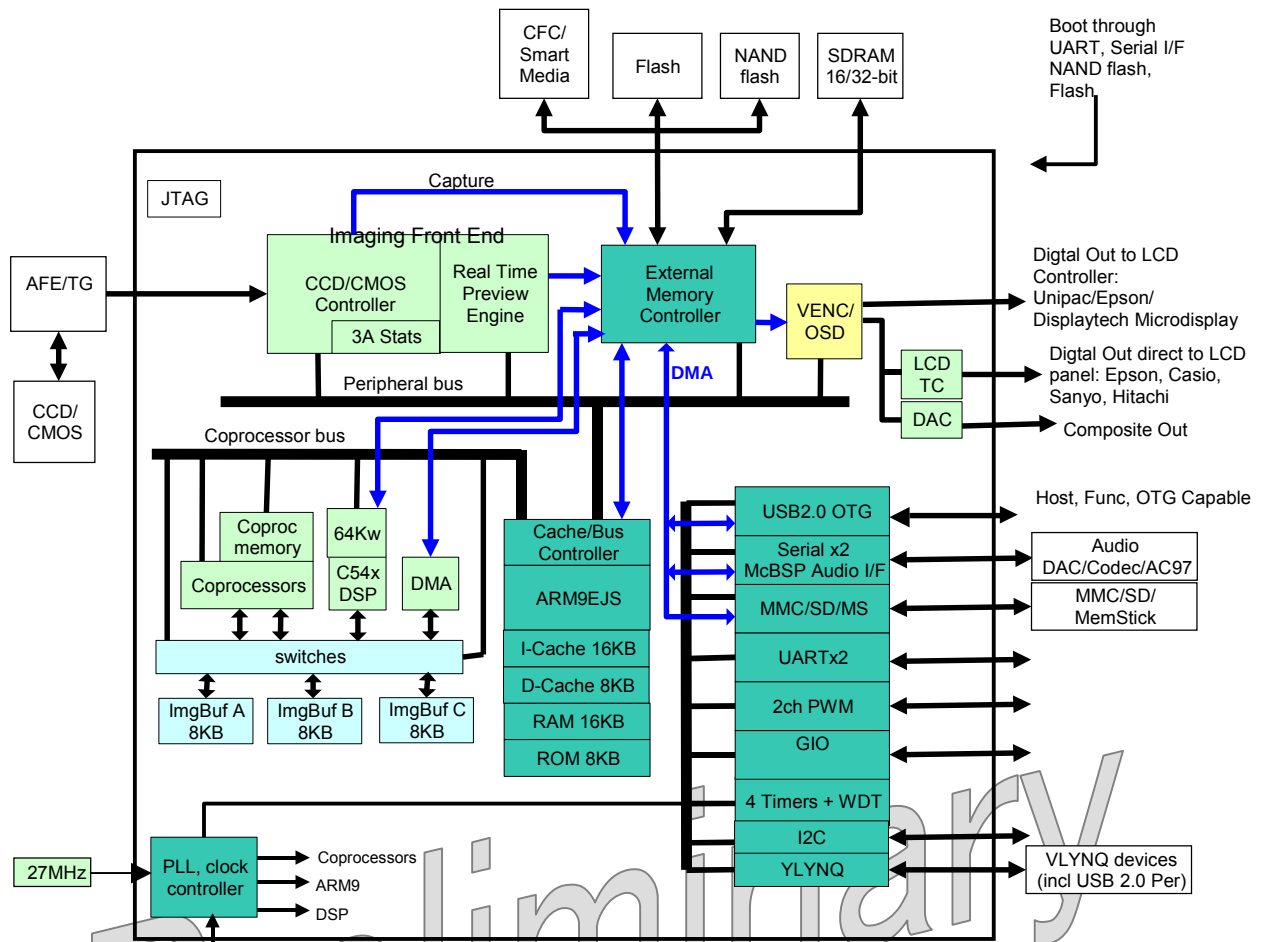


Figure 1 DSC engine (DM320) system architecture

Preliminary

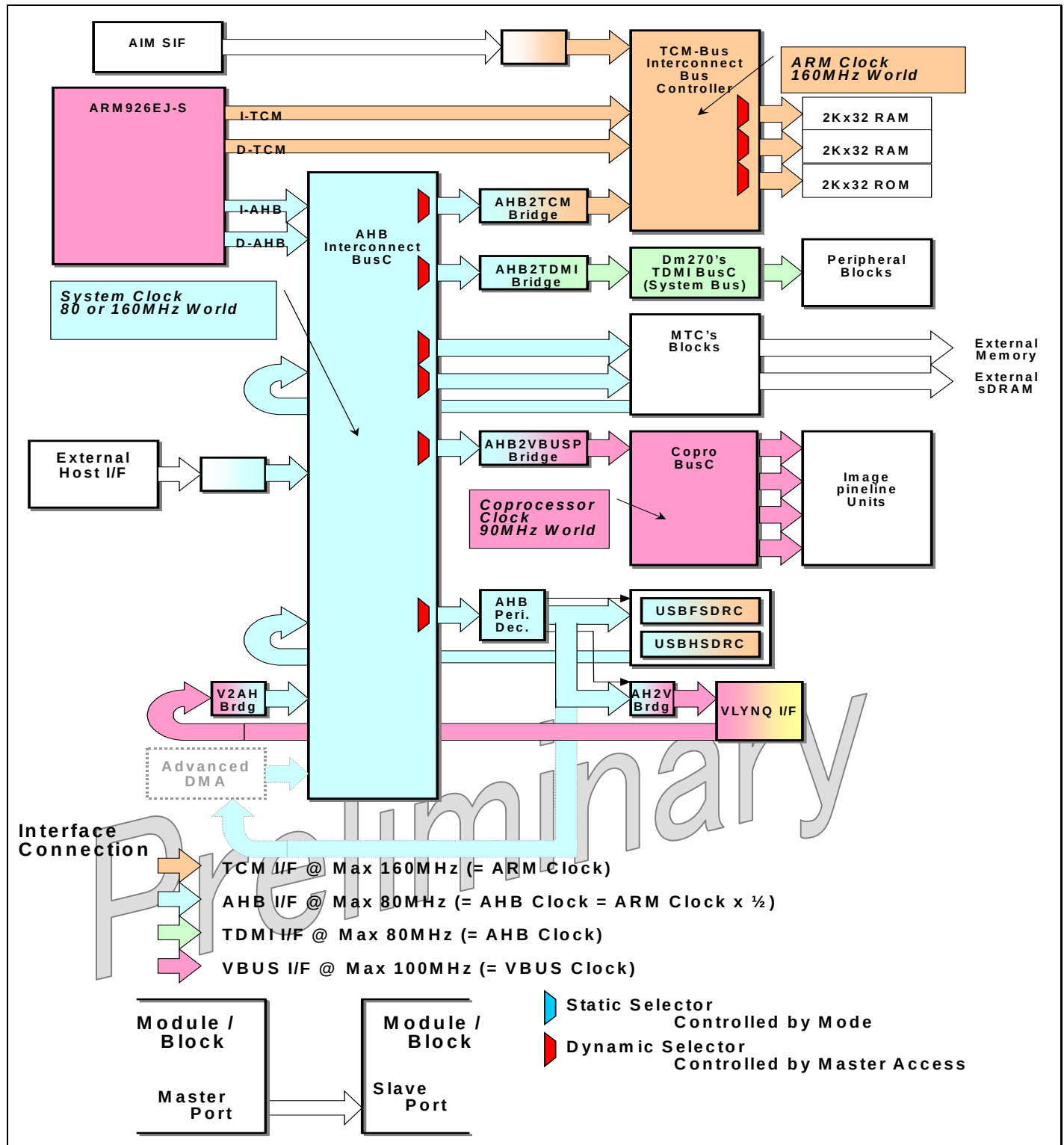


Figure 2 Bus Diagram

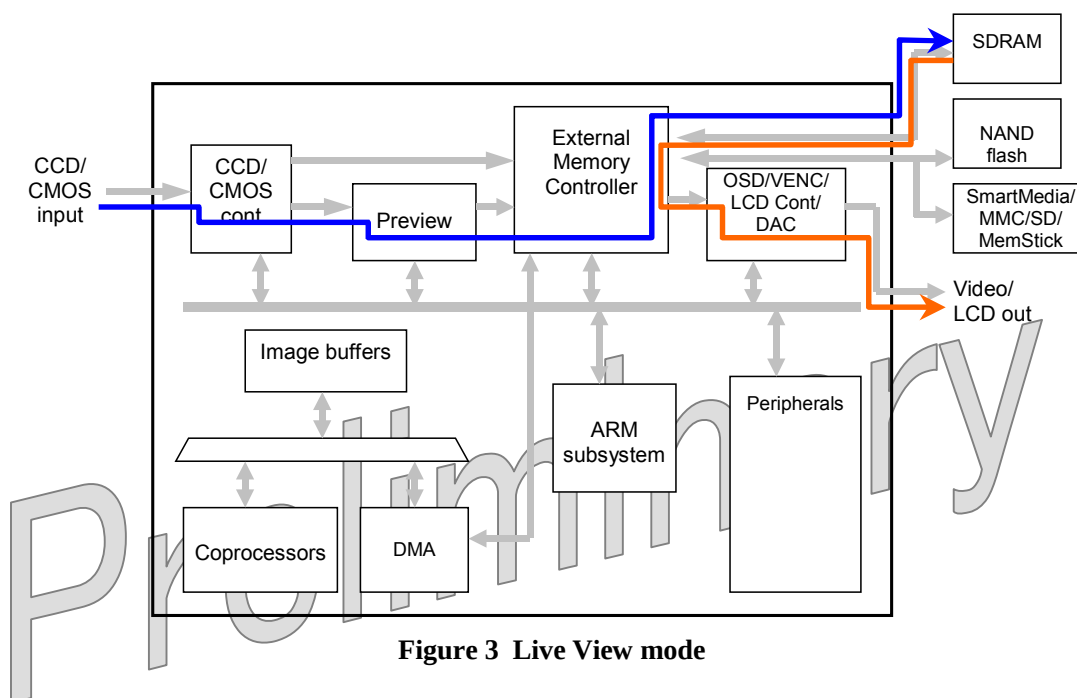
3 System Data Flows

3.1 Preview Mode

In the preview mode, CCD or CMOS imager data comes in through the CCD controller to the preview engine. CCD controller has fault pixel correction, optical black clamping, and required timing control and interface. Preview engine contains a simplified image processing pipeline for AE/AWB, CFA interpolation, down sampling, and gamma lookups. The CCD/CMOS sensor should be configured to draft mode output, and preview engine handles further down-sampling, if necessary, to produce LCD/video resolution. Preview output is written to SDRAM.

DM320 has hardwired 3A (AF/AE/AWB) calculator. Each result is stored in SDRAM, ARM can read its data and controls the AFE.

The display data is read out by the OSD module, and merged/blended with the OSD data. The display data is sent to video encoder (VENC) or LCD controller, which sends composite analog or digital LCD data to the external display device.



3.2 Still Picture Capture Mode

In the still picture capture mode, CCD or CMOS imager data comes in through the CCD controller and SDRAM controller directly to SDRAM first.

When there is a sufficient amount of data to start processing, which could be 20-24 lines with non-interlaced data and slightly over half-frame for interlaced data, ARM is notified via an interrupt signal from CCD controller. ARM then coordinates with iMX, DCT, VLCD, and DMA controller to transfer in the data from SDRAM into image buffer and process the data. The processing flow is software programmable and customer-specific. Typically compressed JPEG bitstream is the final output, and it's written back into SDRAM.

ARM then handles media storage card file system setup, sector header setup, and triggers DMA to write data from SDRAM into the card.

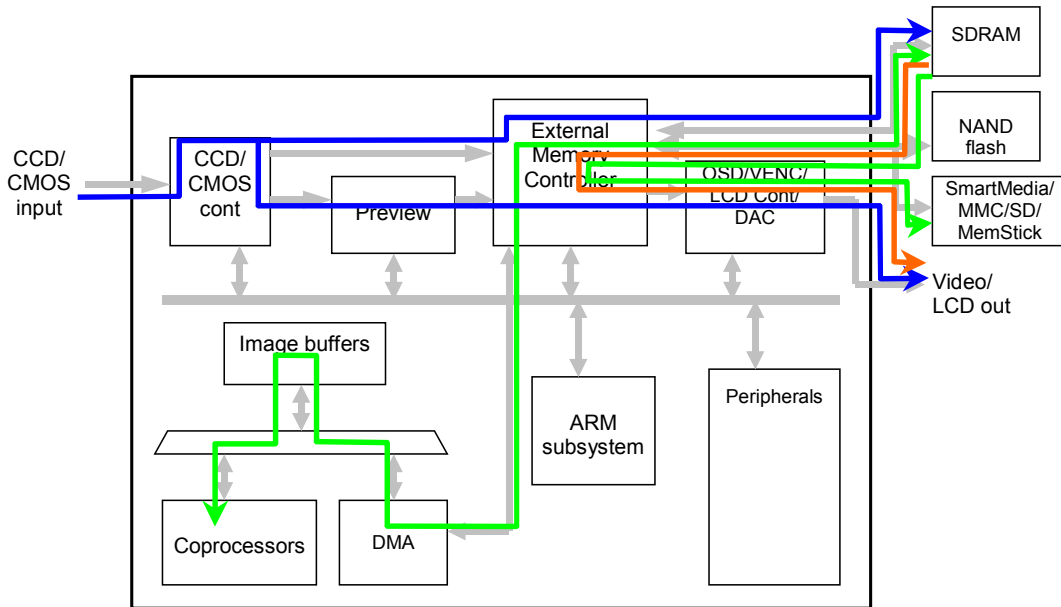


Figure 4 Still Picture Capture mode

3.3 Video Capture Mode

Video capture is similar to still picture capture, except for the input stage. For video capture, we use Preview Engine to down sample the CCD/CMOS imager data to the video capture size (VGA, CIF, QCIF etc) before writing into SDRAM. This offloads much processing in the programmable engines and reduces power consumption due to processing and traffic.

Once sufficient amount of the video data is written into SDRAM, again ARM gets an interrupt from CCD controller and triggers the processing steps. MPEG-1 or MPEG-4 compression is performed with the coordination of ARM, iMX, DCT, VLCD and the DMA controller. The compressed MPEG bitstream is written back to SDRAM.

The video data produced by Preview Engine are also pulled out by OSD module to produce display data for composite analog video or for digital LCD interface.

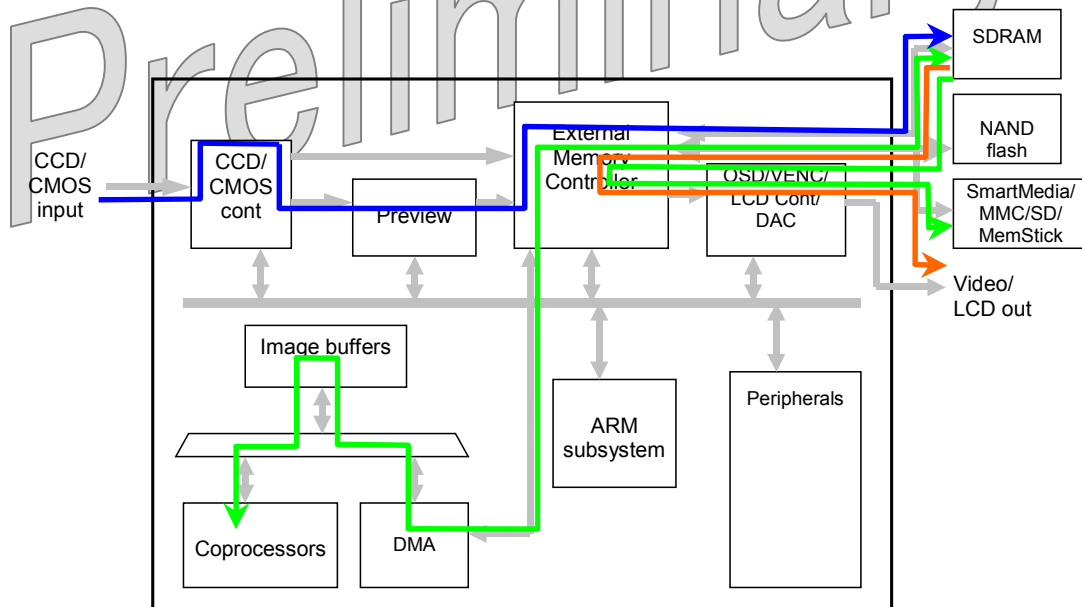


Figure 5 Motion Picture Capture mode

3.4 Still Picture and Video Playback

Still pictures or video clips can be played back on the digital camera. Typically the compressed data is first read out of the media card into SDRAM. Next the compressed data is processed and transported by ARM, coprocessors and DMA to uncompressed and, if necessary, resized into display resolution, and stored back into SDRAM.

The display data is then pulled out by OSD to supply VENC or LCD controller for analog composite video or digital LCD interface.

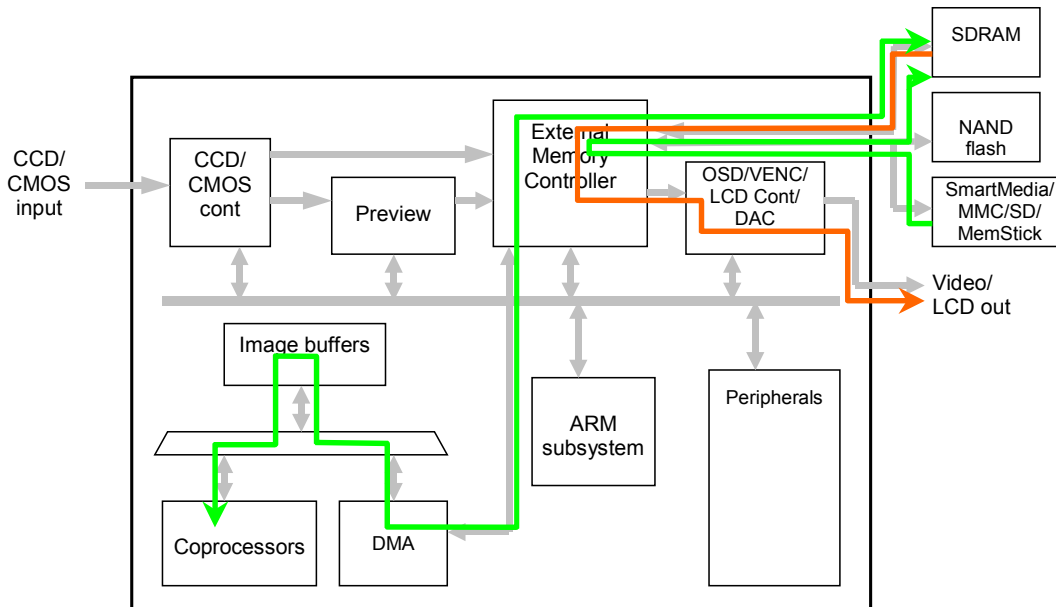


Figure 6 Play Back mode

4 Imaging Peripherals

4.1 CCD Controller

The CCD Controller captures and processes the physical data from the CCD sensor. The A/D converter and Timing Generator for CCD sensor are external to DM320 chip. The CCD controller can provide a HD/VD to external timing generator or synchronize to HD/VD given from external timing generator. The other function of CCD controller is a digital clamping and decimation for physical image capturing.

4.1.1 Features

- ❑ Generates timing signals VD/HD and field ID
- ❑ Can synchronize on externally generated HD/VD signals
- ❑ Supports progressive scan and interlaced CCD
- ❑ Generates optical black clamping control signals
- ❑ Programmable culling pattern
- ❑ Maximum supported pixel clock is 100 MHz
- ❑ Maximal supported CCD imager size 32768 X 32768
- ❑ Digital clamping
- ❑ Black level compensation
- ❑ Median filtering
- ❑ Gain and offset control
- ❑ 10-to-8 bit A-law compression table inside
- ❑ Supports CCIR601/656 Digital YCbCr 4:2:2 (8/16bit) interface
- ❑ Supports Bayer, Non Bayer VGA, and Foveon style patterns

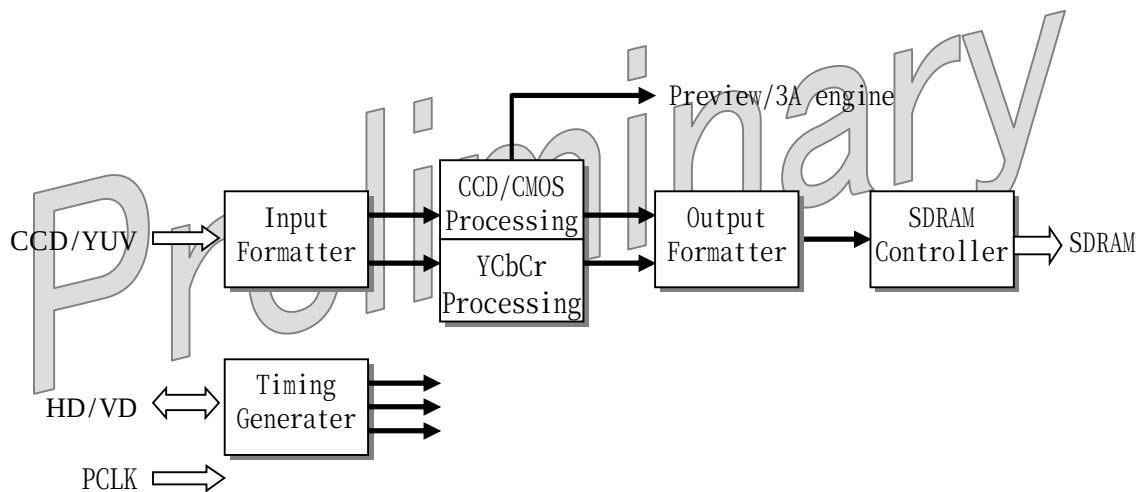


Figure 7 CCD controller Block Diagram

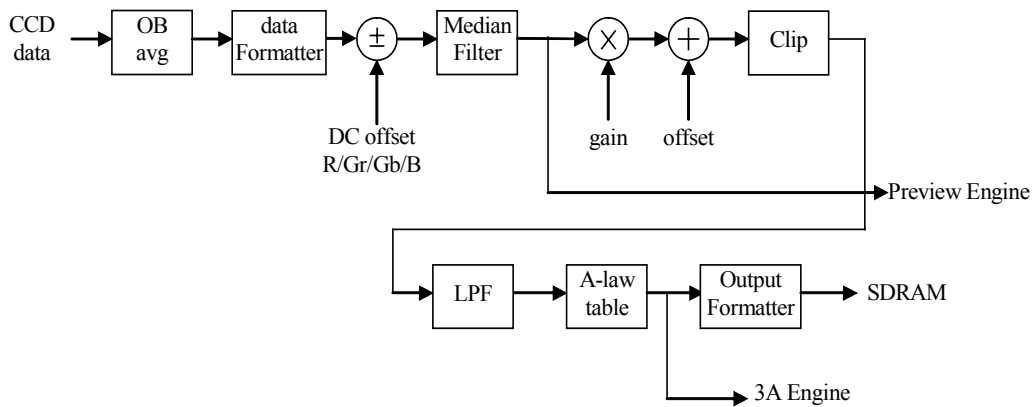


Figure 8 CCD signal processing

4.1.2 Signal Interface

The CCDC interface signals are listed in Table 2. The interface consists of a set of signals used to transfer raw CCD data from an imager to the CCDC. Additionally, the CCDC can be configured to operate in a mode that adheres to the CCIR 601/656 interface specification. The CCIR 601/656 specification provides a standard method, to transfer 8/16-bit, YCbCr-4:2:2 formatted video data.

The CCDC supports 8 to 14-bit wide data signals. When the CCD has fewer than 14 data lines, connect the CCD to the lower or upper data lines C_DATA[13]-C_DATA[0], and ground the upper or lower, unused lines.

Table 1 Data Input Formats

Pin Name	CCD	16-bit YCbCr	8-bit YCbCr
CI7		Cb7,Cr7	Y7,Cb7,Cr7
CI6		Cb6,Cr6	Y6,Cb6,Cr6
CI5	C_DATA13	Cb5,Cr5	Y5,Cb5,Cr5
CI4	C_DATA12	Cb4,Cr4	Y4,Cb4,Cr4
CI3	C_DATA11	Cb3,Cr3	Y3,Cb3,Cr3
CI2	C_DATA10	Cb2,Cr2	Y2,Cb2,Cr2
CI1	C_DATA9	Cb1,Cr1	Y1,Cb1,Cr1
CI0	C_DATA8	Cb0,Cr0	Y0,Cb0,Cr0
YI7	C_DATA7	Y7	Y7,Cb7,Cr7
YI6	C_DATA6	Y6	Y6,Cb6,Cr6
YI5	C_DATA5	Y5	Y5,Cb5,Cr5
YI4	C_DATA4	Y4	Y4,Cb4,Cr4
YI3	C_DATA3	Y3	Y3,Cb3,Cr3
YI2	C_DATA2	Y2	Y2,Cb2,Cr2
YI1	C_DATA1	Y1	Y1,Cb1,Cr1
YI0	C_DATA0	Y0	Y0,Cb0,Cr0

4.1.3 The Typical CCDC Interface

Timing and control of the interface is done using the HD, VD, field ID, pixel clock, and write enable signals. As shown in Table 2, the CCDC controller can either provide HD, VD, and field signals to the CCD imager, or it can use

the HD, VD, and field ID signals provided by the CCD imager. The pixel clock clocks data into the CCDC at a maximum rate of 40MHz. When the CCDC is configured to write data to SDRAM, the write enable signal allows an external device to control whether-or-data the current data will be written to SDRAM.

Table 2 CCDC Signal Interface

Name	I/O	Function
C_DATA[13]- C_DATA[0]	I	Image data loaded by CCD. Bit width can be can be configured to 8 to 14 bits. The polarity of the input image data can be reversed.
C_VSYNC (VD)	I/O	VSYNC. Vertical synch signal. This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the VD signal. When configured as an output the DM320 will supply the VD signal. The polarity of VD can be reversed.
C_HSYNC (HD)	I/O	HSYNC. Horizontal synch signal. This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the HD signal. When configured as an output the DM320 will supply the HD signal. The polarity of HD can be reversed.
C_FIELD[1:0]	I/O	Field identification signal. This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the field identification signal. When configured as an output the DM320 will supply the field identification signal. When the field identification signal is set to be an input to the DM320, this signal can be configured to be latched by the VD signal. The polarity of the field identification signal can be reversed. Up to 4 fields can be handled.
C_PCLK	I	Pixel clock. This signal is the pixel clock used to load image data into the CCDC. The Clock controller can configure to trigger on the rising or falling edge of the PCLK signal. The maximum pixel clock rate is 100 MHz.

4.1.4 CCD Controller Processing Modules

As shown in [fig 8](#), the CCDC consists of several data processing modules that format and enhance raw CCD data. There are two destinations for the processed output data, SDRAM and the Preview Engine. The following sections describe the modules that perform the following functions.

- ❑ Digital clamping
- ❑ Black level compensation
- ❑ Median Filtering
- ❑ Gain & Offset
- ❑ Gamma Correction
- ❑ Output Formatter

4.1.5 Digital Clamp

The first sub-module is the digital clamp block. The image data is latched at the rising or falling edge of the input pixel clock (PCLK). The averaging circuit takes an average of masked (black) pixel values from the image sensor and this value is subtracted from the image data. The user controls the position of the black pixels, the number of pixels (8 or 16) in each line averaged, and the number of lines (8 or 16) averaged.

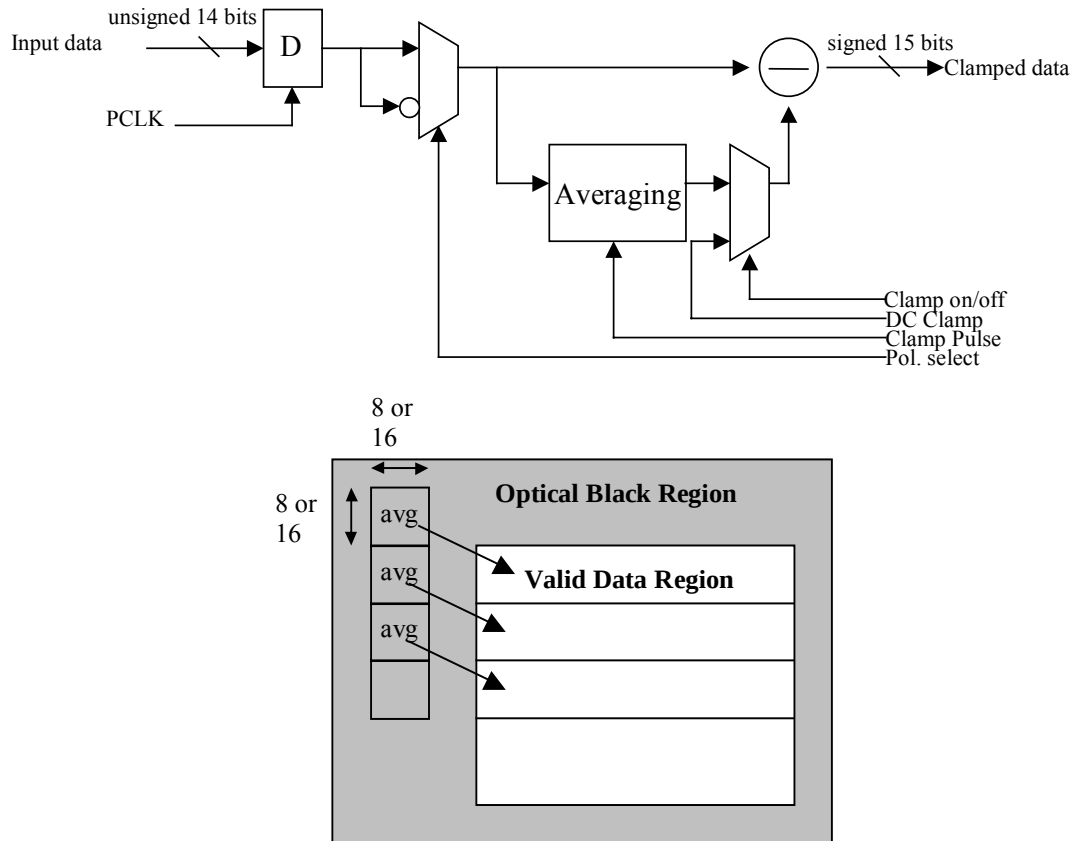


Figure 9 Digital Clamp Block Diagram

Alternately, the user can disable black clamp averaging and select a constant black value for subtraction, instead of using the calculated average value for subtraction.

4.1.6 Black Level Compensation

After the Digital Clamp is applied to the data, Black Level Compensation is applied to the R/Ye, Gr/Cy, Gb/G, and B/Mg signals. The offset applied to the current data sample is selected according to the even/odd phase and the color specified for each phase.

4.1.7 Data Formatter

New CCDs are being developed that have non standard read out patterns. The Data Formatter block allows the DM320 to handle a wide variety of current and future readout schemes. Two line memories and a programmable address generator are used to translate new patterns to that of a standard Bayer pattern (actually Bayer does not have to be the output pattern). This allows the back end processing (noise filters, interpolation, histogram, 3A statistics) to remain unchanged.

4.1.8 Faulty Pixel Correction

Two methods of faulty pixel correction. Mode 0, the Median Filter, shown in [Fig 10](#) calculates the absolute difference between the current pixel (i) and pixel (i-2) and between the current pixel (i) and pixel (i+2). If the absolute differences exceeds a threshold, the average of pixel (i-2) and pixel (i+2) replaces pixel (i). A line of CCD data is input into the Median Filter, so by operating on pixels (i-2), (i), and (i+2) the filter is applied to like pixels in a CCD data CFA pattern. The Median Filter's threshold is configurable and the Median Filter can be enabled or disabled.

```
for (i=0, i< length_pixel, i++){
```

```

sub1 = input_data(i) - input_data(i-2);
sub2 = input_data(i) - input_data(i+2);
if ((abs(sub1) > THRESHOLD && abs(sub2) > THRESHOLD) &&
    ((sub1 < 0 && sub2 < 0) || (sub1 > 0 && sub2 > 0)))
    output_data(i) = (input_data(i-2) + input_data(i+2))/2;
else
    output_data(i) = input_data(i);
}

```

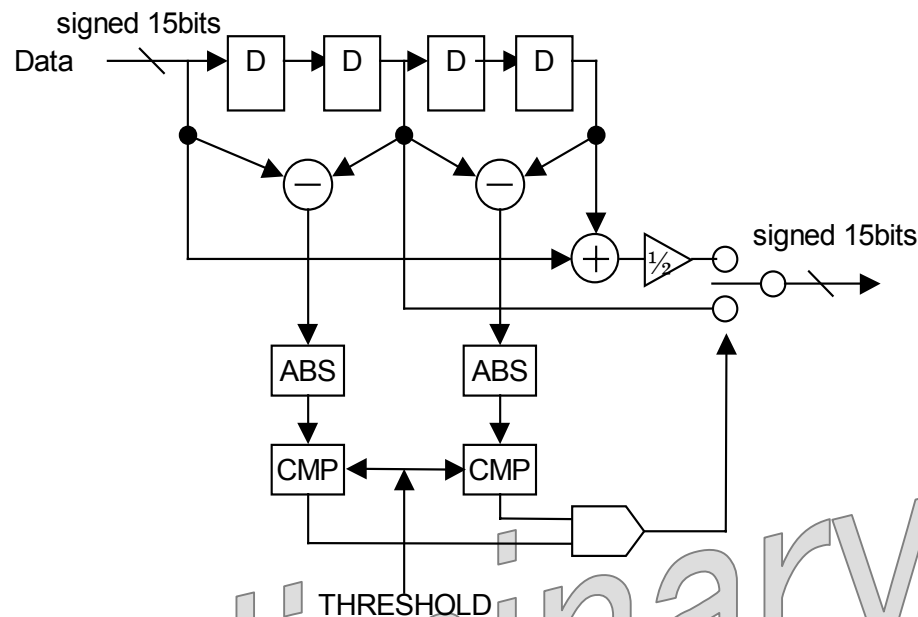


Figure 10 Median Filter (Mode 0)

Mode 1 corrects the current pixel if it differs from both same-color neighbors by more than a programmable threshold. If faulty the pixel is replaced with the median of the 3 pixels.

```

diff1 = data(i) - data(i-2);
diff2 = data(i) - data(i+2);
if ( |diff1| > TH && |diff2| > TH )
    outp(i) = Median (data(i-2), data(i), data(i+2));
else
    outp(i) = data(i);

```

4.1.9 Gain and Offset

A gain factor from 0 to 7.9921875 can be applied to the output of the Median Filter. After multiplying the data by the gain factor an offset can be added to the result. The offset can range from 0 to 1023. The result CCD data of gain control is clipped to signed 15bit data, and finally clipped to unsigned 14bit data after offset control.

4.1.10 Output Formatter

The output formatter block provides options for applying an anti-aliasing filter for horizontal culling. The low-pass filter consists of a simple three-tap ($\frac{1}{4}$, $\frac{1}{2}$, $\frac{1}{4}$) filter. In the data compression pass, any 10 bits of 14bit CCD data are compressed to 8 bits and every four pixels are stored to SDRAM at the same time. This format can secure the less SDRAM capacity. The A-law table has a same characteristic as Voice codec.

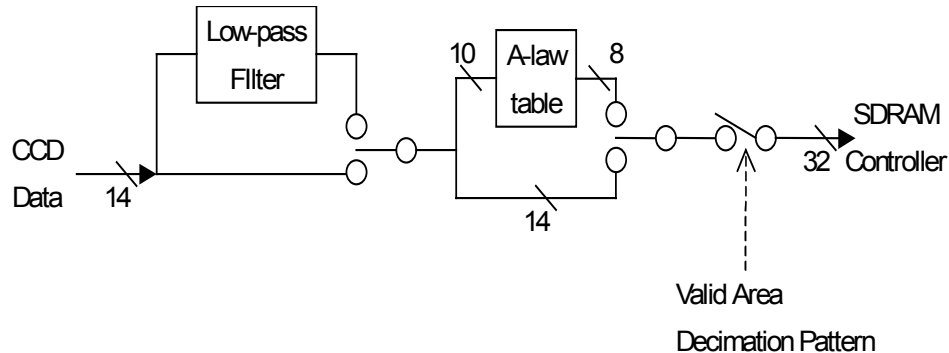


Figure 11 Output Formatter Block Diagram

The final stage is a programmable decimation function along both the horizontal and vertical directions.

Two registers control the horizontal and vertical decimation of CCD Data. The 16-bit CULH register sets the horizontal culling for even and odd lines. The 8-bit CULV register sets the one along the vertical direction. The LSB of CULV represent the top line of the CCD, the MSB is the 8th line. The bits of data from each pixel are stored in the upper bits of a 16-bit SDRAM word, and the unused bit positions are filled with zeros. The SDRAM data format is shown in Table 3.

Table 3 SDRAM Data Format

	Lower word		Upper word	
	LSB(0)	MSB(15)	LSB(16)	MSB(31)
14bit	Pixell0	0	Pixell1	0
13bit	Pixell0	0	Pixell1	0
12bit	Pixell0	0	Pixell1	0
11bit	Pixell0	0	Pixell1	0
10bit	Pixell0	0	Pixell1	0
9bit	Pixell0	0	Pixell1	0
8bit	Pixell0	0	Pixell1	0
8bit pack	Pixell0	Pixell1	Pixell2	Pixell3

The processed CCD data is transferred to the SDRC in units of 32 bytes. The CCDC has an internal, 16-bit by 16-bytes FIFO. Data is transferred from the FIFO to the SDRC in 32-byte bursts.

Data is only written to SDRAM or External Memory when WEN in SYNCEN is set to "1".

Furthermore, the output formatter can configure to any image format by using SDRAM line offset register, and offset control registers.

Figure 12 shows how to construct a frame format in SDRAM.

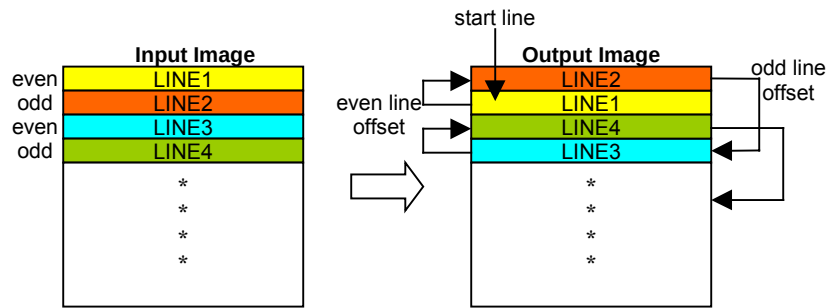
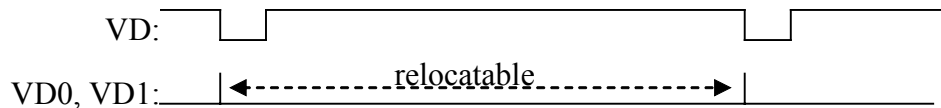


Figure 12 Frame Image format conversion

4.1.11 CCDC Interrupts

The CCDC can generate two different interrupts: VD0 and VD1. The VD0 & VD1 interrupts can be set at any position in VD interval.

CCDC interrupts



4.1.12 Timing Generator

The timing generator provides the use of external sync signals (HD/VD) or internal generated timing signals. The ARM controls width, polarity, position and direction of internal generated signals. 0 shows various CCDC register settings related to the timing. The shaded area is the physical imager size and the gray area is the valid data area. The image data in this area will be processed and stored to external SDRAM. The vertical start position for even and odd fields can be configured independently.

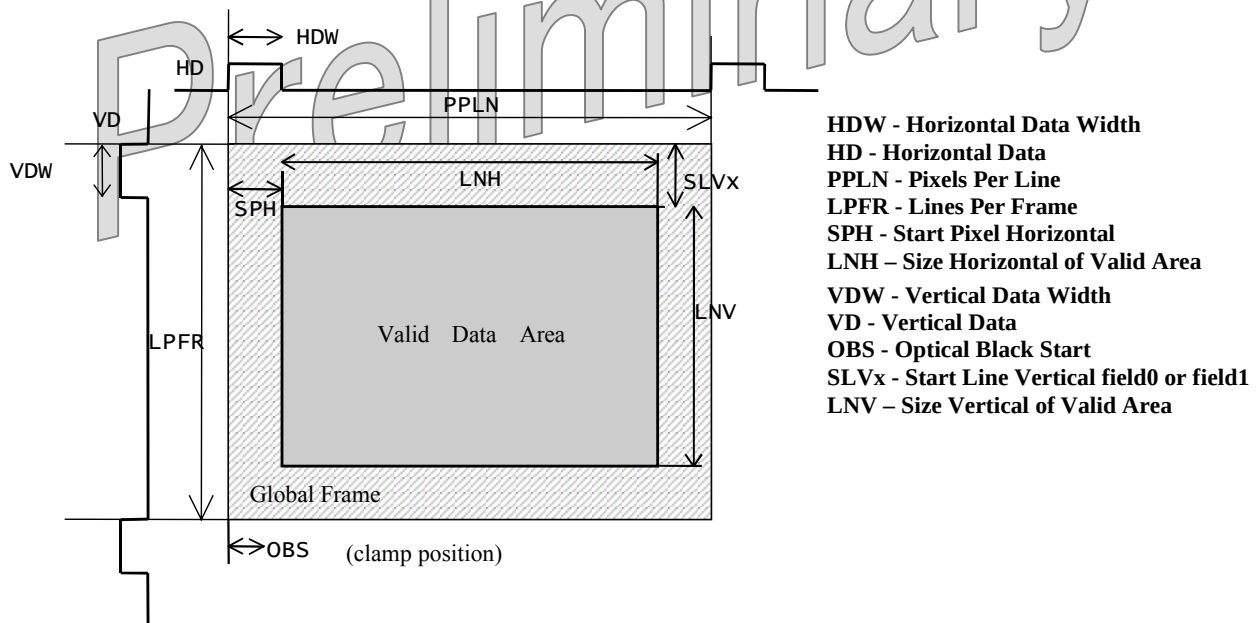
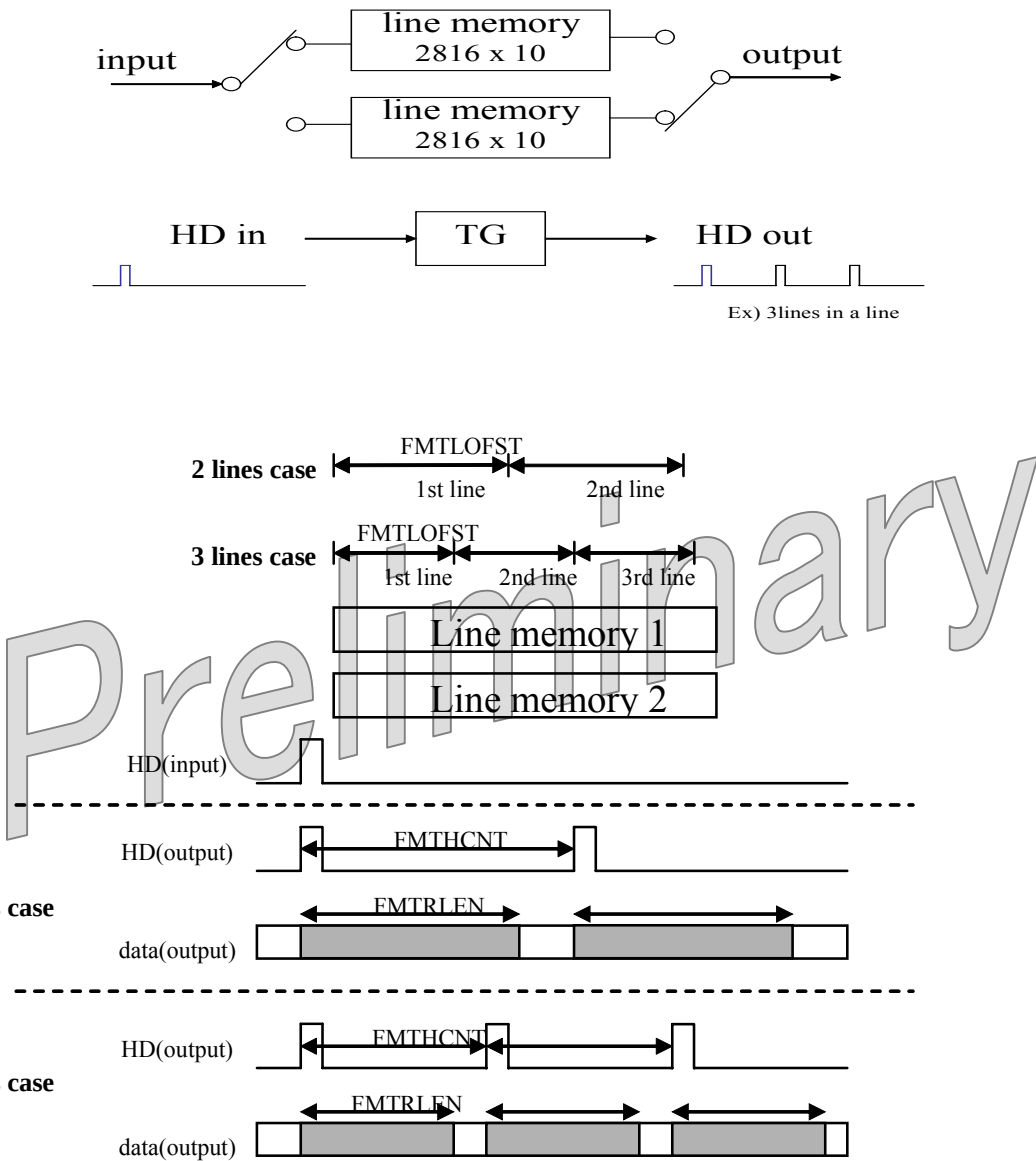


Figure 13 Frame Image Format

4.1.13 VGA movie data formatter

Convert input data includes multiple pixels in a line to each line output



4.1.14 YCbCr signal Processing

Figure 14 shows a YCbCr signal processing flow in the CCD controller block. The CCD controller accepts 4:2:2 sampled YCbCr input data. The luminance and color difference signals are each 8 bits, scaled 0 to 255. The color difference signals are multiplexed into one 8-bit bus beginning with a Cb sample. The Y and CbCr busses may be input parallel(16-bit mode) or may be time multiplexed and input as a single bus(8-bit mode). The single bus may also contain SAV and EAV video timing reference codes(ITU-R BT.656 mode). In CCIR656 mode, the CCD controller is controlled by the SAV(Start Active Video) time code in the 8-bit pixel data. All timing information is transmitted using a 4-byte synchronizaion pattern. A synchronaization pattern is sent immediately before and after each line during active picture and retrace. DM320 decodes SAV code and synchronized at this timing.

The 16-bit or 8-bit YCbCr data is stored in SDRAM as 4:2:2 format. Table 4 shows data format in SDRAM. Y data typically has a range of 16 to 235, however, it is possible to subscribe a DC value to Y signal.

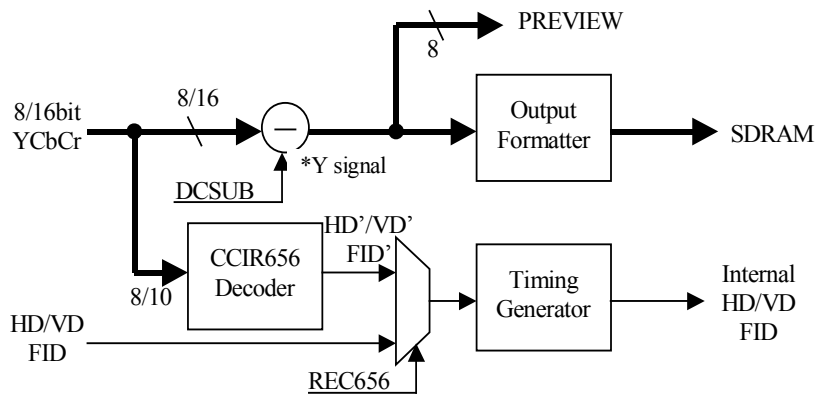


Figure 14 YCbCr signal processing

Table 4 SDRAM Data Format(YCbCr)

SDRAM Address	Lower word		Upper word	
	LSB(0)	MSB(15)	LSB(16)	MSB(31)
N	Cb0	Y0	Cr0	Y1
N+1	Cb2	Y2	Cr2	Y3
N+2	Cb4	Y4	Cr4	Y5

4.1.15 Input/Output port configuration

DM320 allows to have several port configuration as shown in Figure 15. Byte swapping of input/output(see エラー! 参照元が見つかりません。) port or input data through mode is configurable by internal CPU registers. Figure 16 shows a example of system construction.

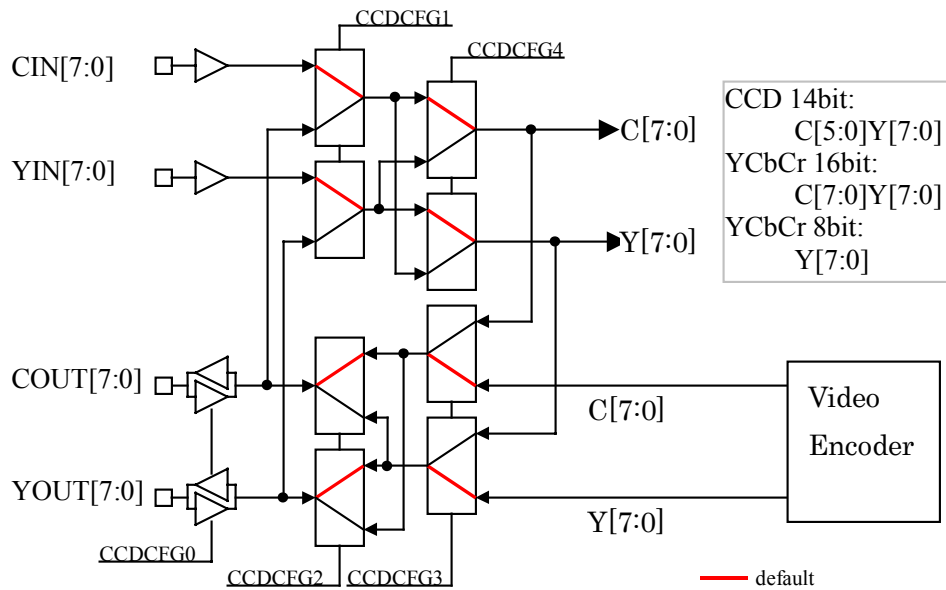


Figure 15 Video Port Configuration

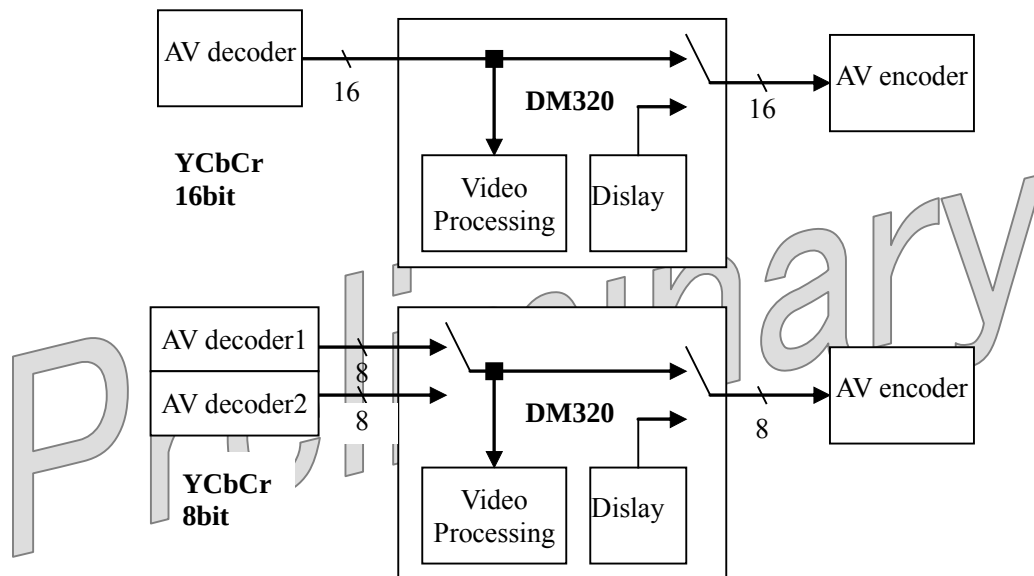


Figure 16 Video System Configuration

4.2 Preview Engine

The Preview Engine is a programmable hardware video processing module that generates image data in YCbCr-4:2:2 format from raw CCD data. The Preview Engine can also be configured to operate in a resize only mode, which allows the raw CCD data to be resized and converted to YCbCr-4:2:2 without applying the processing of every module in the Preview Engine. As shown in Figure 17, several internal modules are used to process CCD data into YCbCr-4:2:2 data. These modules are briefly mentioned in this section and described in detail in the sections that follow.

The input interface module has two input ports, which are connected to the CCD Controller (CCDC) and the SDRAM Controller (SDRC).

The input interface extracts valid raw data from the CCD raw data, and then various modules in the Preview Engine process the raw CCD data.

The histogram function can histogram up to 4 distinct areas into up to 256 bins.

The noise filter module reduces impulse noise in the raw data.

The white balance module applies two gain adjustments to the data: a digital gain and a white balance gain.

The CFA interpolation module implements CFA interpolation, as well as smoothing and a resizing. The output from the CFA interpolation module is always RGB formatted data regardless of the type of color space used by the CCD.

The black adjustment module independently adds an offset value to each RGB component. The offset adjusts the black level of image data.

The RGB2RGB blending module applies a 3x3 matrix transform to the RGB data generated by the CFA interpolation module.

The gamma correction module independently applies gamma correction to each RGB component. Gamma is implemented using a piece-wise linear interpolation approach with a 128 entry look up table for each color.

The RGB2YCbCr conversion module applies 3x3 matrix transformation to the RGB data to convert it to YCbCr data. This module also implements offset, contrast, brightness, and pseudo color suppression.

The Y enhance + 4:2:2 conversion module allows the Luma channel to have programmable non linear sharpening and down samples Cb and Cr, so that the Preview Engine output data is in YCbCr-4:2:2 format.

The output interface module transfers data from the Preview Engine to SDRAM. Data is transferred from the Preview Engine in burst of 16, 32 bit words.

Preview Engine has two different processing paths. Case1: The CCD raw data directly leads to Preview Engine and stores the YUV data to SDRAM. Case2: The Preview Engine reads CCD raw data from SDRAM and stores the YUV data to SDRAM.

Preliminary

4.2.1 Features

- ❑ Real-time hardware processing with complete software control by the ARM
- ❑ Gain control for white balancing at each component
- ❑ Horizontal /Vertical noise reduction filter
- ❑ CFA interpolation for good quality CFA interpolation with reduced false color artifacts
- ❑ Programmable lookup table for Luminance sharpening
- ❑ Programmable coefficients for RGB to YCbCr conversion
- ❑ Separate lookup tables for gamma correction on each of R, G and B components for display
- ❑ Programmable down or up-sampling filter for both horizontal and vertical decimation Range up to x1/64 to x4
- ❑ Programmable Histogram engine (4 windows, 256 bins)
- ❑ Support for RGB Bayer, RGGB, RGBG, tri-color and complementary CCD
- ❑ Inverse A-law table capability

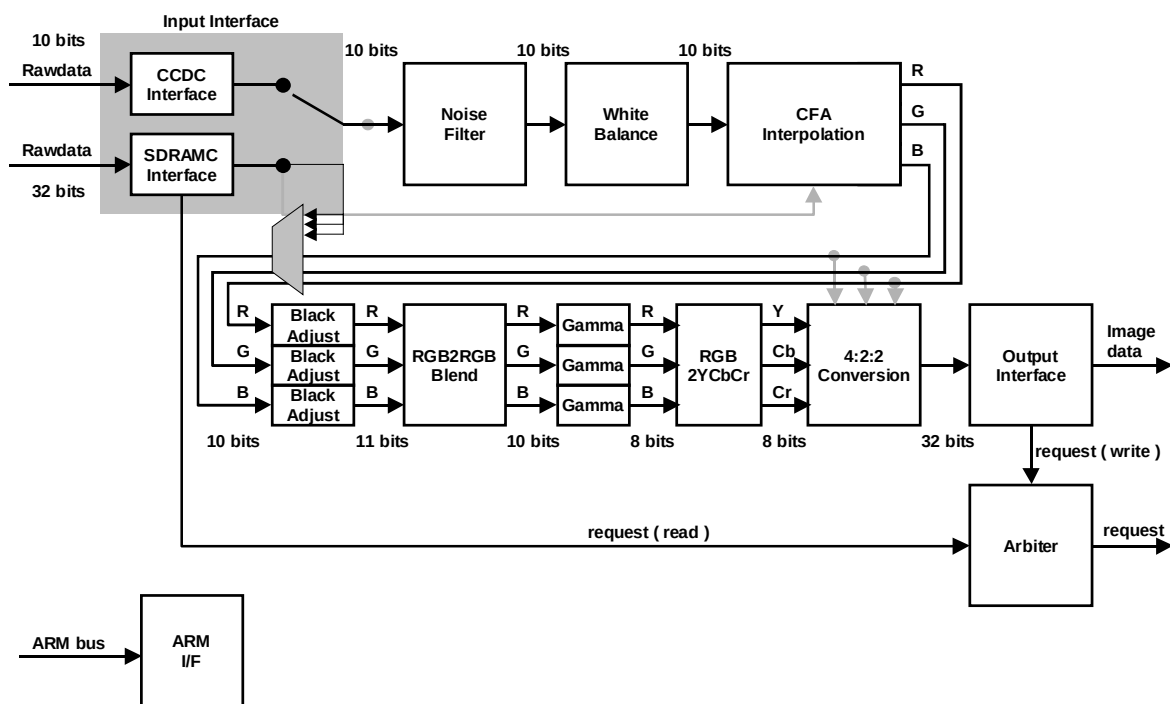


Figure 17 Preview Engine block diagram

4.2.2 CFA Arrangements

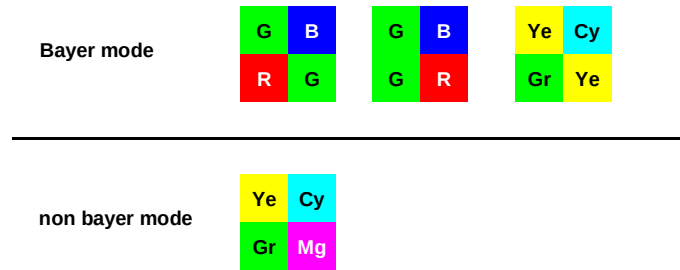


Figure 18 Conventional 2x2 CFA formats

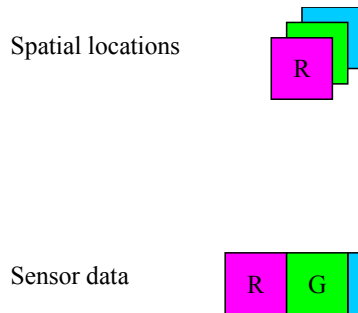


Figure 19 Foveon sensor format

4.2.3 Horizontal noise filter

The noise filter is the first stage of the Preview Engine's image-processing pipeline. The noise filter is a high pass filter that reduces noise in raw CCD data. The filter equation is shown below. In the formula, X_0 , is the current pixel and a is a configurable constant. The noise filter operates on pixels of the same color component, i.e., X_{-HD} , X_0 and X_{HD} . The value of the constant a is defined by the NFRT field in the NFILT register.

$$N' = a \times \frac{X_{-HD} - 2X_0 + X_{HD}}{4} \quad \left(a = \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1\right)$$

If $ABS(N) < Th$ then

$$X_0 = X_0 + N'$$

Note: HD is the horizontal distance between consecutive same color pixels in a line

An on/off switching of this filter can be controlled by register setting.

4.2.4 White Balance

White balance module executes digital gain adjustment and white balance to CCD raw data. Digital gain is for adjusting a total brightness of image and white balance for adjusting a ratio of each color existing in a CFA pattern.

Figure 20 shows a block diagram of this white balance module. There are two multipliers for the two gain adjustment and clip circuits to reduce size of circuits. A digital gain named PVGAIN in this figure is used for gain control of CCD raw data. And WBGAIN is for white balance which is selected automatically by setting register of the CFA pattern.

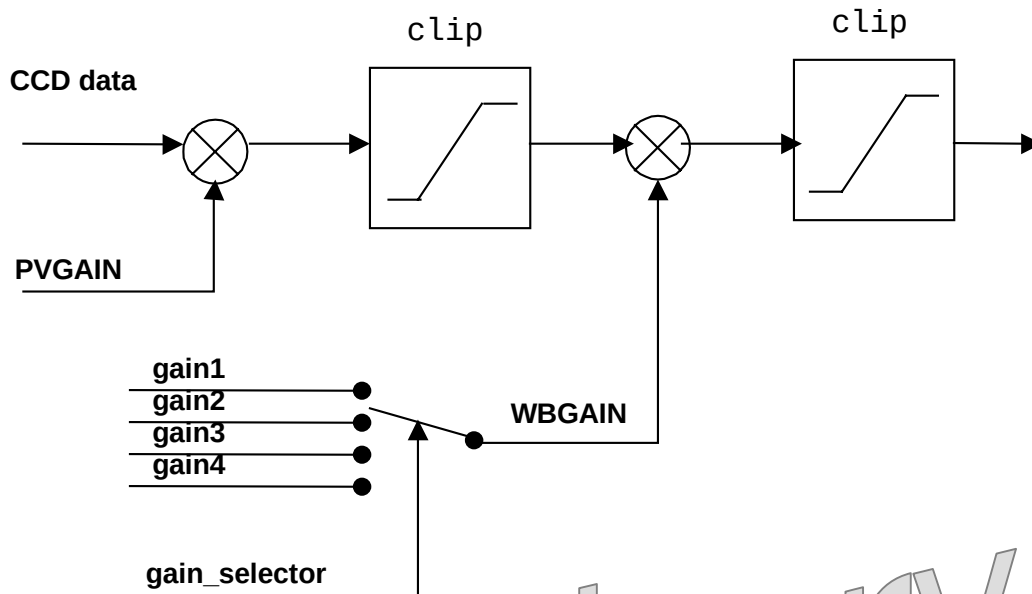


Figure 20 White Balancing in the Preview Engine

4.2.5 CFA Interpolation

The Preview Engine's CFA interpolation module includes many important sub-modules. The sub-modules perform functions such as, horizontal and vertical interpolation, smoothing, horizontal and vertical resizing, color adjustment, and conversion from complementary to RGB. Figure 21 shows a block diagram of the CFA interpolation module. In Figure 17, the data path taken during normal mode is shown by the dark lines, and the data path taken in resize only mode is shown by the light lines.

When the Preview Engine is operating in resize only mode, some of the CFA interpolation modules are bypassed. In resize only mode, only smoothing and horizontal and vertical resizing are applied to the data. The data path taken in resize only mode is shown in shown by the dark lines in Figure 22.

The horizontal interpolation sub-module applies horizontal interpolation using a two tap or a five-tap interpolation filter.

The smoother sub-module applies a low pass filter.

The horizontal resample sub-module applies a seamless horizontal down-sampling or up-sampling (zoom) by x4 over-sampling.

The vertical interpolation and resample sub-module performs a vertical interpolation and a seamless vertical down-sampling or up-sampling by x4 over-sampling using two line memories.

The color selector sub-module extracts the appropriate color from the 4 inputs of the vertical interpolation and resample sub-module.

The color adjustment sub-module executes color adjustment on the interpolated data. This module is bypassed when the Preview Engine is in RGB CCD mode.

The comp2RGB conversion sub-module converts the data from the complementary color space to the RGB color space. The output of the CFA interpolation module RGB data regardless of a type of the color space of used by the CCD imager.

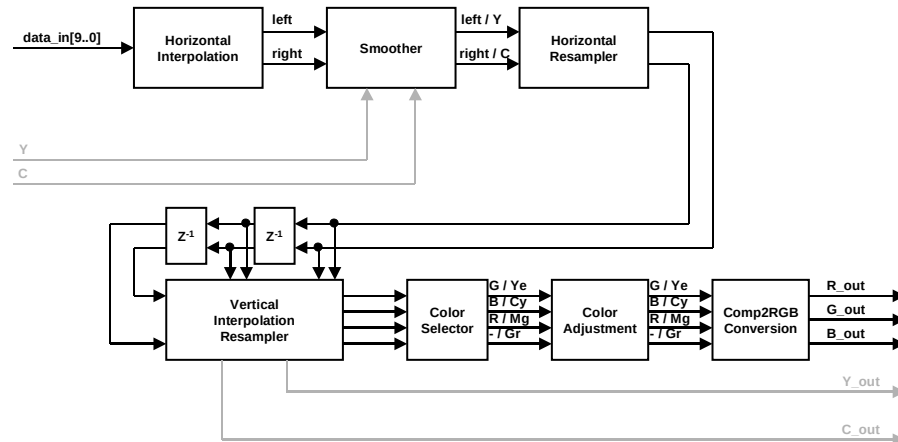


Figure 21 CFA Interpolation Module (Normal Mode)

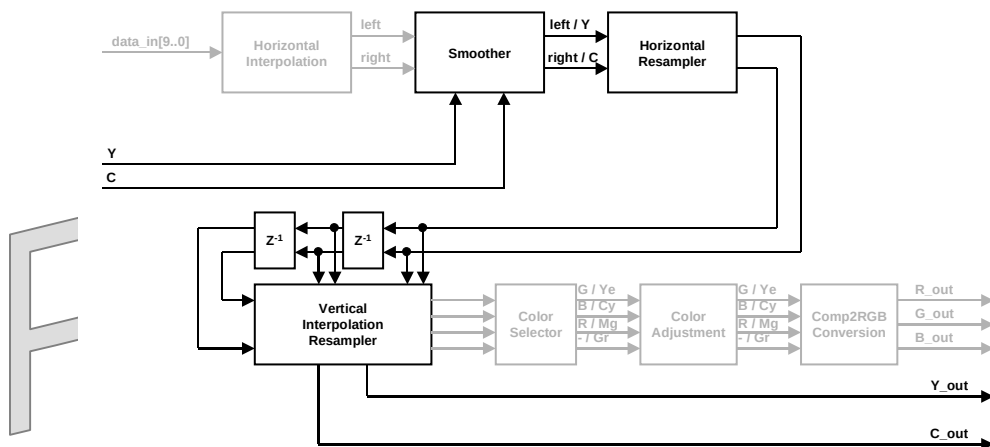


Figure 22 CFA Interpolation Module (Resize Only Mode)

Note: In the case of the Foveon sensor, CFA interpolation is not necessary. Only the horizontal and vertical zoom circuitry in the CFA interpolation module should be active for the Foveon sensor. It is assumed that the Foveon sensor gives our serial RGB data (as illustrated in section 3.7). Therefore, the RGB data needs to be buffered at the output of the CFA interpolation for feeding to the black adjustment module. In other words, all the stages following the CFA interpolation for the Foveon sensor will only operate on one unique RGB data every three pixel clock cycles. To minimize the switching of the gates (and power consumption) the buffer should transmit the same RGB pixel values for 3 consecutive pixel clock cycles. At the final output stage, the 2/3rd repetitive data needs to be thrown away before writing to the SDRAM.

4.2.6 Black Adjustment Sub-Module

After CFA interpolation, black adjustment can be applied to each color component, R, G, and B. A different offset can be applied to each component. The color adjustment sub-module performs the following calculation on each color component.

$$data_out = data_in + bl_offset$$

4.2.7 RGB2RGB Blending Module

The RGB2RGB blending module transforms the RGB data generated by the CFA interpolation module using a 3x3 square matrix transformation in combination with an added offset. The RGB2RGB blending module computes the following calculation.

$$\begin{bmatrix} R_{out} \\ G_{out} \\ B_{out} \end{bmatrix} = \begin{bmatrix} MTX_RR & MTX_GR & MTX_BR \\ MTX_RG & MTX_GG & MTX_BG \\ MTX_RB & MTX_GB & MTX_BB \end{bmatrix} \begin{bmatrix} R_{in} \\ G_{in} \\ B_{in} \end{bmatrix} + \begin{bmatrix} R_offset \\ G_offset \\ B_offset \end{bmatrix}$$

4.2.8 Gamma Correction Module

The gamma correction module performs a gamma correction independently for each color in the RGB color space by using a piece-wise linear interpolation. Each table has 128-entries, and each entry accommodates a 10-bit gain and 10-bit offset. As shown in Figure 1, this module exists independently for each color so that the independent setting is possible.

Figure 4.6 shows a block diagram of the gamma correction module. It is composed of one table and one selector. When the BYPASS bit is asserted, the input data is divided by four.

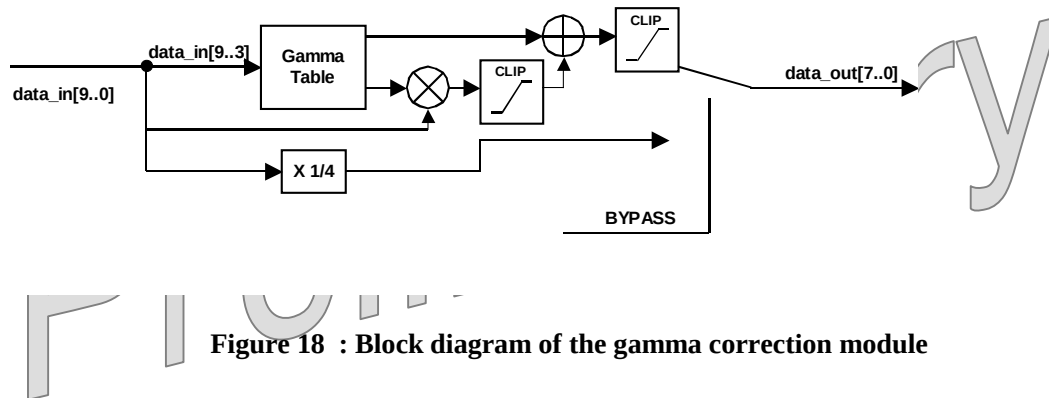


Figure 18 : Block diagram of the gamma correction module

4.2.9 RGB2YCbCr Conversion Matrix

The RGB to YCbCr conversion module converts RGB formatted data to YCbCr formatted data and adjusts Cb and Cr with offset values. The following matrix calculation is used to calculate Y, Cb, and Cr.

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} COEF1 & COEF2 & COEF3 \\ COEF4 & COEF5 & COEF6 \\ COEF7 & COEF8 & COEF9 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} OFFSET_Y \\ OFFSET_Cb \\ OFFSET_Cr \end{bmatrix}$$

In addition to the color space operation, contrast (scaling/multiplication) and brightness (offset/addition) adjustments can be made on the luminance data and pseudo-color suppression can be performed for chrominance data.

The gain factor applied to Cb and Cr during pseudo-color suppression is determined using a configurable function mapping. The functional mapping is shown in Figure 23. The pseudo-color suppression threshold and gain can be set in the CSUP1 register. Use SUPTH to set the threshold constant and use SUPG to set the gain constant.

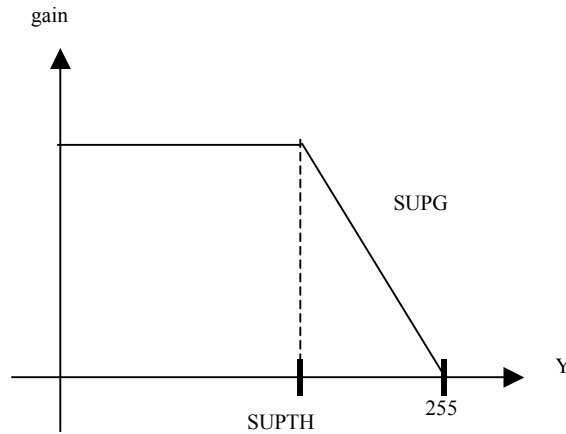


Figure 23 Pseudo-color Suppression Functional Mapping

In addition to the above operations, a non-linear enhancement on the luminance component (Y data) is necessary. Figure 26 shows the operation of the non-linear enhancer.

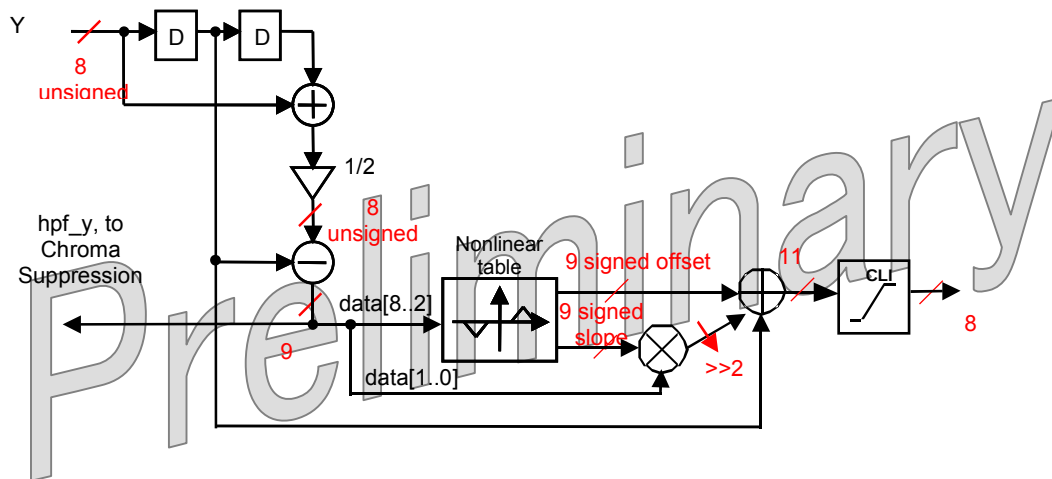


Figure 24 Block diagram of the Luminance non-linear enhancer

4.2.10 Histogram

Histogram function supports the following

- Up to 4 regions (areas)
- Each region has separate on/off control
- Each region has its own start coordinate X/Y (12-bit) and horizontal/vertical sizes (12-bit)
- When the regions overlap, only one region is operated on (selected bin incremented)

- CFA data assumed (RGr lines and GbB lines interleaved). Foveon sensor NOT supported, and data for each color go into separate set of bins
- Bins are counters, counting number of values being in the range associated with the bins
- Per color, per region, there can be 32, 64, 128, or 256 bins
- Data values are first down-shifted then saturated for bin selection
- 1024 x 20-bit memory used

4.2.11 4:2:2 Conversion and Output Interface Modules

The 4:2:2 conversion module converts the image data to YCbCr-4:2:2 format by taking the average of every two Cb and Cr components. After converting the data to YCbCr-4:2:2, the image data is transferred to the SDRC in units of 32 bytes. The SDRC manages the transfer of the image data to SDRAM. The Preview Engine has an internal 16-bit by 16 FIFO that is used buffer data. Data is transferred from the FIFO to the SDRC in, 32-byte bursts. Table 3 shows how the YCbCr image data is formatted in SDRAM.

Table 5 YCbCr-4:2:2 Format

MSB				LSB			
31	24	23	16	15	8	7	0
Y1				Cb0			

4.2.12 4:2:2 SDRAM Input mode

The preview engine has two selectable input ports which are connected to the CCD controller (CCDC) and the SDRAM controller (SDRAMC). There is some difference of available functions between in the CCDC input mode and in the SDRAMC input mode. Table 6 shows the main difference of available functions between the two input modes.

Table 6 Comparison of the two input mode

Functions	CCDC input mode		SDRAMC input mode	
	Normal	Resize Only	Normal	Resize Only
Noise filter	Y	N	Y	N
Digital gain	Y	N	Y	N
White balance	Y	N	Y	N
Smoothing	Y	Y	Y	Y
Down sample	Y	Y	Y	Y
Zoom	N	N	Y	Y
RGB blend	Y	N	Y	N
Gamma	Y	N	Y	N
Chroma suppress	Y	N	Y	N
Chroma offset	Y	N	Y	N
1 shot preview	Y	Y	Y	Y
Histogram - Bayer	Y	-	Y	-

Table 7 shows SDRAM data format which preview engine can perform. In 8bit pack mode, the preview engine also has an inverse A-law table function. It can decompress the 8bit data which is compressed by A-law table in CCD controller. The 8bit input data is extended to 10bit data. In this table Pixel0 data is processed ahead of Pixel1 data.

Table 7 SDRAM Data Format

10bit 8bit pack YCbCr	Upper word				Lower word			
	MSB(31)		LSB(16)		MSB(15)		LSB(0)	
	0	Pixel1			0	Pixel0		
	Pixel3		Pixel2		Pixel1		Pixel0	
	Pixel1(Y1)		Pixel0(Cr0)		Pixel0(Y0)		Pixel0(Cb0)	

4.3 AF/AE/AWB Statistics Engine

DM320 provides real-time 3A (AE/AWB/AF) statistics function. 3A statistics function can be performed either before gamma or after gamma conversion. When 3A statistics calculation is completed, an interrupt will be generated to inform ARM of its completion. ARM then uses the statistics to compute the optimal 3A control parameters.

AE/AWB statistics is calculated using 96 windows. The windows are the same size and organized as an m-by-n matrix. The organization of the matrix, the size of the windows and the method of calculation are user-selectable.

AF statistics is calculated by up to 96 windows. The position and the size of each window can be independently set by software.

4.3.1 Features

1. AE, AWB
 - a. Total number of windows is 96.
 - b. The base position for AE/AWB statistics window matrix can be configured by software
 - c. Number of windows in horizontal/vertical direction is from 1 to 12(H), 8(V)
 - d. Total number of pixels in horizontal/vertical direction is from 2 to 256(H), 64(V)
 - e. Sampling step for calculation in horizontal/vertical direction is from 2 to 32. Total number of 2x2 blocks in a window is up to 256.
 - f. Outputs for each window are the accumulated R, Gr, Gb and B. The results are 16-bit integer stored in SDRAM-readable by ARM
2. AF
 - a. Total number of windows is up to 96
 - b. The calculation can be performed on Green pixels only data in the window.
 - c. The position, size of each window can be independently set by software.
 - d. Two biquad filters are used to filter the green or Y pixel data... The coefficients of both filters are 10-bit wide and can be set by software.

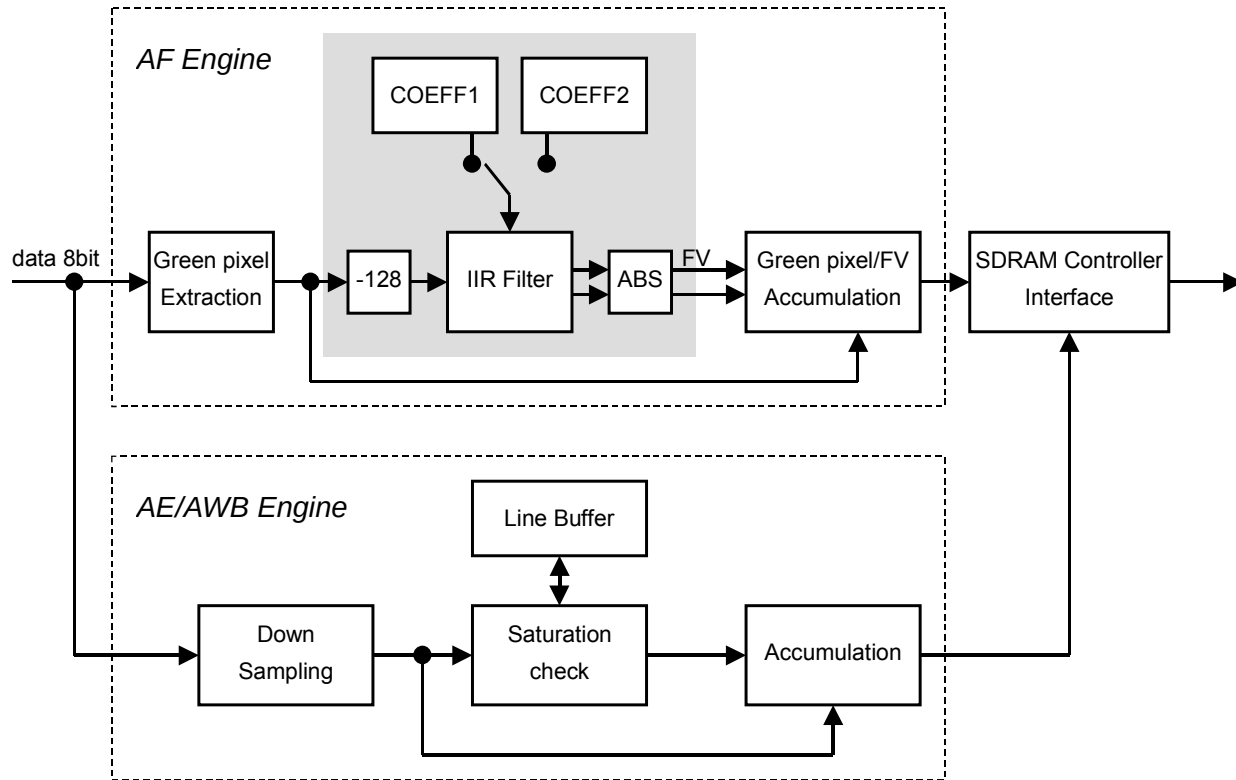


Figure 25 Block Diagram of 3A Hardware Engine

AF Engine

- Green pixel accumulation for each Poxel.
- Focus Value (FV) accumulation for each Poxel.
- [Peak mode]** Accumulate the maximum FV of each line in a Poxel.
Accumulate the peak position of each line in a Poxel.
- [Sum mode]** Accumulate FV in a Poxel.

AE/AWB Engine

- Accumulate the pixels in the sub-sampled 2x2 blocks for each Window (AVE1).
- Accumulate the pixels which are clipped to the limit value in the sub-sampled 2x2 blocks (AVE2).
- Count the number of the sub-sampled 2x2 blocks which are not saturating (CNT2).

4.3.2 AF Engine

Extracted green data are subtracted by 128, and fed to the filter block to calculate the Focus Value, which is the absolute value of the IIR filter output. The maximum FV of each line in a Poxel is acquired if FV mode is set to 'Peak mode'. Value of the green pixels and either the FV or the maximum FV are accumulated in the Poxel, and are stored in SDRAM.

The window configuration of the AF Engine is based on a frame formed by the extracted green pixels.

Table 8 AF Engine Configuration Registers

Register	width	Description
AFEN	1	AF Engine enable 0:disable 1:enable

GPOS	4	Green pixel position in 2x2 block Support 5/6/9/A and 1/2/4/8(added).
PAXW	8	Paxel Width (PAXW+1) (1 – 256)
PAXH	6	Paxel Height (PAXH+1) (2 – 128)
PAXHC	4	Paxel Count for H direction (PAXHC+1) (1 – 12)
PAXVC	3	Paxel Count for V direction (PAXVC+1) (1 – 8)
PAXSH	12	Paxel start position (H) (1 – 4,095)
PAXSV	12	Paxel start position (V) (0 – 4,095)
IIRSH	12	IIR filter start position (H) (0 – 4,095)
AFINCV	4	Line increments in a Paxel (AFINCV+1) (1-16)
FVMODE	1	Focus Value accumulation mode 0:Sum mode 1:Peak mode
COEFF1	11x11	Coefficient table1 for IIR filter (decimal 6bit)
COEFF2	11x11	Coefficient table2 for IIR filter (decimal 6bit)
AFSDRA	23	SDRAM destination address to store the results of AF engine. It is automatically incremented until the two continuous buffers for the two successive frames become full.
AFSDRFLG	1	SDRAM buffer flag to indicate the buffer at which the results are stored.
AFERRFR	2	SDRAM access fail flag on previous buffer flag [0/1].
AFERR	1	SDRAM access fail flag. Clear by ARM write only.

- PAXSH must be equal to or greater than (IIRSH + 1).
- Paxels cannot overlap the last pixel in a line. PAXSH, PAXW and PAXHC must be set accordingly.
- Paxels can overlap either the first or the last line in a frame, but not both of them. PAXSV, PAXH and PAXVC must be set accordingly.
- GPOS can not pick continuous 2 data in a line . DM320 support 5/6/9/A(dm270) and 1/2/4/8 value.
- PAXH=0 is NG. PAXW=0 might be problem in data transfer to SDRAM.

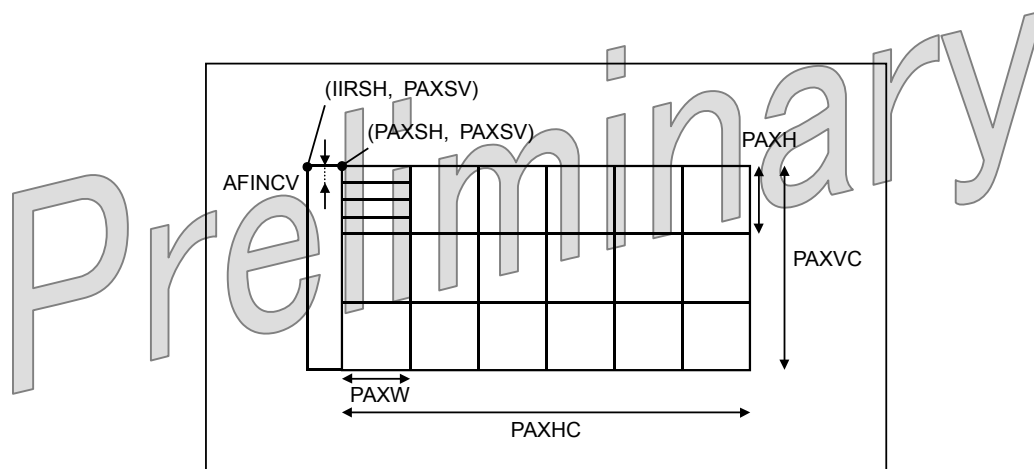


Figure 26 Paxel Configuration

Green Extraction

Ex:Bayer	R	Gr
	Gb	B

Ex: GPOS=6	←	Gr0	←	Gr1	←	Gr2	..	AF Input	Gr0	Gr1	Gr2	..
	Gb0	←	Gb1	←	Gb2	←	..		Gb0	Gb1	Gb2	..

Ex: GPOS=2	←	Gr0	←	Gr1	←	Gr2	..	AF Input	Gr0	Gr1	Gr2	..
	←	←	←	←	←	←	..		←	←	←	..

In the case of GPOS=1/2/4/8, there are non-extracted H-Lines(Just previous data keep). So Do not use these line for AF calculation. You can set "AFINCV" value to skip line.

Then [GPOS=2/ AFINCV=1] will get same output with [GPOS=6/ AFINCV=1] setting.

Also [GPOS=4/ AFINCV=1/ PAXSV(+1)] will get same output with [GPOS=6/ AFINCV=1/PAXSV(+1)].

This case need calculation start point adjustment , cause AF need actual data in calculation start-point.

IIR Filter

There are 2 sets of programmable coefficient tables. FV calculation is performed on both tables. Shift registers of the IIR filter are cleared to zero once in a line, at the next position of IIRSH.

Filter Input	[Value of a green pixel (8bit)] - 128	$-128 \leq \text{filter_in} \leq 127$
Coefficient	Signed 11bit (decimal 6 bit)	$-16 \leq \text{coeff} \leq 15+63/64$
Filter Output	signed 12bit (decimal 4 bit)	$-128 \leq \text{filter_out} \leq 127+15/16$

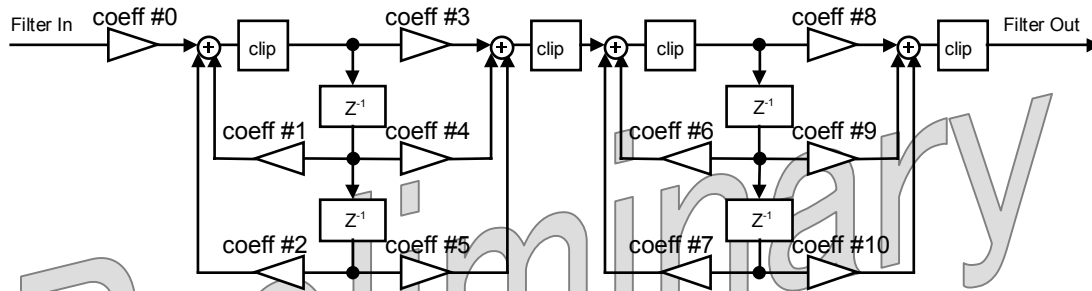


Figure 27 IIR Filter

AF Engine Output

1. Green Sum: 32bit(22bit)
2. FV sum: 32bit(26bit) (decimal 4bit) x 2

Data transfer to SDRAM

Results of the 2 adjacent Paxels in a row are gathered and transferred to the SDRAM at the same time, therefore, if PAXHC is set to 8 and PAXVC is set to 3, there will be 20 sets (= 5 x 4) of data transfer within an image. The results of the first Paxel in an image are stored to the address specified in the register AFSDRA. The address is automatically incremented until the two continuous buffers for the two successive frames become full. SDRAM buffer flag (AFSDRFLG) indicates the buffer at which the results are stored.

Example of data transfer

- Peak mode
- (PAXHC, PAXVC) = (2,1)

First Paxel

0	1	2
3	4	5

SDRAM address (byte address)	31	16	15	0	
AFSDRA	Green sum			(Paxel 0)	1 st transfer start
	FV sum -1			(Paxel 0)	
	FV sum -2			(Paxel 0)	
	reserved				
	Green sum			(Paxel 1)	1 st transfer end
	FV sum -1			(Paxel 1)	
	FV sum -2			(Paxel 1)	
	reserved				
AFSDRA + 32	Green sum			(Paxel 2)	2 nd transfer start
	FV sum -1			(Paxel 2)	
	FV sum -2			(Paxel 2)	
	reserved				
	Green sum			(Paxel 1)	2 nd transfer end
	FV sum -1			(Paxel 1)	
	FV sum -2			(Paxel 1)	
	reserved				
AFSDRA + 64	Green sum			(Paxel 3)	3 rd transfer start
	FV sum -1			(Paxel 3)	
	FV sum -2			(Paxel 3)	
	reserved				
	Green sum			(Paxel 4)	3 rd transfer end
	FV sum -1			(Paxel 4)	
	FV sum -2			(Paxel 4)	
	reserved				
AFSDRA + 96	Green sum			(Paxel 5)	4 th transfer start
	FV sum -1			(Paxel 5)	
	FV sum -2			(Paxel 5)	
	reserved				
	Green sum			(Paxel 4)	4 th transfer end
	FV sum -1			(Paxel 4)	
	FV sum -2			(Paxel 4)	
	reserved				

4.3.3 AE/AWB Engine

The pixels in the sub-sampled 2x2 blocks are accumulated in the window as AVE1. If any of the pixels in a 2x2 block are equal to or greater than the limit value(AVE2LMT), the block is treated as a saturated block. Only the blocks which are not saturating are counted into CNT2. The pixels which are clipped to AVE2LMT in the sub-sampled 2x2 blocks are accumulated in the window as AVE2, and are stored in SDRAM with AVE1.

Table 9 Configuration Registers

Register	width	Description
AEWEN	1	AE/AWB Engine enable 0:disable 1:enable
WINW	7	Window Width (even number) ((WINW +1) x2) (2 – 256)
WINH	6	Window Height (even number) ((WINH +1) x2) (4 – 128)

WINHC	4	Window Count for H direction (WINHC+1) (1 – 12)
WINVC	3	Window Count for V direction (WINVC+1) (1 – 8)
WINSH	11	X position of the Window start point (WINSH x2) (even number)
WINSV	11	Y position of the Window start point (WINSV x2) (even number)
AEWINCH	4	Sampling point increments (X) in a Window (even number) ((AEWINCH+1) x2) (2-32)
AEWINCV	4	Sampling point increments (Y) in a Window (even number) ((AEWINCV+1) x2) (2-32)
AVE2LMT	8	AVE2 limit value
AEWSDBA	23	SDRAM destination address to store the results of AE/AWB engine. It is automatically incremented until the two continuous buffers for the two successive frames become full.
AEWSDBFLG	1	SDRAM buffer flag to indicate the buffer at which the results are stored.
AEWERRFR	2	SDRAM access fail flag on previous buffer flag [0/1].
AEWERR	1	SDRAM access fail flag. Clear by ARM write only.

- The number of accumulated 2x2 blocks in a Window cannot exceed 256. WINW, WINH, AEWINCH and AEWINCV must be set accordingly.
- The number of sub-sampled pixels in a line cannot exceed 2816. WINW, WINHC and AEWINCH must be set accordingly. (Used 1408x16 line memory.)
- Windows can overlap either the first or the last line in a frame, but not both of them. WINSV, WINH and WINVC must be set accordingly.
- WINH=0 is NG. WINW=0(or small value) might be problem in data transfer to SDRAM.

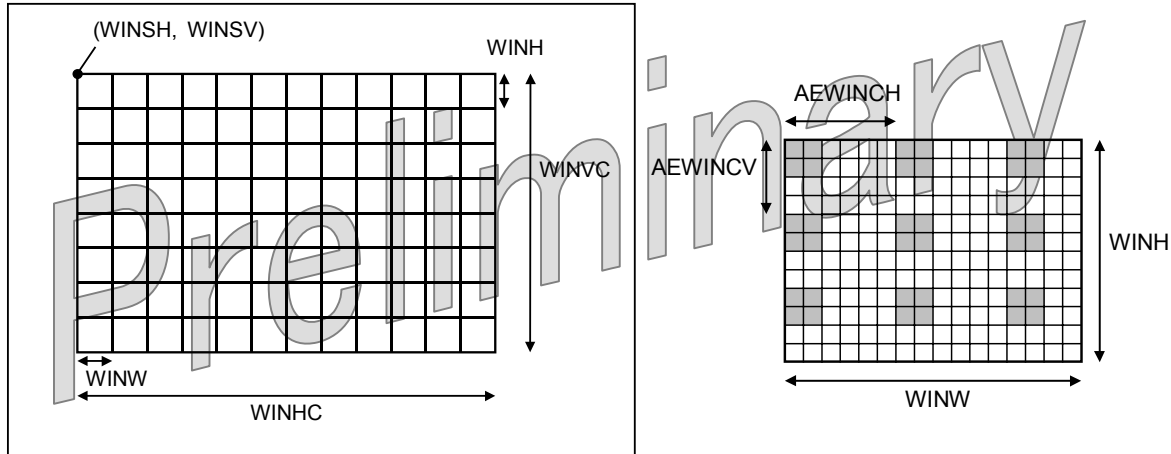


Figure 28 Window configuration

AE/AWB Engine Output

1. AVE1[i][j] – 16bit (Accumulated sub-sampled data clipped to 65,535)
2. AVE2[i][j] – 16bit (Accumulated sub-sampled data which are not saturated clipped to 65,535)
3. CNT2[i] – 16bit(9bit) (Number of sub-sampled data which are not saturated)
 - i: window number
 - j: pixel position in 2x2 blocks (0: upper left, 1: upper right, 2: lower left, 3: lower right)

Data transfer to SDRAM

Results of each window are transferred to the SDRAM at the same time, therefore, if WINHC is set to 8 and WINVC is set to 3, there will be 36 sets (= 9 x 4) of data transfer within an image. The results of the first Window in an

image are stored to the address specified in the register AEWSDBA. The address is automatically incremented until the two continuous buffers for the two successive frames become full. SDRAM buffer flag (AEWSDBFLG) indicates the buffer at which the results are stored.

In special case (small WINW number/ SDRAM access waiting/ ccd-sdr clock), Data transfer might fail. DM320 report fail-condition of SDRAM-transfer to error flag register. Once error happened, AEWERR register keep '1' until clear by ARM write. AEWERRFR registers update error information on each v-sync timing.

Example of data transfer

- (WINHC, WINVC) = (1,1)

First Window

0	1
2	3

SDRAM address (byte address)		31	16	15	0	
AEWSDRA		AVE1[0][1]		AVE1[0][0]		1 st transfer start
		AVE1[0][3]		AVE1[0][2]		
		AVE2[0][1]		AVE2[0][0]		
		AVE2[0][3]		AVE2[0][2]		
		All zero		CNT2[0]		
		All zero				
		All zero				
		All zero				1 st transfer end
AEWSDRA + 32		AVE1[1][1]		AVE1[1][0]		2 nd transfer start
		AVE1[1][3]		AVE1[1][2]		
		AVE2[1][1]		AVE2[1][0]		
		AVE2[1][3]		AVE2[1][2]		
		All zero		CNT2[1]		
		All zero				
		All zero				
		All zero				2 nd transfer end
AEWSDRA + 64		AVE1[2][1]		AVE1[2][0]		3 rd transfer start
		AVE1[2][3]		AVE1[2][2]		
		AVE2[2][1]		AVE2[2][0]		
		AVE2[2][3]		AVE2[2][2]		
		All zero		CNT2[2]		
		All zero				
		All zero				
		All zero				3 rd transfer end
AEWSDRA + 96		AVE1[3][1]		AVE1[3][0]		4 th transfer start
		AVE1[3][3]		AVE1[3][2]		
		AVE2[3][1]		AVE2[3][0]		
		AVE2[3][3]		AVE2[3][2]		
		All zero		CNT2[3]		
		All zero				
		All zero				
		All zero				4 th transfer end

4.4 On Screen Display (OSD) and Graphics Acceleration

The OSD graphic accelerator is responsible for managing OSD data from different OSD windows and blending it with the video. It reads OSD data from SDRAM, and outputs to the on-chip PAL/NTSC Encoder. The OSD module defaults to standby mode, in which it simply sends video to the PAL/NTSC Encoder. After being configured and activated by the ARM CPU, the OSD reads OSD data and mixes it with the video output. The ARM is responsible for turning on and off OSD operations and writing to the OSD data to the SDRAM.

4.4.1 Features

- ❑ Can be configured for two video display windows and two OSD display windows, or can be configured for two video display windows, 1 OSD display window, and 1 attribute window
- ❑ Configurable rectangular hardware cursor can be displayed, its size can be specified, and its color and blending can be specified by a look-up table.
- ❑ Attribute window to provide pixel level blinking and blending
- ❑ The four windows and the one cursor can be displayed simultaneously
- ❑ The background color can be specified
- ❑ Video-Window0 and Video-Window1 are use YCbCr data, and OSD-Window0, OSD-Window1
- ❑ Supports 1,2,4,8 bit wide bitmap data
- ❑ Five blending ratios for the OSD-window can be selected
- ❑ Transparency for the OSD-windows
- ❑ Supports RGB16 or YCrCb 4:2:2 input for OSD-windows
- ❑ Default bit map look-up table in ROM available
- ❑ Programmable bit map look-up table in RAM available (24 bits x 256 words)
- ❑ Interlaced and non-interlaced modes supported
- ❑ Zoom horizontal and vertical with factor x2 and x4 available
- ❑ VGA->NTSC/PAL conversion circuit built in
- ❑ Square -> NTSC conversion for OSD data

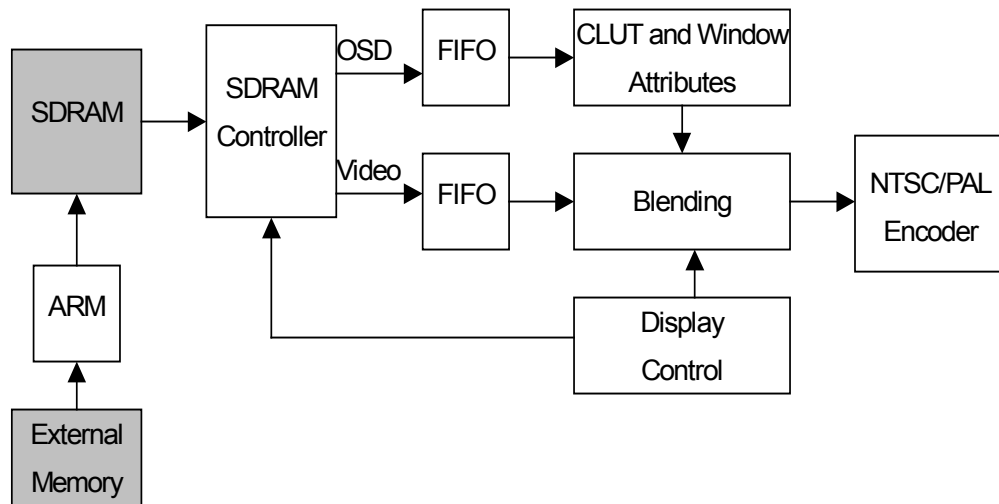


Figure 29 OSD block diagram

4.4.2 OSD data storage

The OSD data has variable size. In the bitmap window, each pixel can be 1, 2, 4, or 8 bits wide. In the YCbCr 4:2:2 window, it takes 8-bit per components, and the components are arranged according to 4:2:2 (Cb/Y/Cr/Y ...) format. In the case where RGB graphics data needs to be used as OSD, the application should perform software conversion to Y/Cr/Cb before storing it. The OSD data is always packed into 32-bit words and left justified. Starting from the upper left corner of the OSD window, all data will be packed into adjacent 32-bit words.

4.4.3 Setting Up an OSD Window

An OSD window is defined by its attributes. Besides storing OSD data for a window into SDRAM by ARM CPU, the application program also needs to update window attributes and other setup in the OSD module as described in the following subsections.

4.4.4 Location Register

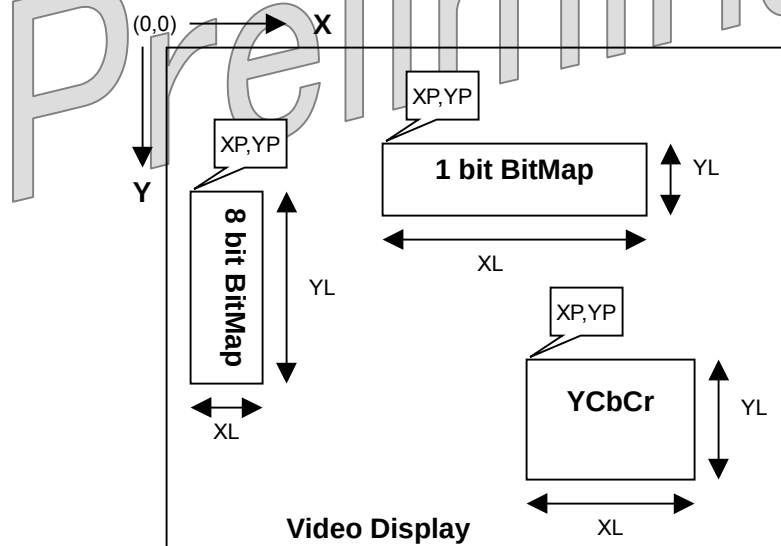


Figure 30 OSD Window Example

The Location register contains X and Y axis locations of the upper left and lower right corners of each window. The application program needs to set up the position of left-upper corner, the length of X and Y axis direction and enable selected OSD windows.

4.4.5 Color Look Up Table

The OSD has the fixed and programmable 256 entry color look up table (CLUT). The CLUT is used to convert bitmap data into Y/Cr/Cb components. In the case of 1, 2 or 4 bitmap pixels, the CLUT can be determined by CLUT registers.

4.4.6 Blending and Transparency

Color blending on the pixel level is also supported. This feature is available for the bitmap displays and hardware cursor only (Window1, 2). If the window color blending is enabled, the amount of blending of each pixel is determined by the blending factor. As shown in Table 10 Blending and transparency in OSD, the window blending supports 8 different levels, according to the selected blending factor.

The hardware also supports a transparency mode with bitmap. If transparency is enabled, then any pixel on the bitmap display that has a value of 0 will allow video to be displayed. Essentially, 0 valued pixels are considered the transparent color, i.e. the background color will show through the bitmap. Table 10 shows the connection between transparency and blending on the same window.

Table 10 Blending and transparency in OSD

Transpare ncy	Blend Factor	OSD window contribution	Video contribution
OFF	0	0	1
	1	1/8	7/8
	2	2/8	6/8
	3	3/8	5/8
	4	4/8	4/8
	5	5/8	3/8
	6	6/8	2/8
	7	1	0
ON	0	if pixel value = 0 0	if pixel value = 0 1
	1	1/8	7/8
	2	2/8	6/8
	3	3/8	5/8
	4	4/8	4/8
	5	5/8	3/8
	6	6/8	2/8
	7	1	0

4.4.7 Hardware Cursor

A rectangular shape is provided using video window1. The cursor always appears on top of other OSD Windows. The user can specify the position (XP, YP), size (XL, YL) or color of the shape. When video window1 is designated as the cursor, only three windows are available for the OSD application. If a hardware cursor is not used, then the application can use video window1 as a regular hardware window. Figure 31 shows an example usage of the hardware cursor.

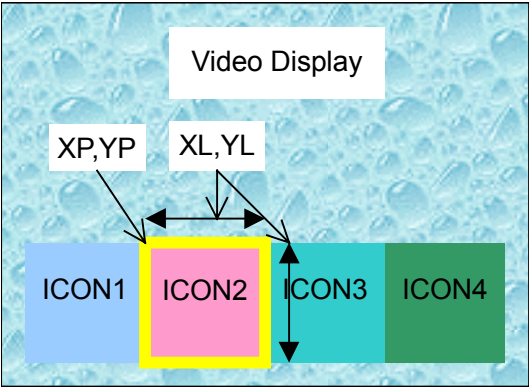


Figure 31 Hardware Cursor in OSD

4.4.8 Window Magnification

The OSD graphic accelerator has a function of magnification horizontally and vertically respectively. Each magnification ratio is bellow.

Table 11 Magnification Ratio

	Horizontal	Vertical
Video Window	$x1 / x2 / x4 / x9/8$	$x1 / x2 / x4 \ x6/5$
Bitmap Window	$x1 / x2 / x4/ x9/8$	$x1 / x2 / x4 \ x6/5$

Preliminary

4.5 NTCS/PAL Encoder

The Video Encoder is used to display images and video on TV monitor and various LCD displays. The Video Encoder converts digital YCbCr signals to analog and digital video signals. The Video Encoder can generate composite NTSC or PAL signals, and 8-bit digital LCD interface signals. To generate these signals, the Video Encoder has digital to analog converter, a digital LCD interface, and a set of signals to perform timing and synchronization. The Video Encoder is shown in Figure 32.

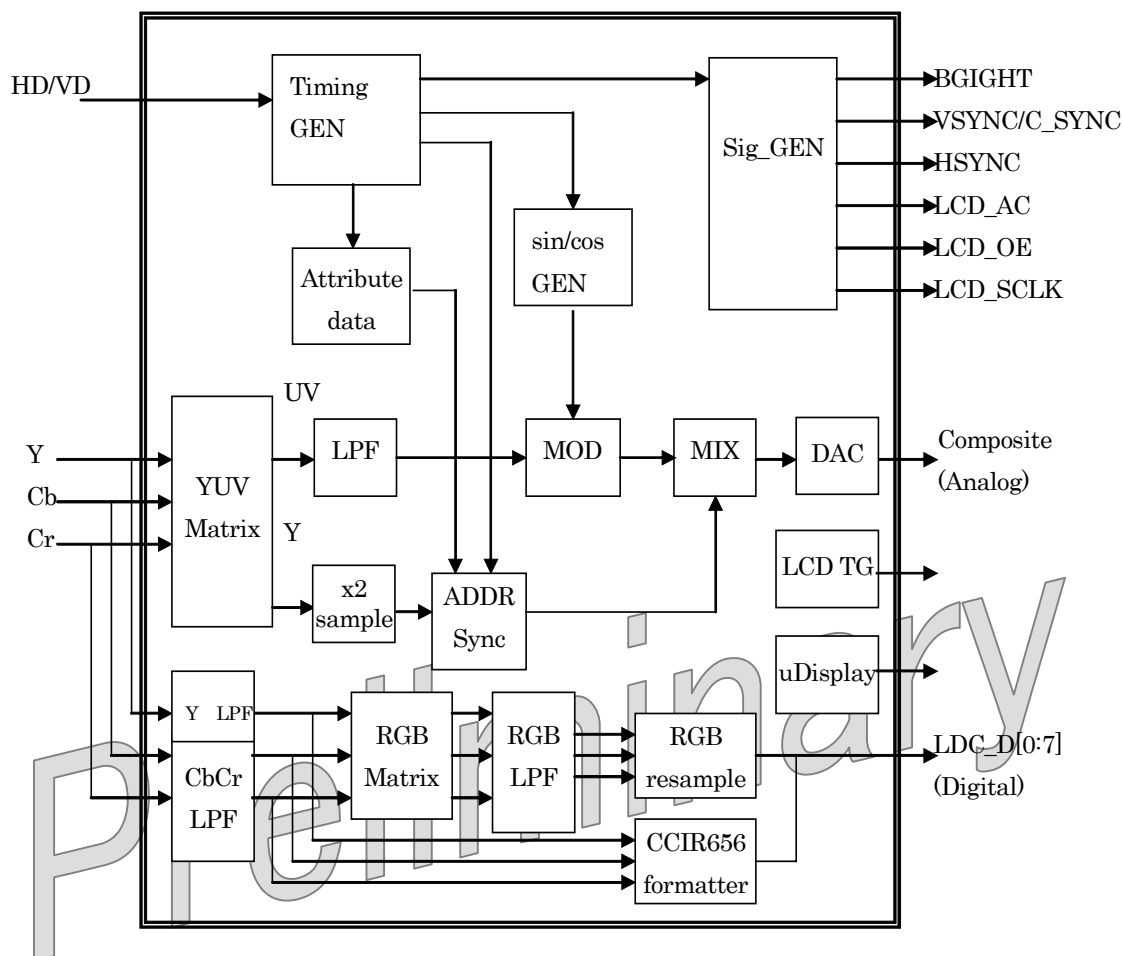


Figure 32 NTSC/PAL Encoder Block Diagram

4.5.1 Composite analog video features

- ❑ Master Clock Input single 27MHz
- ❑ 10-bit D/A Output Interface with output control
- ❑ NTSC/PAL Output Selectable
- ❑ Line 20 Video Attribute Insertion
- ❑ Interlace/Non-Interlace selected output
- ❑ Setup Selection for NTSC
- ❑ VSYNC/ HSYNC Output
- ❑ VSYNC/Composite SYNC Output Selectable
- ❑ 16-235/0-255 Input Amplitude Selectable
- ❑ Video Output Blanking
- ❑ Macro-Vision Support

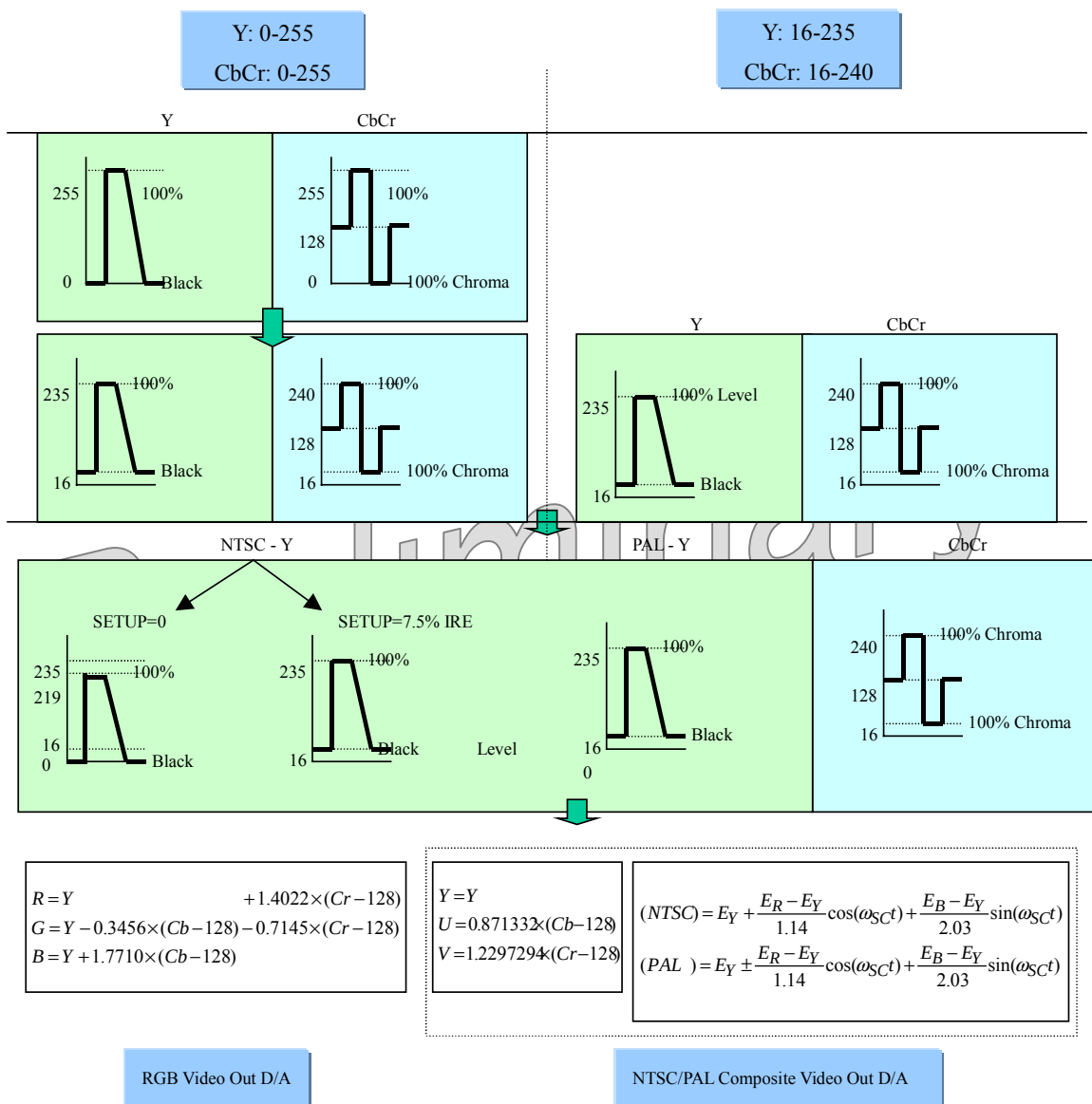
4.5.2 Digital video features

- ❑ Programmable DCLK out for Digital-RGB Output
- ❑ YCbCr or RGB digital output selectable
- ❑ 8-bit Digital RGB Output for Digital LCD
- ❑ 8-bit Digital Color Sequential Output for Micro Display
- ❑ 16-bit Digital RGB for embedded memory displays
- ❑ Chrominance Low Pass Filter 1.5MHz/3MHz
- ❑ Internal Color Bar Generation (100/75%)
- ❑ Digital Low Pass Filter for RGB Output
- ❑ Digital VSYNC/HSUNC output
- ❑ Programmable HSYNC/VSYNC Width
- ❑ Programmable HSYNC/VSYNC Interval
- ❑ Programmable VSYNC/HSYNC Interval
- ❑ VSYNC Output Delay from Input VSYNC at DCLK
- ❑ HSYNC Output Delay from Input HSYNC at DCLK
- ❑ Programmable RGB Output Format of ODD/ Even Line (Digital RGB)
- ❑ Programmable Picture Start Position Horizontal (Digital RGB Output)
- ❑ Programmable Picture Start Position Vertical (Digital RGB Output)

- ❑ Programmable Picture Valid Length Horizontal (Digital RGB Output)
- ❑ Programmable Picture Valid Length Vertical (Digital RGB Output)
- ❑ Back Light Control Signal Output
- ❑ Color Sequential support for Display Tech MicroDisplay

4.5.3 Signal Level

The NTSC/PAL Encoder accepts digital input signals in YCbCr format. The NTSC/PAL Encoder has a selection register determines the manner in which input signal levels are processed. When register value is 1, a value of 16 is processed as black level, and a value of 235 is processed as 100% white level. For chroma signal also, a value of 16 and a value of 240 is processed as 100% chroma level.



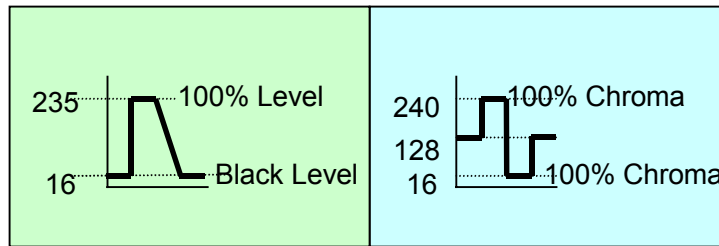


Figure 33 Input Video Data

4.5.4 Signal calculation

Each RGB or Composite video output is transformed by using following equations.

[RGB output]

$$R = Y + 1.4022 \times (Cr - 128)$$

$$G = Y - 0.3456 \times (Cb - 128) - 0.7145(Cr - 128)$$

$$B = Y + 1.7710 \times (Cb - 128)$$

[Y/C, Composite output]

$$Y = Y$$

$$U = 0.871332 \times (Cb - 128)$$

$$V = 1.2297294 \times (Cr - 128)$$

$$NTSC = E_Y + \frac{E_R - E_Y}{1.14} \cos(\omega_{sc}t) + \frac{E_B - E_Y}{2.03} \sin(\omega_{sc}t)$$

$$PAL = E_Y + \frac{E_B - E_Y}{2.03} \cos(\omega_{sc}t) + \frac{E_R - E_Y}{2.03} \sin(\omega_{sc}t)$$

4.5.5 Color signal cutoff frequency select

When output is Y/C or Composite, the chroma signal is restricted to the frequency band by the low-pass filter. On the NTSC/PAL encoder, cutoff frequency is selected using the ARM register. When 0, cutoff frequency is 1.5MHz; When 1, cutoff frequency is 3MHz.

4.5.6 Video attribute insertion

The NTSC/PAL encoder has capability to insert video information into the vertical blanking period. For example, the NTSC/PAL encoder can insert a video attribute which indicates the proper aspect ratio to the video receiver. For NTSC mode, the NTSC/PAL encoder can insert 14-bit video information on line 20 of every field to conform to the EIAJ CPX-1024 Video Aspect Ratio ID specification. Attribute information should be set using ATR1 and ATR0 registers. The ATR2 register should be set with the 6bit CRC data that is calculated by the following equation:

$$G(X) = X^6 + X + 1 \text{ where } X^6, X \text{ are preset to } 1$$

For PAL mode, the NTSC/PAL encoder can insert 14-bit video information on line 20 of every frame to conform to the ETS300-294 Wide Screen Signaling specification. Attribute information should be set in the ATR1 register and ATR2 register.

4.5.7 Digital RGB/YUV Interface

The Video Encoder has an 8-bit digital RGB/YUV interface that can be configured to interface many different types of digital LCD's. DM320 supports various (EPSON/CASIO/SANYO/KODAK etc.) LCD panels. The Video encoder also supports color sequential RGB output for the DisplayTech micro display. A 16 or 18 bit synchronous memory style interface supports LCDs that have embedded buffer memory.

Table 12 : Digital Output Mode pin

	Digital LCD		DM320	DM320	New Function	New Function	New Function	New Function
Pin Name	YUV 16-bits	Standard (DSC25)	EPSON LCD	CASIO LCD	uDisplay	uDisplay Dir.	LCD controller	LCD con. Dir.
H-sync	<-		<-	<-			nCS	O
V-sync	<-		<-	<-	FRAME_START	O	RnW	O
DCLK	<-	DCLK	DCLK	DCLK	CLK	O	CLK	O
						O	DATA_OUT[17]	
						O	DATA_OUT[16]	
Y7	<-	Data7	Data5	Data5	VIDEO_IN[7]	O	DATA_OUT[15]	(I/O)
Y6	<-	Data6	Data4	Data4	VIDEO_IN[6]	O	DATA_OUT[14]	(I/O)
Y5	<-	Data5	Data3	Data3	VIDEO_IN[5]	O	DATA_OUT[13]	(I/O)
Y4	<-	Data4	Data2	Data2	VIDEO_IN[4]	O	DATA_OUT[12]	(I/O)
Y3	<-	Data3	Data1	Data1	VIDEO_IN[3]	O	DATA_OUT[11]	(I/O)
Y2	<-	Data2	Data0	Data0	VIDEO_IN[2]	O	DATA_OUT[10]	(I/O)
Y1	<-	Data1	GCP	STBYB	VIDEO_IN[1]	O	DATA_OUT[9]	(I/O)
Y0	<-	Data0	LP	RIT	VIDEO_IN[0]	O	DATA_OUT[8]	(I/O)
C7	<-	LCD_AC	RES	CP			DATA_OUT[7]	(I/O)
C6	<-	LCD_OE	XINH	STH	DATA_VALID	O	DATA_OUT[6]	(I/O)
C5	<-	Bright	YSCL	STB			DATA_OUT[5]	(I/O)
C4	<-	COMP_SYNC	YSCLD	GPCK	(DISP_CLK)	O	DATA_OUT[4]	(I/O)
C3	<-	PWM_OUT	FRYS	GRES			DATA_OUT[3]	(I/O)
C2	<-		FRYP	POL			DATA_OUT[2]	(I/O)
C1	<-		FRX		(BALANCE)	I	DATA_OUT[1]	(I/O)
C0	<-		DY	GSRT	FULL	I	DATA_OUT[0]	(I/O)
AD[17]							AD[17]	O
:							:	
AD[1]							AD[1]	O
AD[0]							AD[0]	O
MnR							MnR	O
nBS							nBS	O
nRD							nRD	O
nWE0							nWE0	O
nWE1							nWE1	O
nWAIT							nWAIT	I

5 DSP Subsystem

The DSP subsystem is based on the TMS320C5409 DSP from TI catalog. This DSP core and its peripherals are fully code-compatible with other C54x products. The DSP software development tools for the C54x are very mature and can be used to develop, compile and debug the DSP code.

The DSP subsystem includes the C54x core, several blocks of volatile memories, an imaging extension coprocessor (iMX), Discrete Cosine Transform accelerator (DCT) and a variable length coding/decoding accelerator (VLCD).

This chapter gives only an overview of the DSP core and focuses more on its peripherals. For detailed information on the architecture of this 16-bit, DSP core and/or on the associated tools, refer to the C54x user's guides and tools documentation.

To realize image processing, image compression and voice processing, the DSP controls SDRAM DMA transfers to/from external memory and the iMX, DCT and VLCD accelerators. Since programming of the DSP is flexible and easy, users can improve the image processing pipeline and add differentiated functions to a system.

5.1 DSP core

C54x devices are fixed-point digital signal processors (DSPs) from the Texas Instruments TMS320 family. The C54x CPU, with its advanced modified Harvard architecture, features minimized power consumption and a high degree of parallelism.

This processor has one program memory bus and three data memory buses. It also provides an arithmetic logic unit (ALU) that has a high degree of parallelism, application-specific hardware logic, on-chip memory, and additional on-chip peripherals. The basis of the operational flexibility and speed of this DSP is a highly specialized instruction set.

Separate program and data spaces allow simultaneous access to program instructions and data, providing the high degree of parallelism. Two read operations and one write operation can be performed in a single cycle. Instructions with parallel store and application-specific instructions can fully utilize this architecture. In addition, data can be transferred between data and program spaces. Such parallelism supports a powerful set of arithmetic, logic, and bit manipulation operations that can all be performed in a single machine cycle. In addition, this processor includes the control mechanisms to manage interrupts, repeated operations, and function calls.

5.1.1 Features

- ❑ Low-power C54x DSP CPU
- ❑ Software-programmable wait-state generator with bank-switching wait state logic
- ❑ External memory interface
- ❑ Program space
- ❑ Data space
- ❑ I/O space
- ❑ Scan-based emulation logic

In the DM320, the wait state generator and the external memory interface are used to connect the DSP core to the expansion memory of the DSP subsystem and to the image buffers.

5.1.2 DSP peripherals

The DSP CPU core is associated with an HPI interface, an interrupt handler, a parallel interface XIO and a JTAG interface.

The JTAG interface is connected to the JTAG interface of the ARM core through some logic. This logic and the interface are described in more detail in the appendix.

The parallel interface is used to connect the core to the expanded memory, the shared blocks of memory, but as well, this interface is used to control the VLCD engine, DCT and iMX coprocessor. Refer to the appropriate chapter to get more details on each one of these points.

The host port interface (HPI) of the DSP is connected to the DSP Controller of the ARM subsystem. This connection cannot be modified. The HPI cannot serve any other purpose than to communicate back and forth with the ARM. The HPI bridge is described in the ARM-DSP Communication chapter.

The DSP core features as well a DMA controller that can be used to transfer data between the image buffers, the expanded memory block and the internal memory.

The PLL that can normally be found on a standard C54x device has been removed. The DSP PLL is now part of the clock controller module and fully controlled by the ARM.

The only peripherals that can be used by the DSP to communicate directly with external devices to the DM320 are the two multi-channel buffered serial ports and the XF pin. The DSP can control the XF signal by writing into the proper memory map register. This signal is multiplexed with the ARM GPIO pin. This signal can be read as well by the ARM through the DSP controller MMRs.

5.1.3 Multi-channel buffered serial ports

The DM320 has two high-speed, full-duplex multi-channel buffered serial ports (McBSPs), one of which (McBSP1) has its pins multiplexed with the ARM GPIO pins. The McBSP ports allow direct interface to other C54x devices, codecs, and other devices in a system. The McBSPs used in the DSP are based on the standard serial port interface found on other TMS320C54x devices. The two McBSPs that are accessible to the DSP are named McBSP0 and McBSP1.

A brief description of the McBSP included in the DM320's DSP core is given in this section. For detailed information on the DSP's McBSP, refer to the C54x peripheral manual; *The C54x reference set volume 1 (spru131)*.

5.1.4 DSP subsystem memory

Figure 37 shows the DSP subsystem. Shared memory for image processing (Image Buffer, 24 KW) is connected to the DSP expansion memory bus (XIO 16-bit data bus). The DSP has 64 KW of DSP program and data space memory. In addition to the DSP, the VLCD, DCT, iMX, image processing hardware accelerator, and DMA controller (in the Image Buffer module) can access shared memory (Image Buffer). Since the shared buffer is mapped to DSP memory addresses, it can be accessed directly. DSP memory is constructed from three independent spaces: program, data, and I/O. Control and configuration registers for the Image Buffer, iMX, DCT and VLCD are mapped to I/O space.

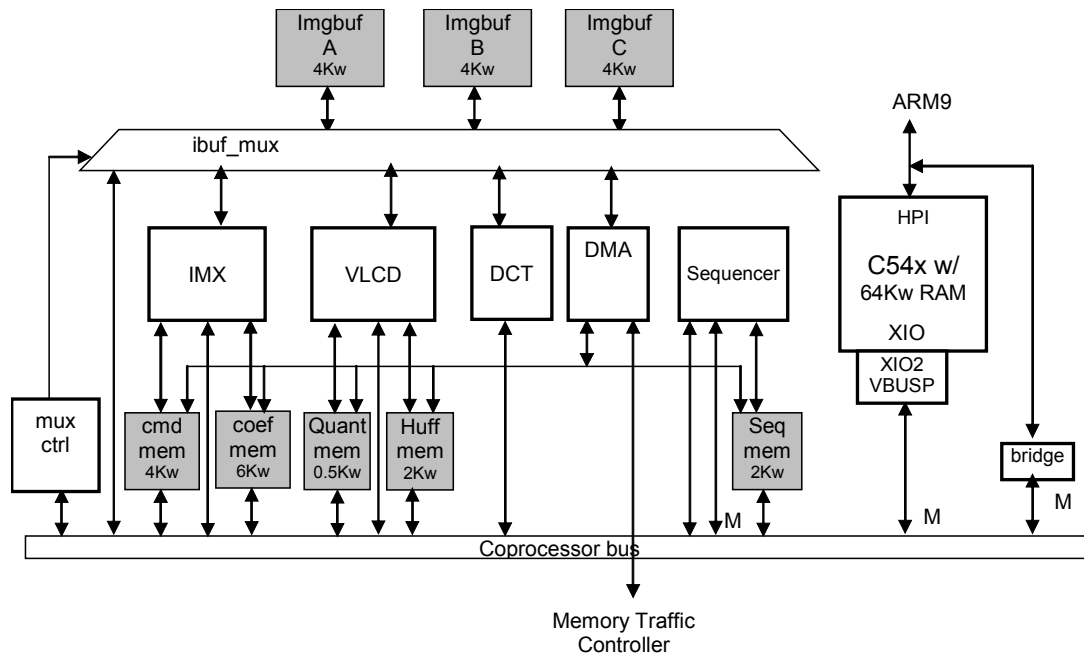


Figure 34 : Coprocessor/DSP Subsystem Block Diagram

5.1.5 Program data and IO spaces

The DSP memory is organized into three individually selectable spaces: program, data, and I/O. Within any of these spaces, DARAM, ROM, SARAM, or memory-mapped peripherals can reside either within the DSP core or off the core.

The program memory space contains the instructions to execute, as well as tables used in execution. The data-memory space stores data used by instructions. The I/O memory space interfaces to the iMX, DCT and VLCD engines and to the image buffer controller.

There are three CPU status register bits that affect memory configuration. The MP/MC, OVLY, and DROM bits are located in the processor mode status register (PMST).

5.1.6 DSP Memory map

The DSP subsystem offers 64 K words (1 word = 16 bits) of DSP internal DARAM memory, 16 K words of ROM memory, and 16 K words of shared RAM memory. Figure 35 shows the DSP memory map after a reset. The program space and the data memory spaces span the addresses 0x0h to 0x1FFFFh. Each block of memory has a width of 16 bits. Some blocks are available for both program and data spaces, such as the DARAM block when OVLY=1.

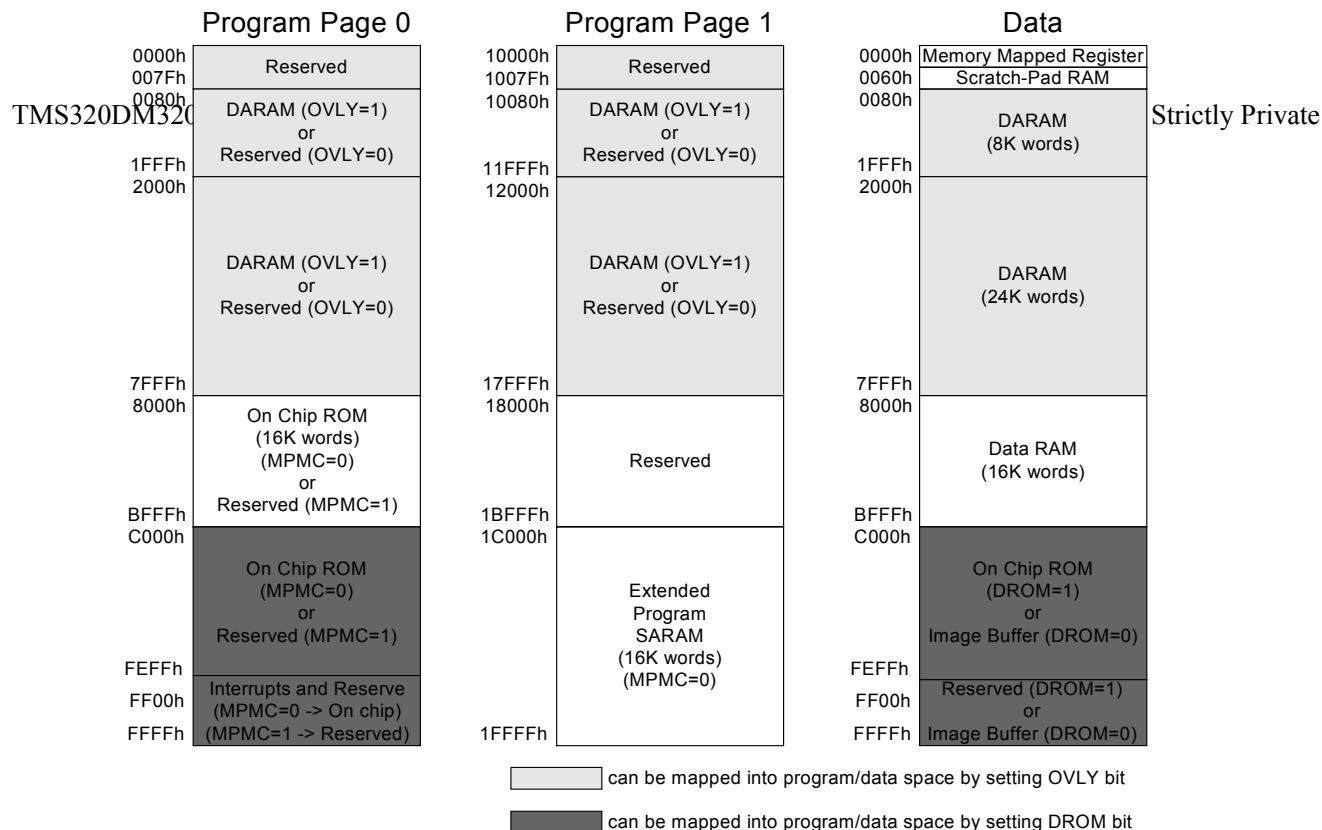


Figure 35 DSP Memory Map

5.1.7 I/O memory space

The C54x devices offer an I/O-memory space in addition to the program memory and data-memory spaces. The I/O-memory space is a 64K-word address space (0000h–FFFFh) and exists only external to the DSP device. Two instructions, PORTR and PORTW, are used to access this space. Read timings vary from those of the program-memory and data-memory spaces to facilitate access to individual I/O-mapped devices rather than to memories.

On the DM320, only some addresses from this space are used. Image Buffer registers, iMX registers, DCT registers and VLCD registers are mapped to I/O spaces. Signals to control the Image buffer clock, iMX buffer clock, DCT buffer clock and VLCD buffer clock are mapped to an I/O space. A signal to interrupt the ARM from the DSP is also mapped to an I/O space. Table 13 shows the address allocation of each module.

Table 13 : DSP I/O Address for Coprocessors

Name	Start Address	End Address
Image Buffer	0x0000	0x007F
iMX Control	0x0080	0x00FF
VLCD Control	0x0100	0x017F
DCT Control	0x0180	0x01FF
Sequencer Control	0x0200	0x027F
Interrupt/clock Control	0x0280	0x02FF

5.2 Image Buffer

The coprocessor subsystem is part of the DSP subsystem. It is designed to enhance DSP's capability in image processing and in other computation-intensive applications. The coprocessor subsystem consists of the following components:

- ❑ Coprocessors (iMX, DCT and VLCD)
- ❑ Sequencer
- ❑ DMA controller (built into the Image Buffer module)
- ❑ Shared memory
- ❑ Shared memory access control
- ❑ The DSP's Interrupt controller
- ❑ Clock and coprocessor manager

All components reside in DSP's data page and/or I/O space, and all components are controlled by the DSP. Figure 34 shows the architecture of the Coprocessor Subsystem. There are three coprocessors in the system, Image Extension (iMX), Discrete Cosine Transform and Variable Length Coder/Decoder (VLCD). The iMX accelerates computation-intensive imaging processing functions such as filtering and interpolation. The DCT provides acceleration for the Discrete Cosine transforms required in various still image and motion picture compression standards. The VLCD provides acceleration for Huffman coding and encoding as well as quantization and inverse quantization required in various still image and motion picture compression standards.

The DMA controller provides a pathway to the ARM subsystem's SDRAM, allowing high-speed data transfer between the ARM subsystem and the DSP Subsystem. The DMA controller is part of the image buffer module. The DMA controller referred to in this section is the DMA controller built into the image buffer module not the DSP's DMA controller. However, the DSP's DMA controller can be used to transfer data between the DSP and each of the shared memories. The DSP's DMA controller cannot be used to transfer data to SDRAM, this can only be done by the image buffers DMA controller.

Each coprocessor and the DMA controller have their own private memory to store data structures and instructions unique to each functional unit. In addition, the DSP, iMX, DCT, VLCD and DMA controller also communicate to each other in a cooperative manner in order to perform useful functions. The communication is made possible by a shared memory, shared memory access control, and DSP interrupts.

The figure below shows the shared memory blocks. There are three types of coprocessor memory: Image buffers, iMX coefficient/command buffer, and VLCD table buffer. Image buffers consist of Buffers A, B and C. Buffers A, B and C are shared memory among the coprocessors and the DMA controller. The iMX coefficient/command buffer is used to store the coefficients and command for the iMX. The VLCD buffer is used to store Huffman tables, quantization coefficients, and other data structures for the VLCD. As mentioned in the previous section, user need to configure the device to add 1 wait state when accessing these memory blocks (Image Buffer A, Image Buffer B, Image Buffer C, iMX coefficient/command buffer, and VLCD table buffer).

The DSP software through registers in the Image Buffer module controls shared memory access. When a coprocessor, the DMA controller, or the DSP is given access to a shared memory buffer that module has a dedicated link to the buffer.

Each coprocessor and the image buffer's DMA controller can interrupt the DSP.

The Clock and Coprocessor Manager manages the clocks and provides top-level control to coprocessors and other functional units in the Coprocessor Subsystem.

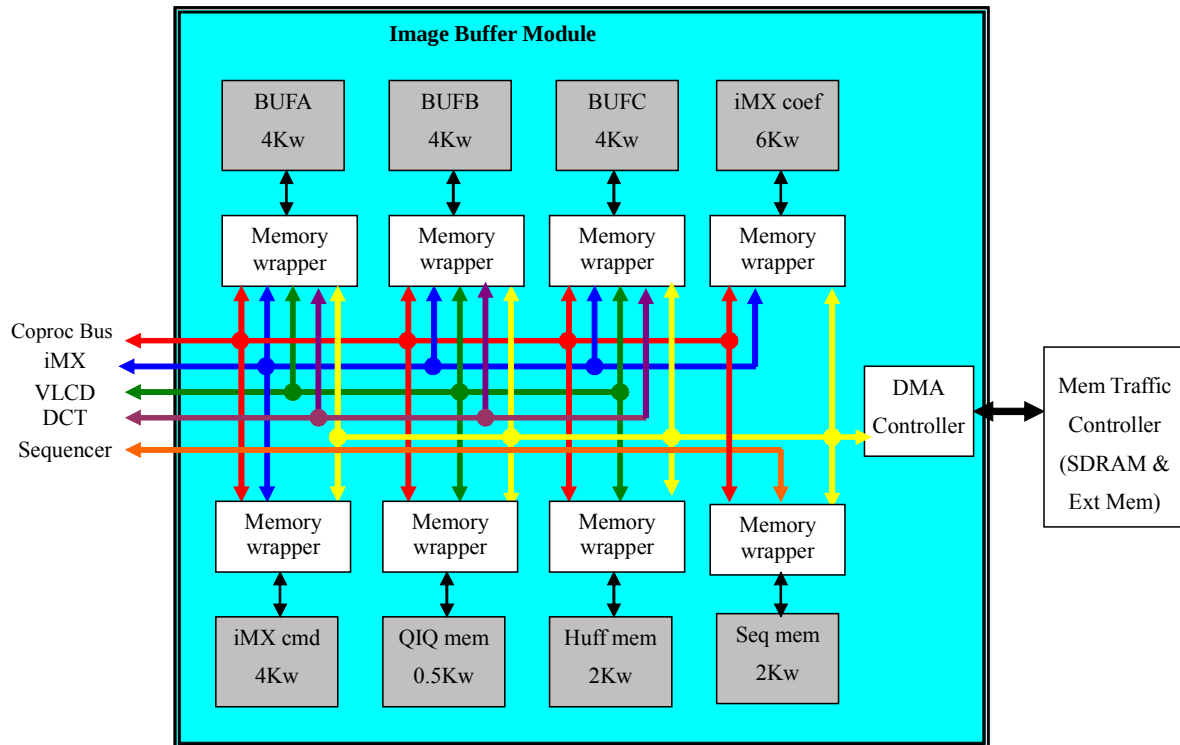


Figure 36 : Shared Memory Block

BUFA, **BUFB** and **BUFC** are used as the working memory for the image processing, and are shared either by DSP, iMX, DCT or VLCD. The memory wrapper is doing this switch logic for the shared memory. Note: a buffer can't be shared by all of access unit at same time, and access switch is controlled by DSP. **iMX coefficient memory** is used as the coefficient and temporary working space for iMX. DSP needs to initialize the coefficient of filter into this memory. The buffer switch is controlled by DSP. **iMX command memory** is used as the program command for the iMX. Before iMX is active, DSP needs to download the program into this memory. The buffer switch is control by DSP. **VLCD quantization table memory** is used as the quantization table for the VLCD engine. Table can be updated by DSP. **VLCD Huffman table memory** is used as the Huffman table for the VLCD engine. Table can be updated by DSP. The buffer switch is control by DSP.

The image buffer module manages access to each of the shared memory blocks and manages DMA transfers from the image buffers to SDRAM or External Memory.

5.2.1 Access Switching

Table 14 shows each memory size and which modules can access each buffer. The DSP/Sequencer or ARM controls which module has access to each buffer by configuring registers.

Table 14 Memory Structure

Memory	Access Width from Each Host/Port (blank = no access)									
	Cop Bus	iMX			VLCD				DCT	DMAC
		ibuf	coef	cmd	ibuf0	ibuf1	hm	qm		
Image Buffer A/B/C	16-b	128-b			32-b	32-b			32-b	32-b

iMX coef	16-b		128-b							32-b
iMX cmd	16-b			16-b						32-b
Huffman mem	16-b						32-b			32-b
QIQ mem	16-b							16-b		32-b
Sequencer mem	16-b									32-b

5.2.2 The Image Buffer's DMA Controller

The Image Buffer module has a DMA controller that allows the DSP to initiate data transfers between the image buffers (Buffer A, Buffer B and Buffer C) and SDRAM or external memory or between the iMX coefficient buffer and SDRAM or external memory. It can also transfer the data to/from iMX cmd, VLCD Q, Sequencer and VLCD Huffman memory. The DMA controller will handle the handshaking sequence to/from SDRAM or the External Memory controller automatically.

The DMA controller supports 2-dimensional data transfers. As shown in Figure 37, DMA_XNUM specifies the number of pixels in the horizontal direction, and DMA_YNUM specifies the number of lines in the vertical direction. By setting each memory lead address (BUF_ADDR, SDR_ADDR) and line offset value (BUF_LOFST, SDR_LOFST), block transfer of second-order image data is possible. This function is effective in cases where processing in MACRO block units is required, such as in JPEG compression.

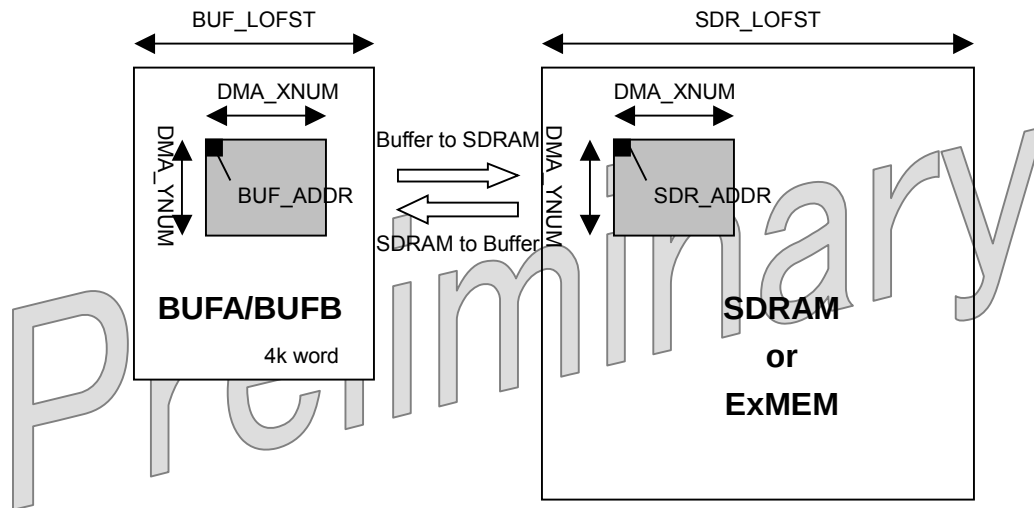


Figure 37 Two dimensional data transfer

DMA controller also has a function mode of byte-to-word conversion. In this mode, the 32bit(4 bytes) read data from SDRAM is converted to 64bit(4 words), and stored into image buffer. Table 15 shows data format in image buffer when byte-to-word conversion is enabled.

Table 15 byte-to word conversion

SDRAM read data	0(LSB) 31(MSB)							
	byte0	byte1	byte2	byte3				
Image Buffer Address	4n (word)		4n+1		4n+2		4n+3	
write data	byte0	0	byte1	0	byte2	0	byte3	0

5.3 Imaging Extension (iMX)

Details describing the internal architecture of the iMX co-processor are generally not available to the user. The description of the iMX given in this chapter is very limited. To use the iMX, you should use the C callable iMX API provided by Texas Instruments. Refer to the documentation provided with the C callable API library for more information on these APIs.

The iMX is a parallel MAC engine for increasing DSP image processing performance. The iMX hardware module consists of a flexible control and memory interface and four MACs. The iMX works well in a shared memory configuration with the DSP, so that regular block-based image processing tasks can be accelerated by the iMX, and the DSP can handle irregular processing as well as task-level control.

The iMX can be used to increase the computation speed of many computations, such as the following computations:

- ❑ 2-D FIR filtering
- ❑ CFA interpolation
- ❑ Color space conversion
- ❑ Chroma down-sampling
- ❑ Edge enhancement
- ❑ False color suppression
- ❑ Table lookup
- ❑ Motion estimation
- ❑ Generic matrix multiplication
- ❑ Generic linear transform

DM320 iMX is a 8-MAC version based on DSC27 iMX, with the following enhancements:

- Logical AND/OR/XOR function to support extracting lower bits, for post-lookup interpolation
- MIN-ID and MAX-ID functions. Basically preserves the pointer value to which the minimal value resides. This helps motion estimation.
- New “CALL” command which enables re-use of an iMX command module
- Sequential access to image buffer A and B for non concurrent process
- Logical Shift

5.3.1 iMX Structure

As shown in Figure 38, the iMX consists of 8 parallel multiply-accumulate units (MACs). For most of the calculations mentioned above, the iMX processes 1440 (180x8) million MACs/sec. Each MAC can perform multiplication, addition, subtraction, absolute value, and accumulate.

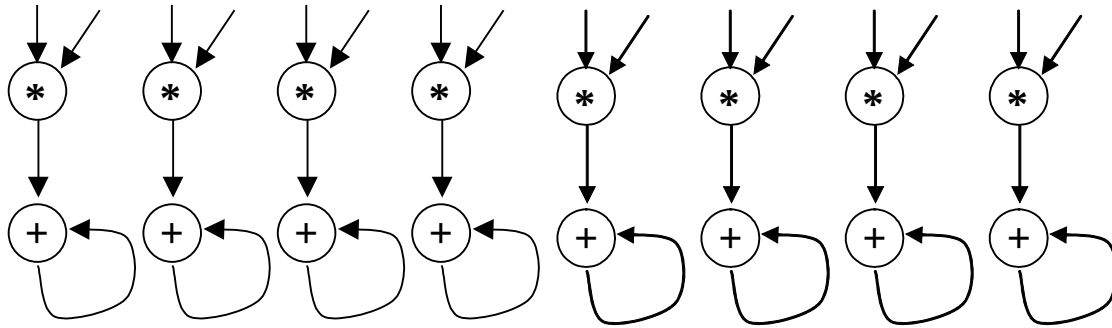


Figure 38 Structure of eight parallel MACs

5.3.2 DSP/iMX Shared Memory Interface

In the DM320 Coprocessor Subsystem we have a C54x DSP and a sequencer communicating with iMX via control/status registers and shared memory blocks. The following block diagram shows the connection of coprocessor modules and memory blocks to the DSP and the sequencer. The Image Buffers A, B, and C are for generic data storage. The other memory blocks have specific functions normally, but the iMX coefficient memory can also be used for intermediate data storage.

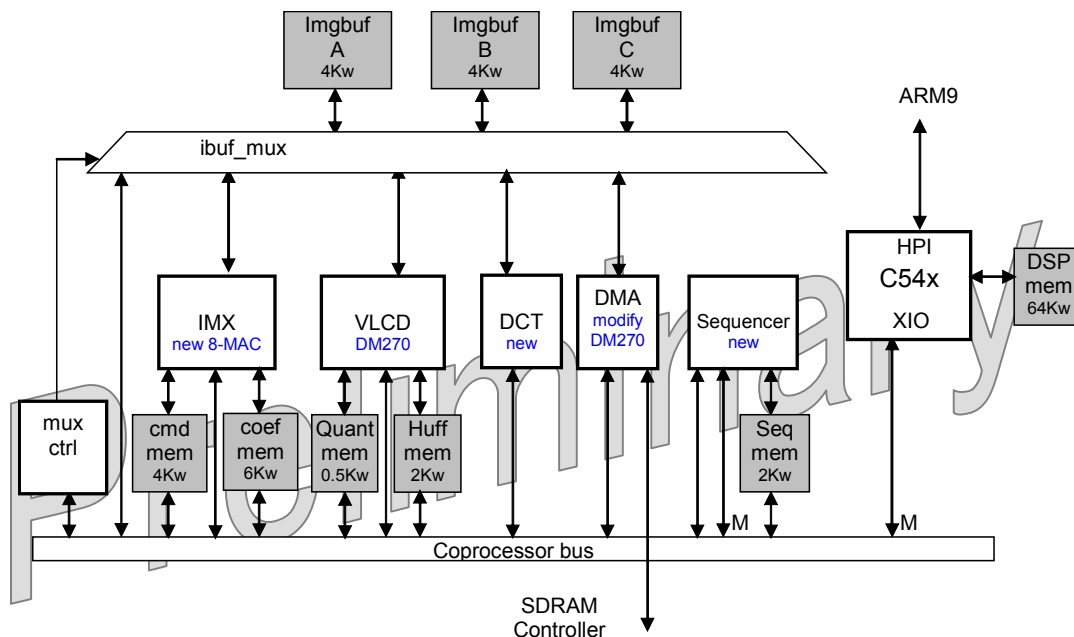


Figure 39 Co Processor Memory Configuration

To simplify buffer swapping programming, shared memory interface is designed such that iMX does not access multiple image buffers at the same time. To iMX all three buffers are on the same memory space, and DSP or Sequencer controls which of the three are accessible from iMX.

DSP cannot see all three image buffers at the same time, either, due to the size of the buffers and of the other memory blocks we need to fit into DSP's data memory space on the DSP XIO bus. A table is shown next for the DSP memory map.

To make it easier for DSP to process and cycle through the image buffers without relocating the address pointers, we have a paging scheme in the hardware. When this feature is enabled, in the image buffer A space, we can see A, B, or C, determined by a register.

Name	Start Address	End Address	Size (Words)
Image Buffer A/B/C	0xC000	0xCFFF	4K
iMX Coef. Buffer	0xD000	0xE7FF	6K
iMX Cmd Buffer (paged)	0xE800	0xEFFF	2K
VLC Huffman Buf	0xF000	0xF7FF	2K
Sequencer Memory (paged)	0xF800	0xFBFF	1K
VLC Q buffer	0xFC00	0xFDFF	512

Preliminary

5.4 DCT Engine

In JPEG and MPEG-1/2/4 compression standards, DCT (discrete cosine transform) is specified as the method in the compression flow to transform blocks of image in the spatial domain to the 2-D frequency domain. DCT concentrates the signal energy to the lower frequency components to facilitate compression. IDCT (inverse discrete cosine transform) is numerically the inverse of DCT, and is used in the decompression flow to recover image from frequency domain data.

In these image/video coding standards, fixed size, 8x8 2-D DCT/IDCT are used. Consequently, the computation required is a standard and fixed function, and in silicon implementation, it is often advantageous to implement DCT/IDCT in hardware, as opposed to on programmable microprocessor or DSP. Hardware implementation offers better cost/performance/power tradeoffs than software.

This hardware DCT module in DM320 performs 2-D 8x8 DCT and IDCT. The DCT module and other coprocessors together provide high performance and power-efficient imaging and video coding and decoding capabilities in DM320.

Performance will be 2 cycles per DCT coefficient for forward or inverse 2-D transforms.

5.5 Sequencer

The sequencer is a simple microcontroller for synchronizing DMA with iMX, VLCD, DCT, and the DSP. It also has simple arithmetic functions to perform simple buffer management, register programming, and looping control. The sequencer runs at $\frac{1}{2}$ the clock rate of the accelerators.

5.6 VLCD Engine

Preliminary

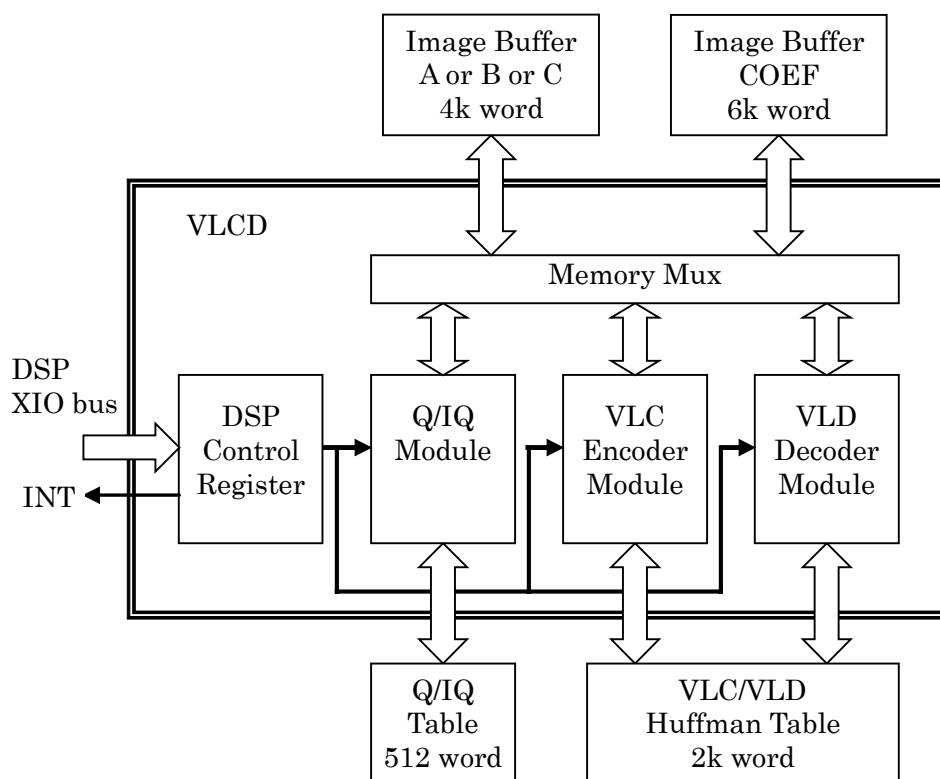


Figure 40 VLCD Block Diagram

DM320 VLCD module has QIQ,VLC and VLD functions for JPEG,MPEG standard. Quantizer and Inverse Quantizer (Q/IQ) module of the DM320 DSP subsystem is a high-performance, intelligent and highly programmable hardware accelerator for the quantization and de-quantization of DCT coefficients. The Q/IQ is highly autonomous and is capable of carrying out almost all the steps of quantization and de-quantization process with little or no help from the DSP.

VLCD module supports the quantization and de-quantization of the following standards:

- ❑ Quantization, zigzag(& h/v alternated) scan for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ Inverse Quantization, zigzag (& h/v alternated) scan for JPEG, MPEG1/2, MPEG4 and H.263.
- ❑ Quantization, zigzag(& h/v alternated) scan for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ Huffman encode for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ Huffman decode for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ Quantization, zigzag scan for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ CBP (Coded Block Pattern) detection
- ❑ Auto block skip for un-coded block (CBP)
- ❑ Auto insertion of Macro Block Header or SYNC Markers

With highly pipelined architecture, at its peak speed, Q/IQ module is capable of processing 180M DCT coefficients per second.

Q/IQ interfaces with the rest of the DSP subsystem via Image Buffers(A/B), Coefficient memory, Q/IQ Table and a set of control registers.

Q/IQ can access the Image Buffers(A/B) or Coefficient memory. Q/IQ can also transfers data to and from the Image Buffer(A/B) or Coefficient memory. Input DCT data must be placed in the image buffers(A/B) or Coefficient memory prior to the start of the quantization or de-quantization process. Q/IQ read the input data from and writes output data to the Image Buffer(A/B) or Coefficient memory.

Q/IQ Table contains more configuration parameters in addition to what is already specified in the control registers. During the Q/IQ process, Q/IQ engine reads the parameters from the Q/IQ Table. The additional parameters provide more flexible control over the Q/IQ process. The parameters include quantization/de-quantization coefficients, DC predictor data, zigzag scan path and block operators.

The VLCD coprocessor handles Huffman encode and decode operation for image compression algorithms such as JPEG, MPEG-1/2/4, and H.263. It operates with Huffman tables preloaded by DSP, via shared memory blocks. The VLCD is highly autonomous and is capable of carrying out all the steps of Huffman coding and decoding process with little or no help from the DSP.

VLC/VLD interfaces with the rest of the DSP subsystem via Image Buffers(A/B), Coefficient memory, VLC/VLD Huffman Table and a set of control registers. VLC/VLD can access the Image Buffer(A/B) or Coefficient memory. VLC/VLD can also transfers data to and from the Image Buffer(A/B) or Coefficient memory. The quantized DCT data must be placed in the image buffers(A/B) or Coefficient memory prior to the start of the VLC. The JPEG/MPEG bitstream data must be placed in the image buffers(A/B) or Coefficient memory prior to the start of the VLD. VLC/VLD read the input data from and writes output data to the Image Buffer(A/B) or Coefficient memory. VLC/VLD Huffman Table contains more configuration parameters in addition to what is already specified in the control registers. During the VLC/VLD process, VLC/VLD engine reads the parameters from the VLCD Huffman Table.

DSP configures and controls VLCD module via the control registers, which are mapped into DSP's IO space. The control registers allow the DSP to program Q/IQ and VLC/VLD or various industrial or user-defined standards, start the quantization or/and de-quantization process and VLC or VLD process and monitor its progress. The control registers also contain a set of address pointers to identify the locations of input data and output data in the Image buffers(A/B) or coefficient memory and the location of additional configuration parameters in the Q/IQ Table and VLC/VLD Huffman Table.

6 ARM Subsystem

The ARM subsystem is made up of the ARM9EJ-S RISC processor core, Instruction cache, Data cache, MMU, static RAM memory, and peripherals. The ARM controls the various peripheral modules including the timers, interrupt controller, video interface, burst compression engine, on-screen display, USB, and serial ports. The user interface should run on this processor.

In a system, the ARM is responsible for handling many system functions such as system-level initialization, configuration, user interface, user command execution, connectivity functions, and overall system control. The general purpose ARM handles these functions because it has a larger program memory space and better context switching capability, and is thus more suitable for complex, multi-tasking, and general purpose control tasks compared to the on-chip DSP.

The ARM subsystem of the DM320 offers the following features:

6.1 Features

- ❑ 32-bit RISC processor
- ❑ Maximum operating frequency 160 MHz
- ❑ 16-Kbyte built-in RAM
- ❑ 16-K byte Instruction cache
- ❑ 8-K byte Data cache
- ❑ MMU
- ❑ 32/16-bit command sets
- ❑ Memory format: Little Endian or Big Endian Format
- ❑ 2 interrupt systems (FIQ, IRQ)
- ❑ 16 external terminal interrupt systems
- ❑ x4 hardware timers
- ❑ Watchdog timer
- ❑ JTAG base emulation
- ❑ External memory interface

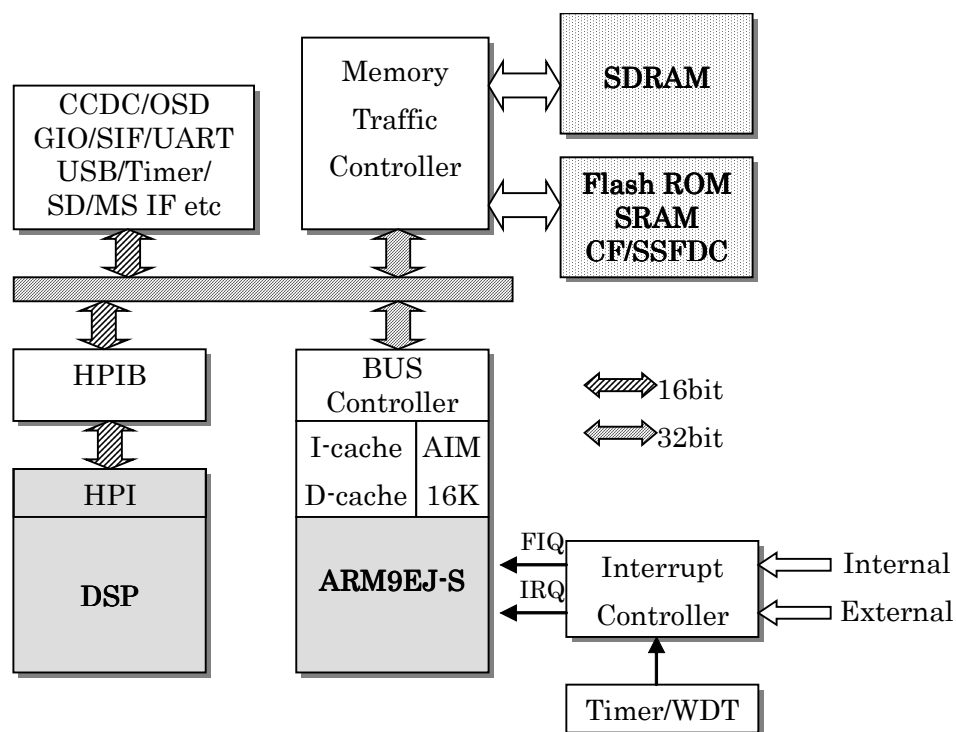


Figure 41 ARM Subsystem

6.2 ARM926EJ-S RISC Processor Core

The ARM926EJ-S core is integrated in the DM320 chip. The ARM9E core is specified to run at up to 160 MHz. The 926EJ includes new signal processing extensions to enhance 16-bit fixed point performance using a single-cycle 32 x 16 multiply-accumulate (MAC) unit, and implements a 32/16-bit instruction set giving excellent code density, maximizing savings on system cost. The ARM926EJ-S core also includes ARM technology which enables the direct execution of Java bytecodes in hardware. The ARM subsystem has its own 16 Kbytes of local static RAM.

6.3 Cache (I-cache and D-cache) system

I-cache in DM320 contains 16kbyte RAM for caching instruction. D-cache contains 8kbyte RAM for caching data.

- ❑ 8-word (256bit) line size
- ❑ 4-way set associative
- ❑ Pseudo-random or round-robin replacement
- ❑ DCache supports write-through and write-back (or copy-back) cache operations
- ❑ Cache lock down supported
- ❑ Two Dirty bit in DCache tag-RAM
- ❑ Support cache clear function

6.4 Memory Map

The ARM9 in the DM320 has access to all the peripherals, external memory and co-processors.

In addition, DSP memory can be also accessed by ARM through HPI Bridge.

The characteristics of the different memory spaces accessible by the ARM are summarized in the Table 16 and figure below.

Table 16 Memory Structure

Area Name	Start address	End address	Size(byte)
ARM Internal RAM*	0x0000:0000	0x0000:3FFF	16 K
ARM Internal ROM*	0x0000:4000	0x0000:5FFF	8 K
Internal Peripheral	0x0003:0000	0x0003:0FFF	4 K
DSP Memory	0x0004:0000	0x0005:FFFF	128 K
Internal Peripheral #2	0x0006:0000	0x0006:0FFF	4 K
Coprocessor	0x0008:0000	0x0009:FFFF	128 K
External Memory	0x0010:0000	0x110F:FFFF	256 M
Compact Flash	0x4000:0000	0x40FF:FFFF	16 M
SSFDC	0x4800:0000	0x48FF:FFFF	16 M
External SRAM	0x5000:0000	0x50FF:FFFF	16 M
External SRAM	0x6000:0000	0x60FF:FFFF	16 M
VLYNQ	0x7000:0000	0x73FF:FFFF	64 M
Mirror External Memory	0xFFFF:0000	0xFFFF:FFFF	64 K

Preliminary

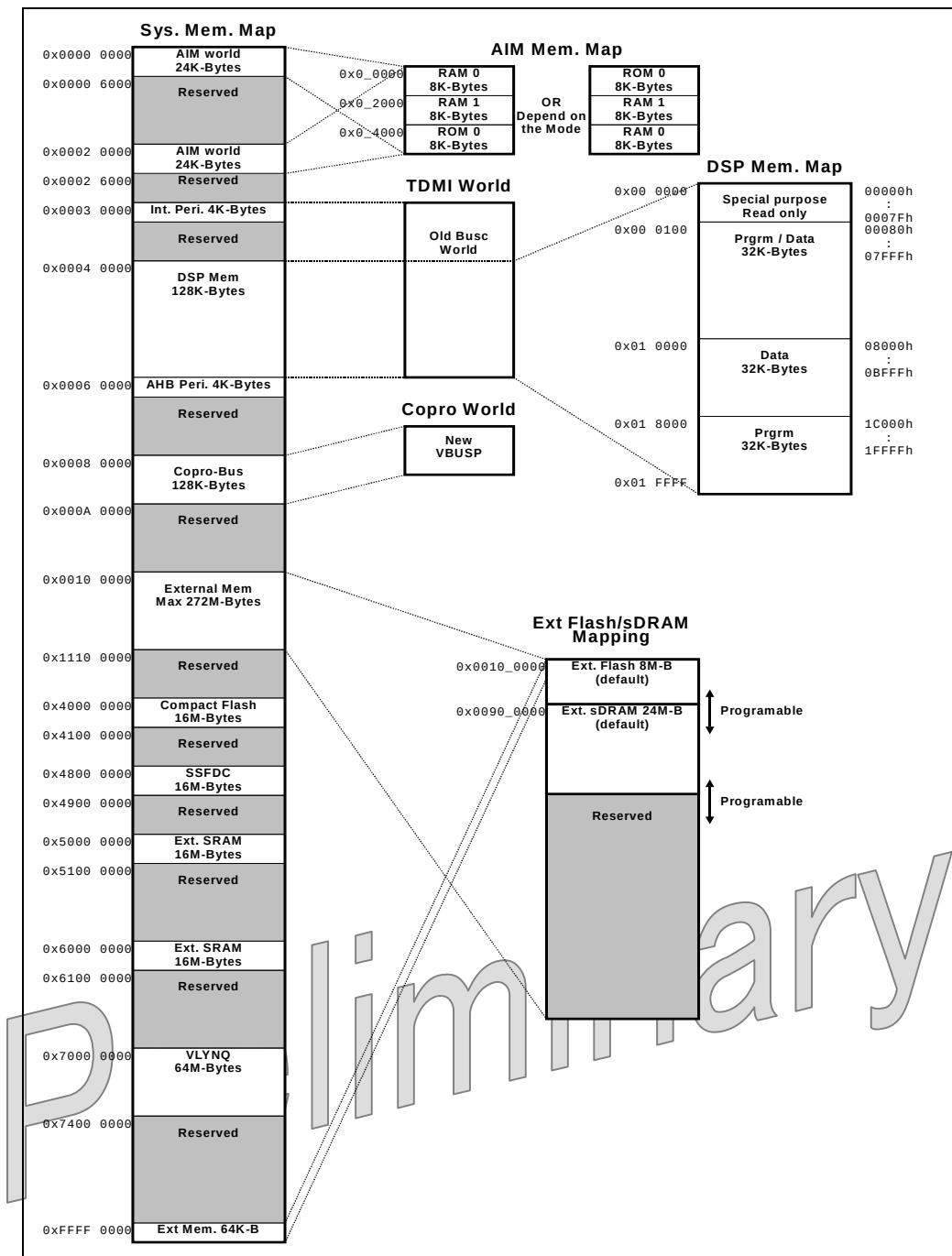


Figure 42 : DM320 ARM926 Memory Map

6.4.1 ARM Internal RAM

This is ARM internal memory (IRAM). Since ARM can access this memory region in 1 cycle (no wait state), the RTOS (real time OS) or application programs can be operated more quickly here. Contents of this memory region can be access as byte (8 bit), half word (16 bit) or word (32 bit).

6.4.2 Peripheral Registers

Table 17 Peripheral module memory map

Module	Start Addr	End Addr	Size
Timer 0	0003:0000	0003:007F	128
Timer 1	0003:0080	0003:00FF	128
Timer 2	0003:0100	0003:017F	128
Timer 3	0003:0180	0003:01FF	128
Serial Port 0	0003:0200	0003:027F	128
Serial Port 1	0003:0280	0003:02FF	128
UART0	0003:0300	0003:037F	128
UART1	0003:0380	0003:03FF	128
Watch Dog Timer	0003:0400	0003:047F	128
MMC/SD	0003:0480	0003:04FF	128
Interrupt Controller	0003:0500	0003:057F	128
GIO	0003:0580	0003:05FF	128
DSP Controller	0003:0600	0003:067F	128
OSD	0003:0680	0003:06FF	128
CCD controller	0003:0700	0003:077F	128
	0003:0780	0003:07FF	128
Video Encoder	0003:0800	0003:087F	128
Clock Controller	0003:0880	0003:08FF	128
BUS Controller	0003:0900	0003:097F	128
SDRAM Controller	0003:0980	0003:09FF	128
External Memory I/F	0003:0A00	0003:0A7F	128
Preview Engine	0003:0A80	0003:0B7F	256
Hardware 3A	0003:0B80	0003:0BFF	128
	0003:0C00	0003:0C7F	128
Memory Stick	0003:0C80	0003:0CFF	128
	0003:0D00	0003:0D7F	128
I2C	0003:0D80	0003:0DFF	128
	0003:0E00	0003:0E7F	128
	0003:0E80	0003:0EFF	128
USB Full Speed OTG	8000:0000	8000:00FF	256
	8000:0100	8000:01FF	256
USB DMA	8000:0200	8000:02FF	256
	8000:0300	8000:03FF	256
	0006:0000	0006:02FF	768
VLYNQ	0006:0300	0006:03FF	256
	0006:0400	0006:0EFF	3072
AHBBUSC	0006:0F00	0006:0FFF	256

6.4.3 DSP Memory

When the ARM accesses the DSP internal memory through the HPI bridge module (HPIB), one wait-state is inserted automatically. In addition to it, extra wait cycles will be inserted depending on the wait signal that is generated in HPIB module.

6.4.4 External Memory

As external memory interfaces, DM320 has an SDRAM interface and a memory card or general-purpose memory interface. ARM can access SDRAM via the SDRAM controller module. Also, it can access memory card or flash ROM via the external memory interface module. In access from ARM to external memory, a wait state is automatically inserted, and its cycle varies depending on SDRAM frequency, external memory access time and so forth.

The SDRAM memory access is controlled by the SDRAM controller.

The dynamically allocated external memory, which is directly accessible by the ARM, is a maximum of 256 Mbytes, and can be divided into a maximum of 6 regions (SDRAM, and EM0 to EM4). By changing the memory map dynamically, external memory of a different capacity can be used efficiently. Each of the 6 regions can be used by setting a boundary address. Byte (8-bit), half-word (16-bit) and word (32-bit) types of accesses are possible.

6.5 Interrupt Controller

The Interrupt Controller (INTC) has 46 interrupt channels: 30 internal peripheral interrupt channels and 16 external interrupt channels. Table 18 lists each interrupt and the interrupt's corresponding function.

The INTC has the following characteristics:

- ❑ 47 interrupt channels (16 external channels)
- ❑ Interrupt mask each for channel
- ❑ Each interrupt channel can be mapped to an FIQ or IRQ type interrupt
- ❑ Configurable hardware interrupt priority
- ❑ Configurable interrupt entry table

Table 18 DM320 Interrupts

Identify Number	Interrupt Source
0	TIMER0
1	TIMER1
2	CCD TIMER0
3	CCD TIMER1
4	CCDC VD0
5	CCDC VD1
6	CCDC VD2
7	OSD
8	SERIAL with DMA
9	SERIAL
10	External HOST
11	IMG BUFFER
12	UART0
13	UART1
14	USB0
15	USB1
16	VLYNQ
17	MTC0
18	MTC1
19	GIO0
20	GIO1

21	GIO2
22	GIO3
23	GIO4
24	GIO5
25	GIO6
26	GIO7
27	GIO8
28	GIO9
29	GIO10
30	GIO11
31	GIO12
32	GIO13
33	GIO14
34	GIO15
35	PREVIEW0
36	PREVIEW1
37	WDT
38	I2C
39	CLKC
40	E2ICE
41	ARM COMMRX
42	ARM COMMTX
43	“SD/MMC I/F 0” or “MS0”
44	“SD/MMC I/F 1” or “MS1”
45	(RSV) or “MG”
46	RSV

Each interrupt channel can be mapped to an FIQ or IRQ type interrupt, and each channel can be enabled or disabled. The ARM core supports two interrupt types: FIQ and IRQ. See the ARM9 User's Manual for detailed information about the ARM's FIQ and IRQ interrupts.

The INTC supports hardware configurable interrupt priority, and entry address generation. Entry address generation minimizes time spent jumping to interrupt vectors. When an interrupt occurs the corresponding interrupt vector is stored in the INTC's ENTRY register. The interrupt service routine can read the ENTRY register and jump to the interrupt vector. Thus the ARM does not have to execute any parsing software to associate the interrupt with the correct interrupt vector.

6.5.1 INTC Methodology

Figure 43 illustrates the INTC methodology. As shown in Figure 43, interrupts are mapped to FIQ or IRQ type interrupts by setting the FISELn register. When an interrupt occurs the status is reflected in either the FIQn or IRQn registers. Interrupts are enabled or disabled, masked, by setting the EINTn register. Note: even if an interrupt is masked, the status of each interrupt is still reflected in the FIQn and IRQn registers. The INTC generates the entry address of the pending interrupt with the highest priority and stores the entry address in the FIQENTRY or IRQENTRY register. Interrupt priority can be configured in the INTPRIn registers, which are described below. The entry address is equal to a base address plus an offset, and the base address can be set in the EABASEn registers. Each of the INTC reregisters is described below.

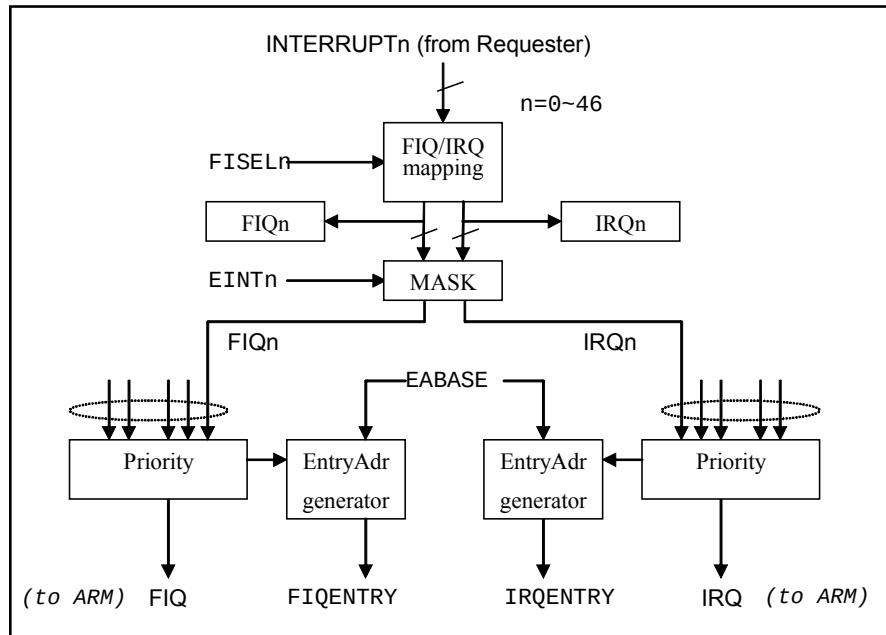


Figure 43 INTC Methodology

6.5.2 Hardware Configurable Interrupt Entry Table

The hardware configurable interrupt entry table can be used to minimize the time spent jumping to interrupt service routines. The entry table, created in software, can be stored anywhere in memory, but it must be on an 8-byte boundary. The structure of the entry table is shown in Figure 44.

Initialization:

1. Mask FIQ and IRQ
2. Mask all interrupt request by MASK register
3. Mapping the each interrupt to FIQ or IRQ by selection register
4. Setting up the each priority by priority configuration register
5. Setting-up a interrupt table base register(EABASE)
6. Enable the corresponding interrupts
7. Enable the FIQ and IRQ

Using the entry table to process interrupts works as follows:

1. ARM receives common FIQ/IRQ interrupt signal
2. ARM responds to interrupt signal and jumps to common FIQ or IRQ vector (depending on whether an FIQ or IRQ type interrupt occurred)
3. Common FIQ(0x001C) /IRQ(0x0018)vector
4. The software located in memory at the common interrupt vector reads the FIQ entry or the IRQ entry registers to get the interrupt service routine address, and then jumps to this address, the interrupt handler corresponding to $INTx$.
5. The interrupt is cleared by setting the bit corresponding to $INTx$ in FIQ, IRQ status register

6. Return to the common interrupt vector routine from the interrupt handler
7. Return to the state of program execution that existed prior to when the interrupt occurred from the common interrupt vector routine

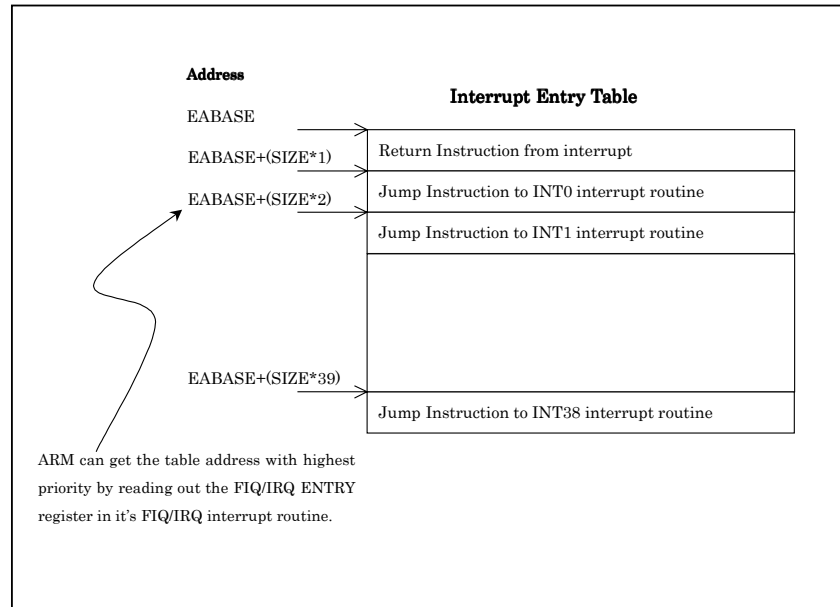
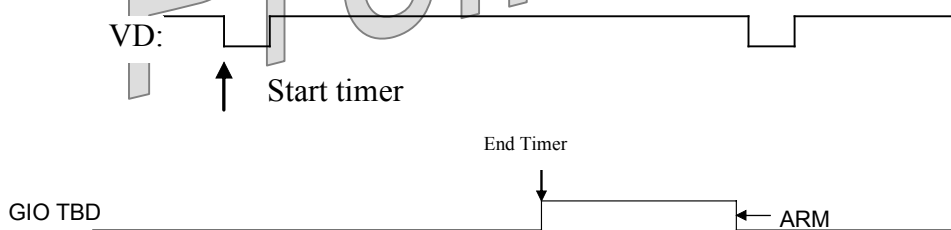


Figure 44 Interrupt Entry Table

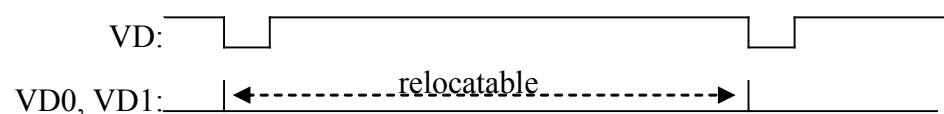
6.6 Timers, Watch-Dog Timer

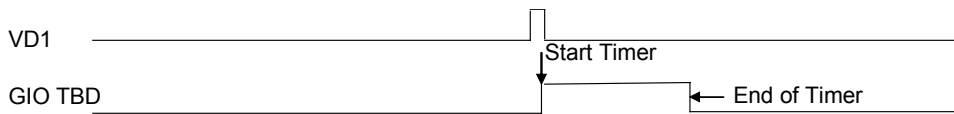
The ARM subsystem has four timers. The output of each timer is input into an ARM interrupt controller. Each of the general-purpose timers can run in one of two modes: one-shot mode and free-run mode. In one-shot mode, an interrupt only occurs once and then the timer must be explicitly reset to begin the timing operation again. In free-run mode, when the timer generates an interrupt, the timer counter is automatically reloaded to start the count operation again.

The outputs of Timer0 and Timer1 are multiplexed to GIO pins (GIOTBD). The intent is to use these timers to precisely control a mechanical shutter and a strobe. For control of a mechanical shutter the following behavior is desired. (note that the GIO output does not change state when the timer is started)



Control of the strobe requires the following behavior.





The Watch Dog Timer (WDT) is used to determine when a system is running out-of-control and to initiate the system's recovery from an out-of-control state. If the WDT is not reset within a configurable time, the WDT automatically interrupts the ARM or resets the ARM. This ARM reset signal can be monitored externally through GIO pin(GIOTBD). The WDT uses the external clock connected to the M27XI pin. In normal operation a 27MHz clock is connected to M27XI.

6.7 PLL / Clock Controller

The PLL / Clock Controller module generates multiple clock frequencies from a single crystal input. This module generates an internal interim frequency as a multiplier of the input crystal, then divides this frequency down to obtain the necessary clocks to drive the ARM9, IMX, Coprocessors, USB and SDRAM. The input frequency is user definable within the allowable range of 20-35MHz, however under normal conditions would need to be 27MHz to allow for accurate NTSC/PAL video output. The internal interim frequency is the direct output from the PLL and can be a divided up to 4 ($\times 1, 1/2, 1/3, 1/4$) of the PLL output frequency. Up to four kind of output frequencies can be generated from this interim internal frequency.

As shown in Figure 45 the CLKC has two analog PLLs built into it, and you can freely set the frequencies of the ARM clock, SDRAM clock, and DSP clock using these PLLs. To reduce system power, each clock frequency can be slowed down and each clock can be disabled.

The Clock Controller has the following external inputs:

1. PCLK: External pixel clock input
2. SYSCLK: External system clock input(optional)
3. MXI: Externally attached X'tal oscillator input
4. M48XI: Externally attached 48MHz X'tal oscillator input
5. CLKSEL Internal PLLs are bypassed and SYSCLK is used

In normal operation, a 27MHz signal is applied to MXI and a 48MHz signal is applied to M48XI. The CCD Controller (CCDC) pixel clock is connected to PCLK. The maximum pixel clock frequency is 100 MHz. The DM320 passes-through MXI and M48XI to output signals MXO and M48XO, respectively.

The Clock Controller generates the following 16 signals:

1. CLK_ARM ARM clock for ARM core and its related modules
2. CLK_DSP DSP clock for DSP core and its related modules
3. CLK_AXL Accelerator clock for iMX, DCT, VLCD and Image Buffers
4. CLK_SDR SDR clock for external SDRAM and its related modules
5. CLK_XI X'tal clock such as TMR, WDT, UART
6. CLK_CCD CCD Controller clock
7. CLK_ENC Video Encoder clock
8. CLK_OSD OSD clock
9. CLK_USB48 USB clock
10. CLK_MMC/MS Multimedia Card / Secure Digital clock/Memory Stick
11. CLK_VLYNQ Clock for Vlynq core
12. CLK_I2C Clock for i2c controller
13. CLK_AHB Clock for AHB bus
14. CLKOUT1 Programmable output to GIO
15. CLKOUT2 Programmable output to GIO
16. CLKOUT3 Programmable output to GIO

As shown in Figure 45, the input signal CLKSEL is used to select which clock will be used as the ARM clock prior to power-on and reset. By default, the ARM PLL is bypassed during power-on and reset, and either MXI or SYSCLK are used as the ARM clock. When the state of CLKSEL is low, MXI is used and when the state of CLKSEL is high, SYSCLK is used. After the ARM is rebooted, the ARM can configure several clocks by setting various registers in the Clock controller module.

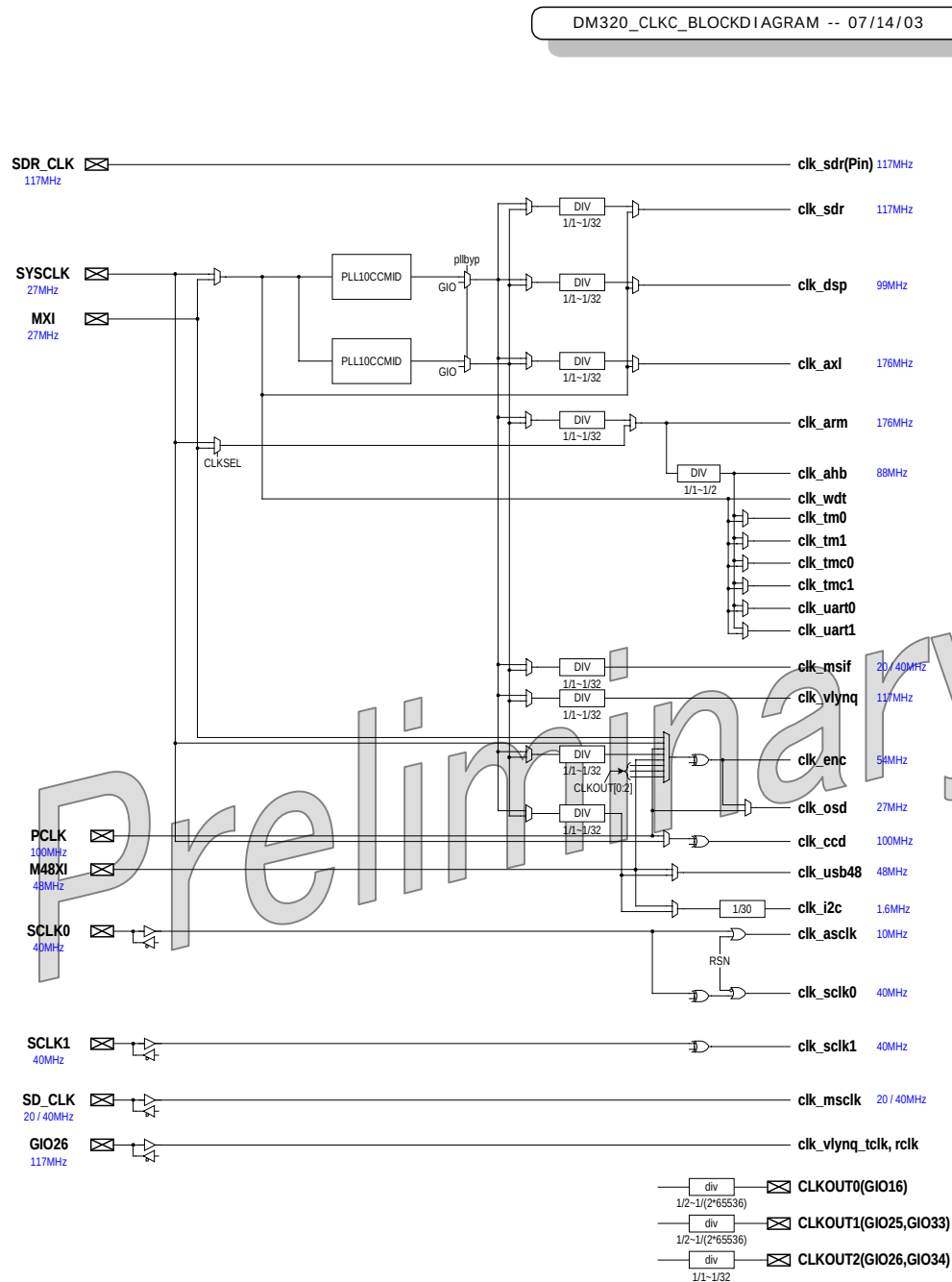


Figure 45 Clock Controller Block

6.8 ARM Boot-up Sequence

DM320 provides three different ARM boot-up sequences. There are CS0, serial boot-up mode, Internal ROM and HOST boot-up modes.

6.8.1 CS0 Boot

Upon power up and reset, the ARM program counter is reset to 0xFFFF:0000, which is mapped to 0x0010:0000 (start of the CS0 address space). The ARM should then execute a boot sequence from CS0 region to start the DM320 system. After turn-off the power on reset signal to first access to CS0(FLASH_CE) region takes about 10 ARM clock cycles. In this mode, the external pin of BTSEL0 and BTSEL1 should be tied to low (gnd) level.

6.8.2 Serial boot-up mode

DM320 is downloadable externally through serial interface(SIF0) and UART(UART0) pins. During reset, the external CPU can access to ARM internal memory by using SIF/UART. This interface consists of four signals serial clock input(SCLK0), enable signal input(SDEN0), serial data input(RXD0) and serial data output(TXD0). The external CPU can write a necessary data to ARM internal memory(AIM), for instance boot code, and read out the data from AIM.

The followings show the features:

- ❑ Full addressing from 0x0 to 0x007F:FFFF accessible
- ❑ Access mode is a byte mode only
- ❑ Address can be incremented automatically after gotten first address
- ❑ Support both of write and read
- ❑ Effectively during power on reset is low level, eg.during DM320 is on reset.
- ❑ Effectively during SDEN0 is low level

Figure 46 and Figure 47 show the timing diagram for serial interface. The first received bit indicates read/write command and it follows 15 bits address bits. When the command bit is low, being the write mode and when the command bit is high, being the read mode. Following the command and address bits, data of 8 bits length can be received in the write mode. In the read mode, data of 8 bits length read from AIM will be transferred through data out pin. The enable signal is active low. The address to be accessed is incremented automatically during enable signal is low. The minimum unit of the transfer is defined 8 bits. If the data are less than 8 bits will be discarded at the last transferring unit. The interface has below protocols.

- ❑ The data to be transferred is MSB first.
- ❑ The maximum frequency of SCLK0 is 10MHz.
- ❑ Performed while the power on reset signal is in assertion.

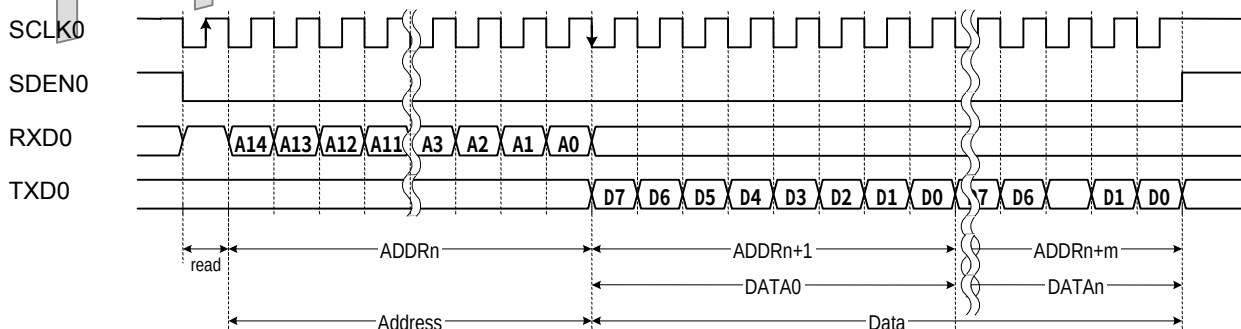


Figure 46 Serial data read timing chart

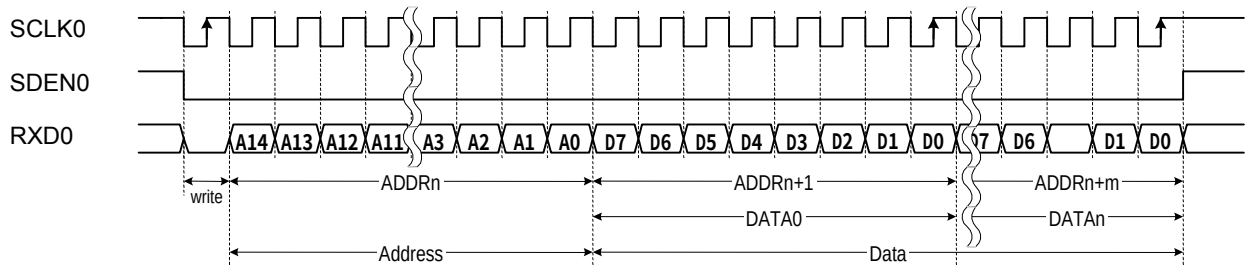


Figure 47 Serial data write timing chart

6.8.3 External CPU boot-up mode

External CPU can access to ARM internal memory directly through EMIF bus pin. It also enables downloading an ARM boot code externally.

6.8.4 Internal ROM boot-up mode

The ARM has 8 KB of ROM which contains boot code.

ARM ROM Content:

- NAND-flash bootloader
- MP3 encoder data table (from DM270)

Bootloader Spec outline:

User defines the following set of information for each code section:

- Source Address (in NAND-flash)
- Destination Address (anywhere in ARM volatile memory map - AIM, SDRAM, etc.)
- Section size (bytes)

It is assumed that, when in NAND-flash/ROM boot mode, the 8KB ROM is mapped to ARM address space (0x0000-0x1FFF).

The ROM bootloader will

- disable interrupts
- read / process these data triplets and copy the data from Source Addr in NAND to Destination Addr for specified number of bytes
- process will continue until data is encountered that indicates 0 bytes to be transferred.
- after all data block are copied, 16KB ARM IRAM is remapped to 0x0000-0x3FFF before transferring control to a start address specified after the data specifications.

Users are responsible for properly defining the application or boot software program in NAND-flash and also the section information.

Hardware configuration

The BTSEL1, BTSEL0 pins defines how ARM will operate after power-on reset. Table 19 shows pin configuration.

Table 19 ARM boot-up configuration

BTSEL1	BTSEL0	Function
0	0	CS0 boot up (External flash)
0	1	Serial/Host
1	0	Internal ROM
1	1	CS0

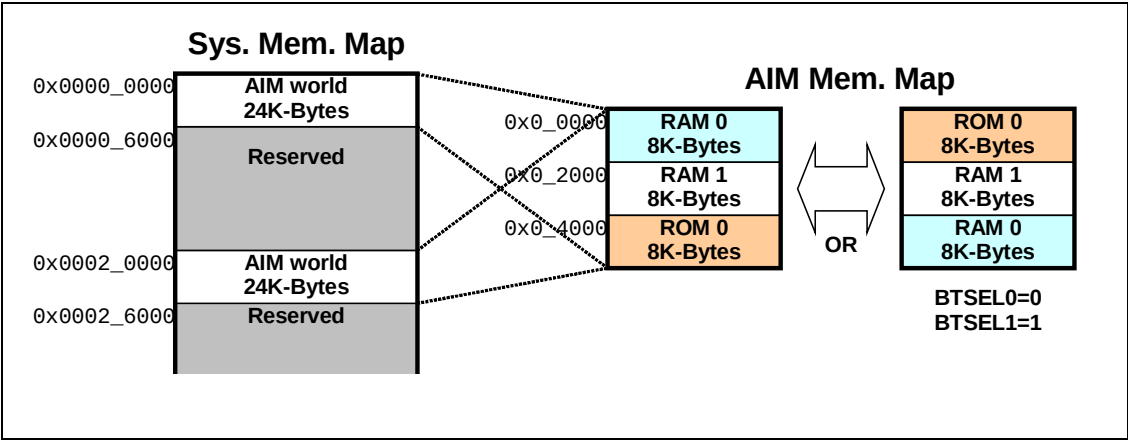


Figure 48 : DM320 AIM Memory Map

Preliminary

7 Memory Traffic Controller

When processing data, it is often necessary to have access to large memory spaces. The internal memory of the DM320 chip is often too small to fit all the data and programs. To solve this problem, the external memory interface controller offers the possibility to access a large amount of external memory. Since the external memory interface is configurable, many different types of memories can be connected to the external interface. In fact the external memory interface includes two specialized interfaces to control and access CompactFlash and SmartMedia cards.

The memory traffic controller(MTC) can be separated into two sub-blocks: the external memory interface (EMIF) and the SDRAM controller(SDRAMC).

Another sub-block of memory traffic controller is DMA controller(DMAC). This DMA controller allows fast and efficient memory transfers between the different external memory regions.

7.1 Memory map

The complete memory map of the DM320 is represented in the Table 20. The external memory region consists of 256 Mbytes of ARM memory space that starts at address 0010:0000h.

Table 20 ARM memory address map

Area Name	Start address	End address	Size(byte)
ARM Internal RAM*	0x0000:0000	0x0000:3FFF	16 K
ARM Internal ROM*	0x0000:4000	0x0000:5FFF	8 K
Internal Peripheral	0x0003:0000	0x0003:0FFF	4 K
DSP Memory	0x0004:0000	0x0005:FFFF	128 K
Internal Peripheral #2	0x0006:0000	0x0006:0FFF	4 K
Coprocessor	0x0008:0000	0x0009:FFFF	128 K
External Memory	0x0010:0000	0x110F:FFFF	256 M
Compact Flash	0x4000:0000	0x40FF:FFFF	16 M
SSFDC	0x4800:0000	0x48FF:FFFF	16 M
External SRAM	0x5000:0000	0x50FF:FFFF	16 M
External SRAM	0x6000:0000	0x60FF:FFFF	16 M
VLYNQ	0x7000:0000	0x73FF:FFFF	64 M
Mirror External Memory	0xFFFF:0000	0xFFFF:FFFF	64 K

After reset is canceled, the ARM program counter jumps to address 0 of the CS0 region.

The memory traffic controller is the main point of communication between the different modules of the DM320. The CCD Controller, Preview Engine, OSD, Image Buffer, HPIB, DMAC, EMIF, and ARM are connected directly to the SDRAM controller. The Image Buffer, HPIB and DMAC can be connected either to the SDRAM controller either to the EMIF. Depending on the memory location accessed, the ARM will either access the SDRAM controller, either the EMIF.

7.2 Blocks Diagram

The DAEM controller is the main point of communication between the different modules of the DM320. The CCD Controller, Preview Engine, OSD, Image Buffer, DMAC, EMIF, and ARM are connected directly to the SDRAM controller. The CCDC can be connected either to the SDRAM controller either to the EMIF. Depending on the memory location accessed, the ARM will either access the SDRAM controller, either the EMIF.

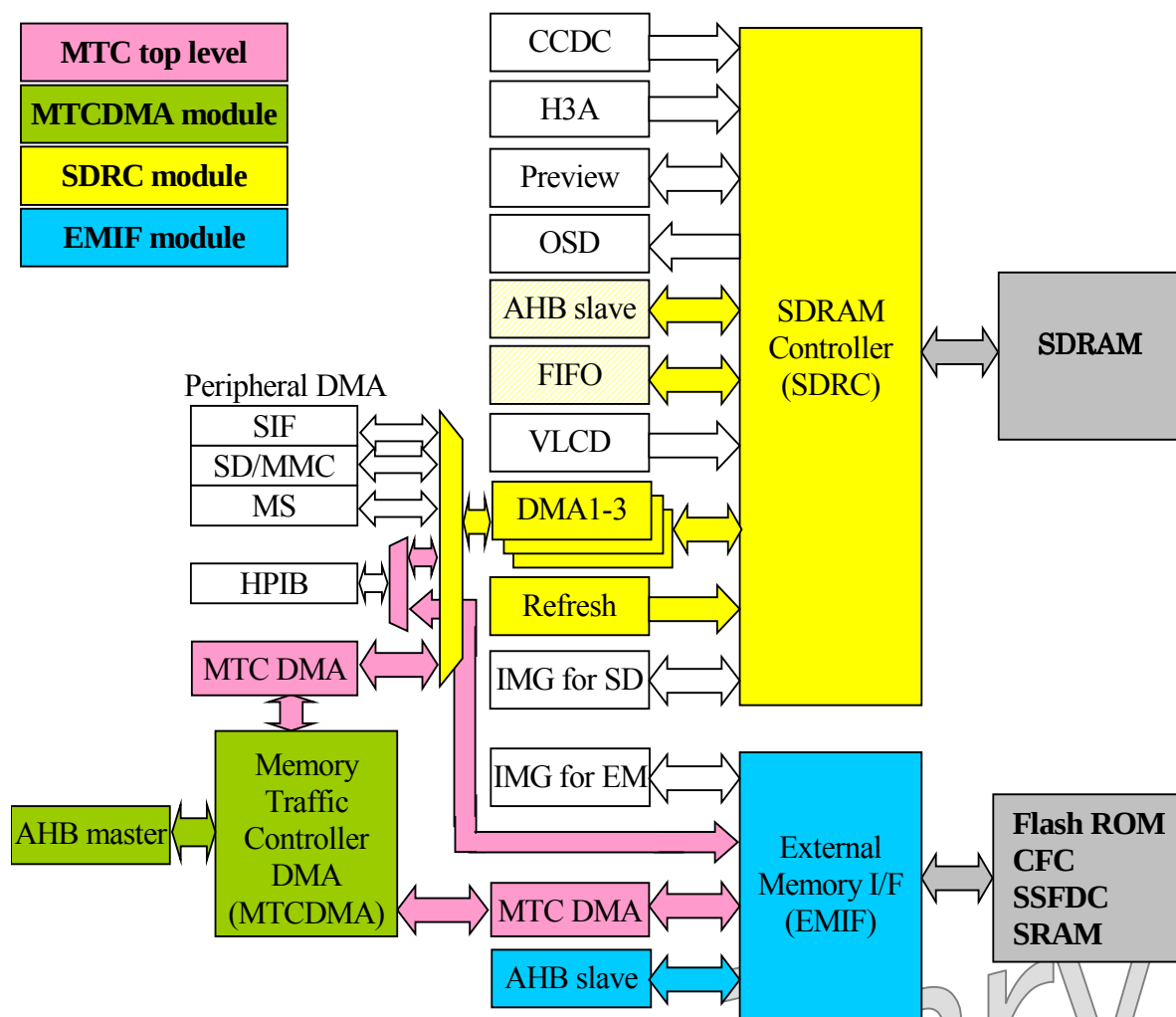


Figure 49 Memory Traffic Controller blocks diagram

7.2.1 Access to Memory Traffic Controller(MTC) Areas

The memory traffic controller can access 128Mbytes of memory separated into 6 regions:

CS0: external memory region 0 (FLASH memory interface)

CS1: external memory region 1 (CompactFlash interface)

CS2: external memory region 2 (SmartMedia interface)

CS3: external memory region 3 (general purpose device)

CS4: external memory region 4 (general purpose device)

SDRAM memory region

Many different types of general-purpose memory can be attached to each region of the external memory interface, such as ROM, EPROM, and SRAM. There are five control output terminals from CS0 through CS4 that can be directly connected with these general-purpose memories without an external decoding circuit, and their timing can be controlled by software. For example, because it is a low-speed device in which output data is not immediately set to high impedance, a bus IDLE state cycle can be input between that write cycle and the next memory access cycle.

Thus, data collision on the data bus can be avoided. All of the interface timing characteristics of each external memory region are configurable. Except for the external flash memory region 0 (CS0), the start address of each external memory region are configurable. Furthermore, the data busses of CS0, CS1, CS3, and CS4 can be set to 8-bits or 16-bits. The data bus width of CS2 is always 8-bits.

An SDRAM memory region is set up in the same way as the other external memory areas.

External memory regions 0, 1, and 2, have special features that allow them to easily be used as a FLASH memory interface, CompactFlash interface and SmartMedia interface, respectively. The CompactFlash and SmartMedia interfaces share some pins, therefore it is not possible to simultaneously access a CompactFlash and SmartMedia card.

7.2.2 Boundary Addresses and Sizes of the DAEM Regions

The external memory area has a total addressable length of 128 Mbytes in the ARM memory map. The external memory area can be divided into a maximum of six regions (SDRAM, CS0-CS4). The boundary address of each memory region is set by specifying the physical address value of the ARM memory space. The boundary address is set in six different boundary address registers (DPSTR0-5). The boundary addresses are specified in units of 1 Mbyte. The default start addresses, which are set after reset, are shown in Table 21

Table 21 Initial values of offset

Variable region name	Start address	Memory Size	Corresponding memory	Corresponding Chip Select
External memory region 0	0010:0000h	0100:0000h	FLASH memory	FLSH_CE
SDRAM region	0090:0000h	1000:0000h	SDRAM	SDR_CS
External memory region 1	4000:0000h	0100:0000h	Compact Flash	CFE1,CFE2
External memory region 2	4800:0000h	0100:0000h	Smart Media	CFE1
External memory region 3	5000:0000h	0100:0000h	General	EM_CE0
External memory region 4	6000:0000h	0100:0000h	General	EM_CE1

If the regions are set so that they overlap, the chip select is valid for the region of higher priority ranking. If possible, try not to overlap memory areas.

Table 22 Chip select signal priority order

Variable region name	Priority
External memory region 0	Low ↑ ↓ High
SDRAM region	
External memory region 1	
External memory region 2	
External memory region 3	
External memory region 4	

7.3 External Memory Interface

7.3.1 Hardware Connection

As external memory interfaces, DM320 has a general-purpose memory interface. The ARM can access SDRAM via the SDRAM controller module. The ARM can access general-purpose SRAM or flash ROM, a CompactFlash card, or a SmartMedia card via the external memory interface module. In access from ARM to external memory, a wait is automatically inserted, and its cycle varies depending on SDRAM frequency, external memory access time and so forth.

The functions of several pins in the of the external memory bus are shown in Table 23.

Table 23 External Memory Bus Pins

Pin Name	I/O	Explanation
ARM_A[22:0]	O	ADDRESS bus Byte address for 8 bit device and half word address for 16 bit device is outputted. ARM_A[22]: -REG for CF/CF+
ARM_D[15:0]	I/O	DATA bus
EM_WE	O	Write enable code (FLSH_CE, CFE1, CFE2, EM_CS0, EM_CS1)
EM_RDWR	O	READ/WRITE Strobe code
EM_BEH	O	Upper byte enable code
EM_BEL	O	Lower byte enable code
EM_OE	O	Read enable code (FLSH_SE, EM_CS0, EM_CS1)
EM_WAIT	I	External wait code (EM_CS0 or EM_CS1)
CFE1	O	CF/CF+ [-CFE1] Smart Media [-CE]
CFE2	O	CF/CF+ [-CFE2] Smart Media [ALE]
CFOE	O	CF/CF+ [-OE] Smart Media [CLE]
CFIORD	O	CF/CF+ [-IORD] Smart Media [-RE]
CFIOWR	O	CF/CF+ [-IOWR] Smart Media [-WE]
CFRDY	I	CF/CF+ [READY/-BUSY] Smart Media [READY/-BUSY]
CFWAIT	I	CF/CF+ [-WAIT]
IOIS16	I	CF/CF+ [-IOIS16]

7.3.2 Timing control

The timing for each external memory region can be configured by various register settings. This allows many different types of devices, which have different interface timing requirements, to be connected to each of the external memory regions. Also, an idle state can be inserted in order to prevent bus collision when performing continuous memory accesses. Table 24, Table 25 and Table 26 show the configurable timing characteristics for each external memory region, and Figure 50 shows the corresponding signal timing diagrams.

Table 24 Configurable Timing Parameters for CS0/CS3/CS4

Parameter	Explanation	clock variable	Variable unit
CYCLE	Cycle no.	2 ~16 clock variable	1 CLK unit
CS_SU	Setup no. of CS	0~15 clock variable	1 CLK unit
WE_SU	Setup no. of WE	0~15 clock variable	1 CLK unit

OE_SU	Setup no. of OE	0~15 clock variable	1 CLK unit
CS_WAIT	Wait no. of. CS	1~16 clock variable	1 CLK unit
WE_WAIT	Wait no. of. WE	1~16 clock variable	1 CLK unit
OE_WAIT	Wait no. of. OE	1~16 clock variable	1 CLK unit
IDLE	Address no.	0~3 clock variable	1 CLK unit

Table 25 Configurable Timing Parameters for CS1

Parameter	Explanation	clock variable	Variable unit
CYCLE	Cycle no.	2~32 clock variable	1 CLK unit
CS_SU	Setup no. of CS	0~15 clock variable	1 CLK unit
WE_SU	Setup no. of WE	0~15 clock variable	1 CLK unit
OE_SU	Setup no. of OE	0~15 clock variable	1 CLK unit
CS_WAIT	Wait no. of. CS	1~32 clock variable	1 CLK unit
WE_WAIT	Wait no. of. WE	1~32 clock variable	1 CLK unit
OE_WAIT	Wait no. of. OE	1~32 clock variable	1 CLK unit
IDLE	Address no.	0~3 clock variable	1 CLK unit

Table 26 Configurable Timing Parameters for CS2

Parameter	Explanation	clock variable	Variable unit
CYCLE	Cycle no.	2~16 clock variable	1 CLK unit
WE_SU	Wait no. of WE	0~15 clock variable	1 CLK unit
OE_SU	Wait no. of OE	0~15 clock variable	1 CLK unit
WE_WAIT	Wait no. of. WE	1~16 clock variable	1 CLK unit
OE_WAIT	Wait no. of. OE	1~16 clock variable	1 CLK unit
READ_SU	Set up no. of read	3~18 clock variable	1 CLK unit
IDLE	Address no.	0~3 clock variable	1 CLK unit

Preliminary

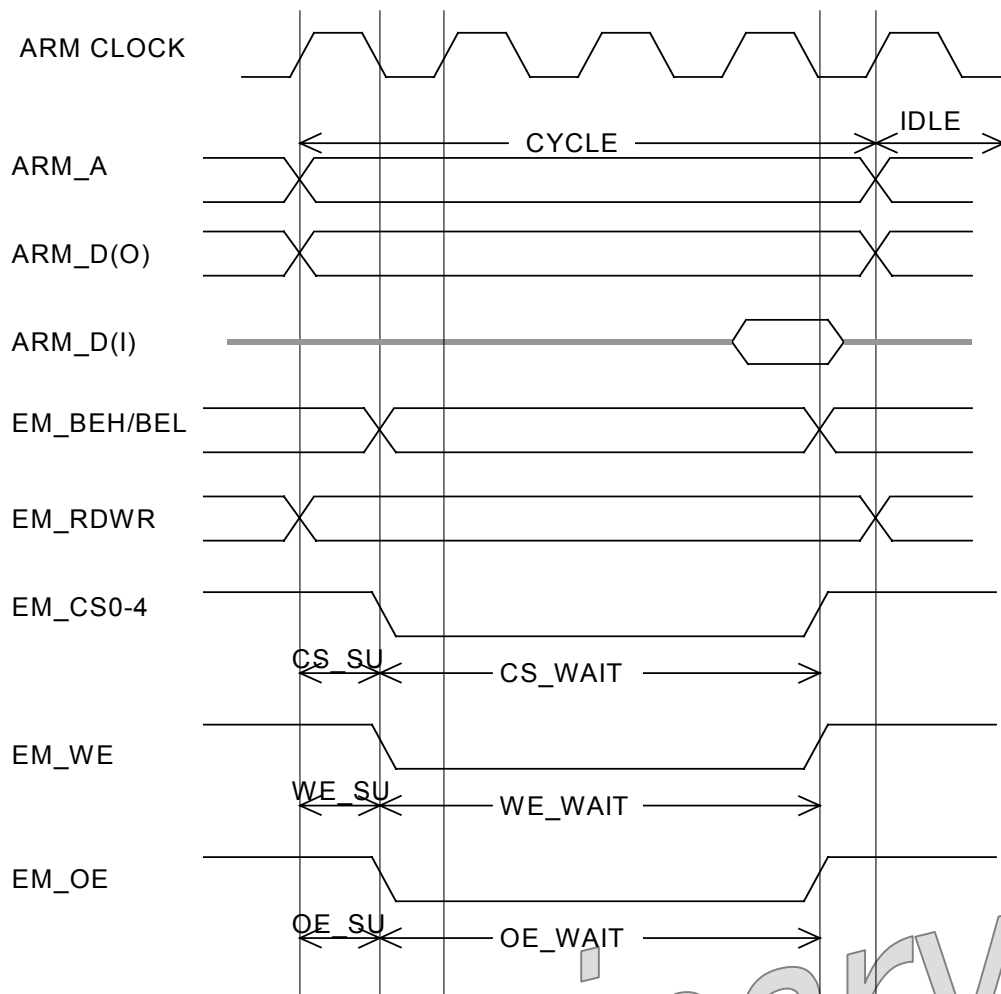
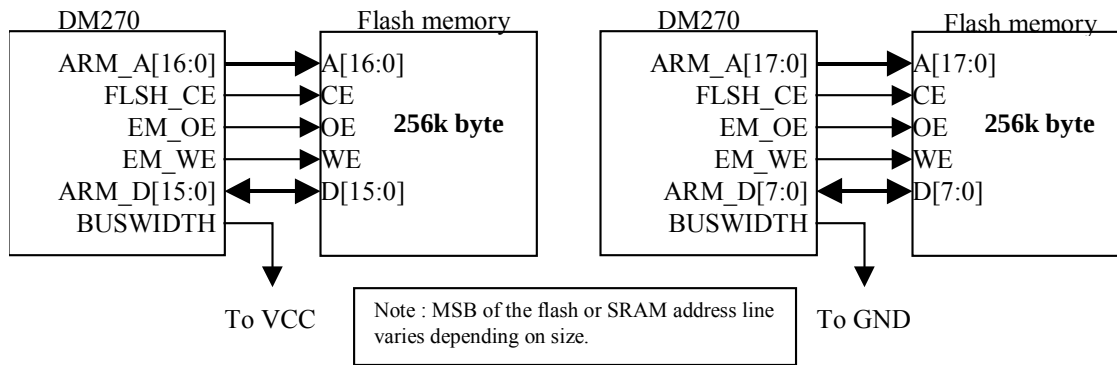


Figure 50 External Memory Bus Timings

7.3.3 External Memory Region 0: The FLASH Memory Interface

Systems that use asynchronous FLASH ROM for program memory should attach the FLASH ROM to external memory region 0, since the ARM reset vector includes an instruction to jump to the start address of external memory region 0. The start address of external memory region 0 is 0x10:0000, and this address cannot be changed.

Figure 51 shows the FLASH memory interface. The FLASH interface uses ARM_A, FLSH_CE, EM_OE, EM_WE, ARM_D, and BUSWIDTH. The write enable signal (EM_WE) is used commonly by all devices connected to the external memory interface. When the ARM accesses the flash region, FLSH_CE becomes active. In addition to FLASH memory, it is also possible to connect SRAM or PROM devices to the FLASH interface.

**Figure 51 Flash Memory Interface Data Bus**

External memory region 0 supports 8-bit and 16-bit wide FLASH. As shown in Table 27, the BUSWIDTH pin is used to set the data bus width of the region 0 to 8 or 16 bits.

Table 27 FLASH_CE Bus Width

BUSWIDTH state	FLASH_CE Bus Width
Low	8-bit
High	16-bit

Although the FLASH data bus width can be set to 8 or 16-bits, a system may need to access 8, 16, and/or 32-bit data from the FLASH memory. As shown in Table 28, the external memory interface will automatically perform the correct number of accesses for an 8, 16, or 32-bit accesses, given an 8 or 16-bit wide FLASH memory. When using an 8-bit wide FLASH memory, 1 access is required for an 8-bit access, 2 accesses are required for a 16-bit access, and 4 accesses are required for a 32-bit access. When using a 16-bit wide FLASH memory, 1 access is required for an 8-bit access, 1 accesses are required for a 16-bit access, and 2 accesses are required for a 32-bit access. To perform an 8-bit read from a 16-bit wide FLASH memory, the required 8-bit portion is loaded and the remaining 8-bits are.

Table 28 Relationship Between Access Size and Flash Memory Data Bus Width

Access Size	8 bits wide	16 bits wide
8-bit access	1 time	1 time
16-bit access	2 times	1 time
32-bit access	4 times	2 times

7.3.4 External Memory Region 1: The CompactFlash Interface

External memory region 1 can be used as to interface CompactFlash cards. The CompactFlash interface is shown in Figure 52. The interface supports memory mode and IO mode, however, true IDE mode is not supported.

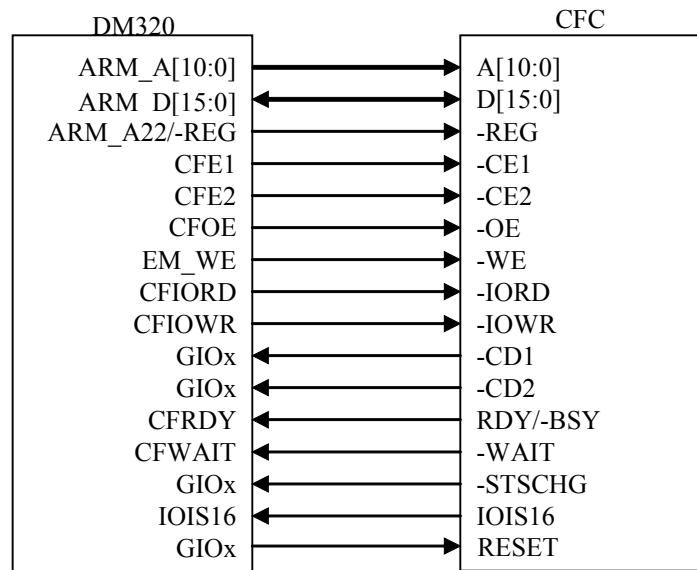


Figure 52 CompactFlash Interface

The CompactFlash interface supports dynamic bus sizing. The data bus width will be 8-bits or 16-bits depending on the level of the IOIS16 pin. When IOIS16 is low, the CompactFlash interface operates with an 8-bit bus, and when IOIS16 is high, the interface operates with a 16-bit bus. Similar to the FLASH interface, the correct number accesses are automatically performed when 32-bit, 16-bit, and 8-bit data is transferred, given that the data bus is set to be 8 or 16-bits.

The CompactFlash Interface Memory Map is shown in Table 29. The table shows the offset address for the CFC memory space and the CFC IO space. The ARM software accesses the memory space and IO space starting from the offsets at 00:0000 and 00:0800 respectively. The offset addresses are relative to the start address of external memory 1.

Table 29 CompactFlash Interface Memory Map

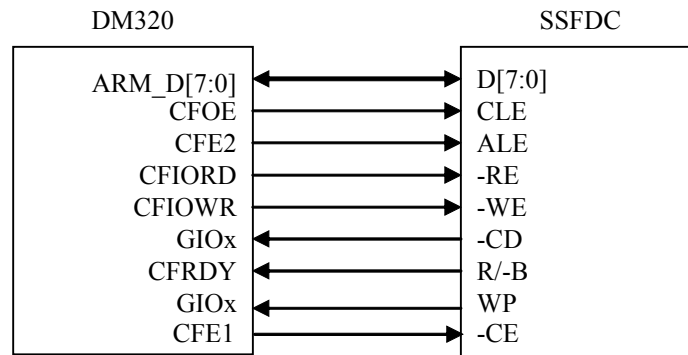
External Memory Area	Offset Address	Size	CompactFlash Interface Function
CS1	00:0000 – 00:07FF	2-Kbytes	CFC Memory Space (Attribute and Common)
	00:0800 – 00:0FFF	2-Kbytes	CFC IO Space

Accessing the CompactFlash memory space (attribute and common memory region)

When the CompactFlash card memory space is accessed (when operating in memory mode), the REG signal controls which region is accessed. The REG signal is output on pin ARM_A22. When REG is low in memory mode, the attribute memory region can be accessed. When REG is high in memory mode, the common memory region can be accessed. When operating in I/O mode, the REG signal is always output.

7.3.5 External Memory Region 2: The SmartMedia Interface

External memory interface 2 has a built-in SmartMedia interface. Figure 53 shows the SmartMedia interface. The interface uses ARM-D[7:0], ARM_A20, ARM_A21, CFIORD, CFIOWR, CFRDY, and any available external GIO pin. See the TMS320DM320 for more detail on the SmartMedia interface.

**Figure 53 SmartMedia Interface**

Since SmartMedia uses one 8-bit data bus for command, address, and data, external memory interface 2 is an 8-bit wide interface. Similar to the FLASH interface, when 32-bit, 16-bit, and 8-bit data is transferred, an 8-bit access is automatically generated 4 times, 2 times, and 1 time respectively.

The SmartMedia Interface Memory Map is shown in Table 30. This table shows the offset address for command, address, and data. The ARM software should write SmartMedia commands to the address offset at 00:0000, write SmartMedia addresses to offset 00:0004, and should read and write SmartMedia data from offset 00:0008. The offset addresses are relative to the start address of external memory region 2.

Table 30 SmartMedia Interface Memory Map

External Memory Area	Offset Address	Size	Smart Media Interface Function
CS2	00:0000	4-bytes	Smart Media Command Write
	00:0004	4-bytes	Smart Media Address Write
	00:0008	4-bytes	Smart Media Data Read/Write

The DM320 supports SmartMedia error detection and correction.

The SmartMedia interface supports hardware error detection. During a read, hardware error correction code (ECC) computation is performed every 256-bytes. The result of the computation can be read from ARM register in EMIF. Although the hardware performs ECC computation, the ARM software must perform ECC data correction. If 512+16 byte page smart media is used, 256 bytes of data is read, then the ECC data is evacuated all at once, and after the ECC data is cleared.

7.3.6 External Memory Regions 3 and 4

The width of external memory areas 3 and 4 can be either 8 or 16 bits.

The EM_WAIT signal is available for region 3 and region 4, which enables asynchronous device interface capability. This feature is not available in any other external memory region. When EM_WAIT is enabled for CS3 or CS4, each time one of the regions is accessed. If EM_WAIT is driven low during a bus cycle by an external device, an extra wait state (hardware wait-state) is added to the programmed wait states (software wait-states).

For instance, if CS_WAIT = 5 and EM_WAIT is held low for 5 clock cycles, the total number of wait states will be 10.

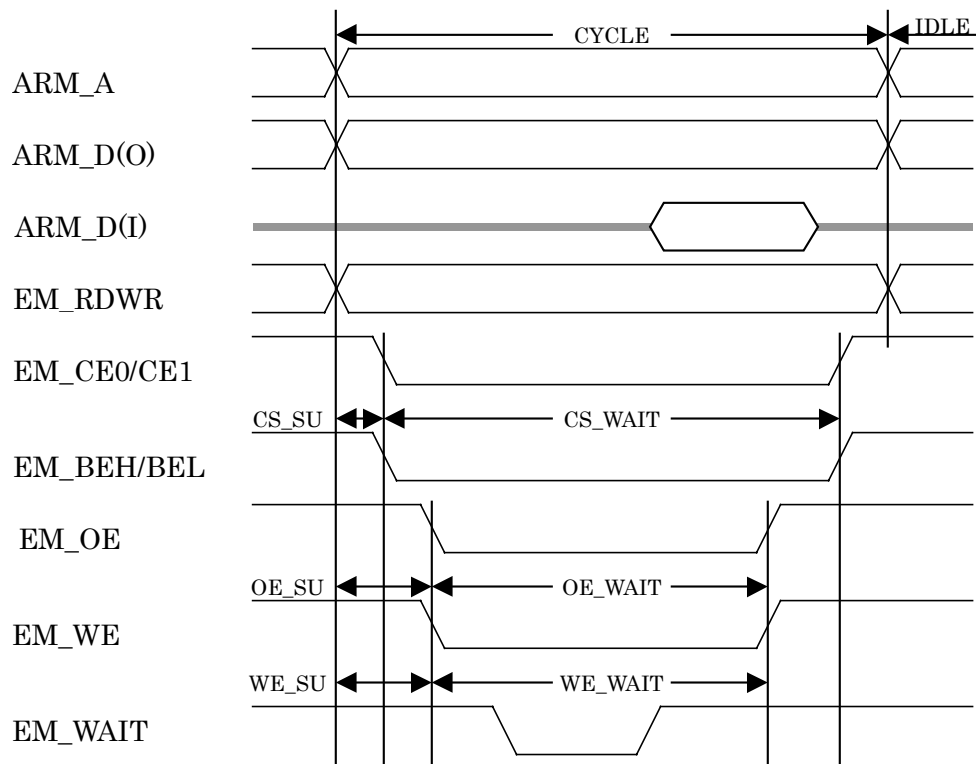


Figure 54 External memory bus timing with hardware wait-states

The EMIF bus can be opened for an external device by software. By setting the external memory bus open register in the external memory interface module, all I/F signals (ADDRESS/DATA/CS*/WE/OE) are opened (output Hi-Z). By using this function, it is possible to share memory among external memories. In addition, in DM320, the SDRAM bus can be opened to an external device. All SDRAM buses (ADDRESS/DATA/RAS/CAS/CS/WE/DQM/CKE/CLK) can be opened (output Hi-Z) by setting the SDRAM bus open registers in the external memory interface module. By using this function, SDRAM can be shared between external devices or can be tuned off the power of SDRAM.

7.4 SDRAM controller

The SDRAM controller, shown in Figure 55, acts as the primary interface between SDRAM and all functional blocks such as the processors (ARM, DSP), CCDC, Preview Engine, OSD and DMA controller. This block supports SDRAM timing of a maximum of 100 Mhz, and provides continuous data access with low overhead. The start address and size of the SDRAM memory region are configurable within the external memory region of the ARM memory map. The SDRAM controller supports a DMA controller interface. The SDRAM access priority can be modified. The SDRAM controller supports refresh commands and can place the device in a self-refresh mode prior to go in a power down mode. The refresh interval is programmable. The latency for the column access strobe (CAS) is programmable 2 or 3.

7.4.1 Features:

- ❑ Supports 16/64/128/256/512/1024/2048 M SDRAM
- ❑ Supports both 32-bit-wide SDRAM or 16-bit-wide SDRAM
- ❑ Maximum 100 MHz operation
- ❑ Access in WORD, HALF WORD, and BYTE units
- ❑ Mode setting, power down and self-refresh commands
- ❑ Programmable auto-refresh interval
- ❑ CAS latency 2 or 3 cycles can be selected
- ❑ Maximum SDRAM size 256 Mbytes
- ❑ DMA transfer with peripheral modules
- ❑ Variable SDRAM access priority
- ❑ SDRAM Controller Access

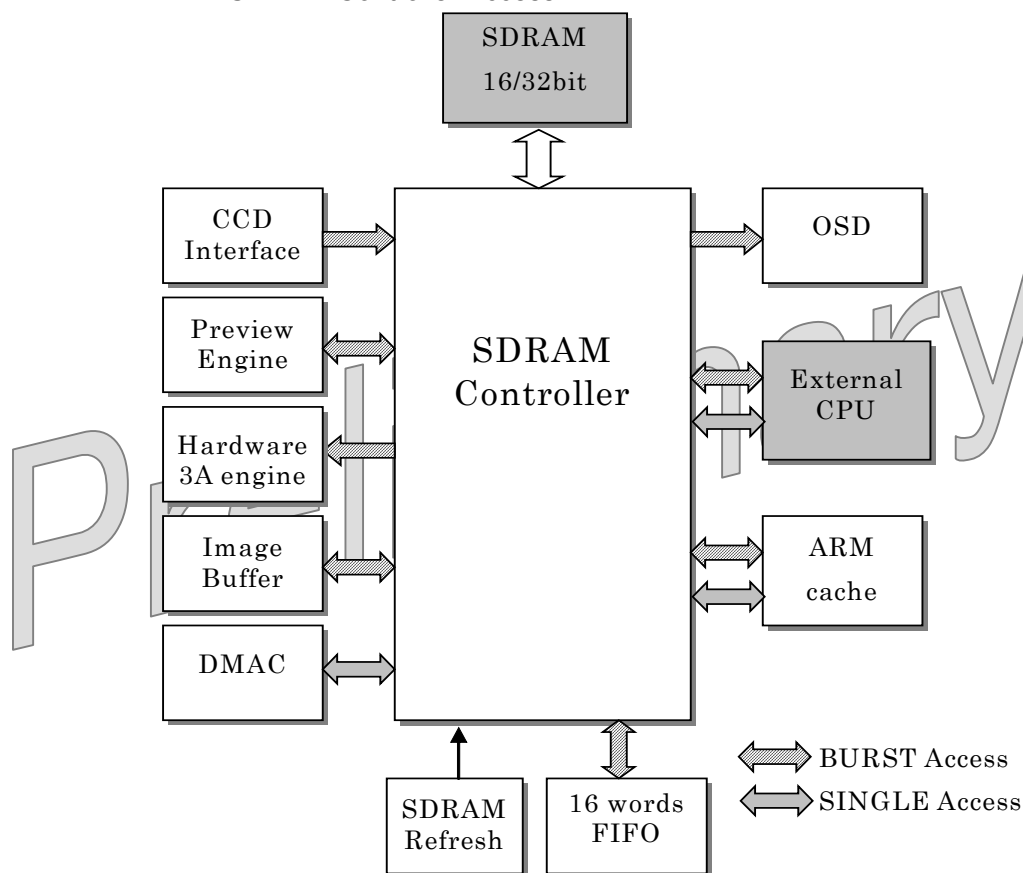


Figure 55 SDRAM Controller Block

The SDRAM region can be set within the external memory region, similar to the other external memory regions 0-4. The region is set by specifying the physical address of the external memory region in the start address register. After reset, it becomes 0x900000h.

The SDRMA controller can access 2048-Mbits (256-Mbytes) of SDRAM and supports both 16 and 32-bit SDRAM devices. Each access can be done in word, half, word or byte units from the ARM side.

7.4.2 Hardware Connection

The functions of several pins in the of the SDRAM bus are shown in Table 31.

Table 31 SDRAM Bus Pins

Pin Name	I/O	Explanation
SDR_D[31-16]	I/O	SDRAM data bus (32bit mode)
SDR_D[15-0]	I/O	SDRAM data bus (16/32bit mode)
SDR_A[12-0]	O	SDRAM address bus
SDR_A[14-13]	O	SDRAM bank address
SDR_RAS	O	SDRAM Row address strobe
SDR_CAS	O	SDRAM Column address strobe
SDR_WE	O	SDRAM Write enable
SDR_CS	O	SDRAM Support chip select
SDR_CKE	O	SDRAM clock enable
SDR_DQM3	O	SDRAM mask for DQ[31:24]
SDR_DQM2	O	SDRAM mask for DQ[23:16]
SDR_DQM1	O	SDRAM mask for DQ[15:8]
SDR_DQM0	O	SDRAM mask for DQ[7:0]
SDR_CLK	O	SDRAM clock

7.4.3 Memory Structure Setting

The DM320 can connect to SDRAM of a maximum of 2048 Mbits (256-Mbytes). The address that is connected differs depending on the memory structure of the SDRAM, as follows.

Table 32 Correspondence between memory structure and address

Memory structure	ADDRESS	Number of banks	BANK	Size in Mbyte	Size in Mbit
2k x 256 WORD	SDR_A10-SDR_A0	2	SDR_A13	4	32
		4	SDR_A14:13	8	64
4k x 256 WORD	SDR_A11-SDR_A0	2	SDR_A13	8	64
		4	SDR_A14:13	16	128
4k x 512 WORD	SDR_A11-SDR_A0	2	SDR_A13	16	128
		4	SDR_A14:13	32	256
8k x 512 WORD	SDR_A12-SDR_A0	2	SDR_A13	32	256
		4	SDR_A14:13	64	512
8k x 1024 WORD	SDR_A12-SDR_A0	2	SDR_A13	64	512
		4	SDR_A14:13	128	1024
8k x 2048 WORD	SDR_A12-SDR_A0	2	SDR_A13	128	1024
		4	SDR_A14:13	256	2048

7.4.4 SDRAM command

The ARM can issue the following commands to SDRAM by settings in the registers in the SDRAM controller :

MRS (mode register set)
 PREA (pre-charge all)
 REF (auto refresh)
 SELF (self refresh entry)
 SELFC (self refresh end)
 PDN (power down mode)

When an MRS command is issued, the following data settings are performed in the SDRAM mode register.

MRS Value

SDRAM Address pin	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
Meaning	Reserved				0	0	Read Latency			S/I	Burst length		
DM320 output value	0	0	0	0	0	0	0	CL	0	0	0	BST 1	BST 0

7.4.5 SDRAM Initialization

SDRAM has a power on procedure. Since the procedure differs depending on the SDRAM that is used, SDRAM must be initialized by software using the SDRAM command. Also, since the DQM (input/output mask) signal is high output as default, after the mode register set command (MRS) is finished, DQM must be set to low level. These SDRAM settings are performed by setting in registers in the SDRAM controller.

A typical sequence is given below:

Apply power and start the clock. Attempt to maintain the NOP command at the input.

Maintain stable power, stable clock, and the NOP condition for a minimum of 100us or 200us. (Minimum time depends on devices. Please refer to the datasheet.)

Pre-charge all banks by the Pre-charge All (PREA) command.

Assert a minimum of 2 or 8 Auto-refresh commands. (Assert times depend on the devices. Please refer to the datasheet)

Program the mode register by the Mode Register Set command (MRS).

7.5 MTC DMA Controller

The Memory Traffic Controller has a built-in DMA controller. As shown by the solid arrows in Figure 56 EMIF DMA Controller, the MTC DMA controller can transfer data between SDRAM and external memory regions 0-4 via AHB. This diagram also shows that data can be transferred between external memory and AHB bus slave like an external host I/F, ARM TCMs, SDRC or EMIF itself and so on.

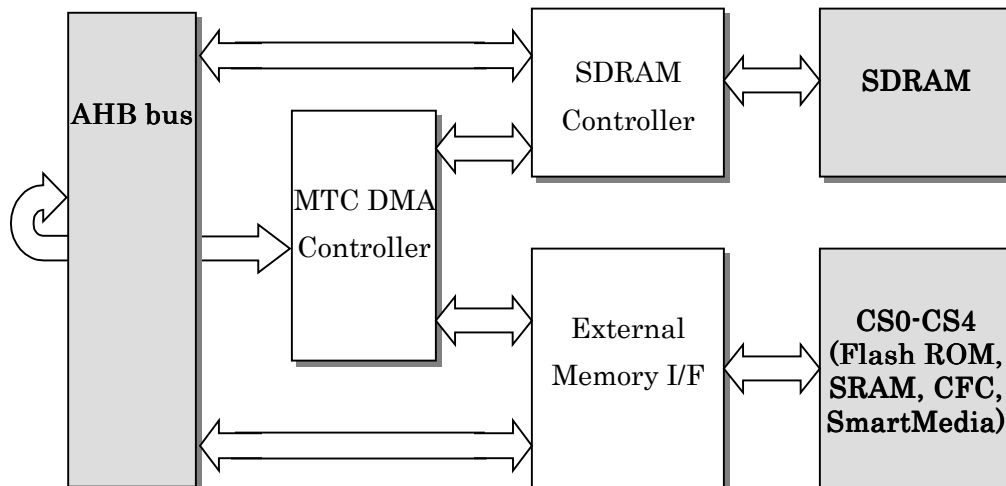


Figure 56 EMIF DMA Controller

7.5.1 Sources and destinations devices

To setup a DMA transfer using the MTC's DMA controller you must configure the MTC's device, the AHB and MTC's device start addresses, and the transfer size.

Table 33 DMA transfer source, transfer destination devices

Transfer AHB slave device	Transfer MTC device
ARM Internal Memory	External general-purpose memory CS0
External HOST Interface	External general-purpose memory CS1
USB Controller	External general-purpose memory CS2
External Memory Interface(EMIF in MTC)	External general-purpose memory CS3
SDRAM Controller(SDRC in MTC)	External general-purpose memory CS4
	SDRAM

7.5.2 Sources/Destinations Addresses and Transfer Size

The registers to configure the DMA start address for AHB slave and MTC's device as well the transfer direction and the transfer size are shown in the following figures and diagram. The MTC's device address is relative to the start of the each region used, and the addresses must be a multiple of four. Also, the Transfer size must always be specified in a multiple of 4. The AHB slave address is absolute address in ARM memory map. The direction bit determine source and destination.

Register	Description
ARMADDR1,2	AHB Slave Start Address. The Absolute Address.
MEMADDR1,2	MTC's device Start Address. The Relative Address.
DMASIZE	Transfer Size in Word(4 byte).
DMAMEMSEL	MTC's device select. CS0-4, SDRAM
DMACTL	Configure Endian, Transfer Direction and DMA Start.

DSP's Image Buffer and HPIB have a built-in DMA controller that can transfer data to the EMIF in addition to SDRAM the SDRAM controller.

7.6 SDRAM & External Memory I/F Pin Sharing

The sharing of pins between SDRAM and flash memory is carried out on bus cycle by bus cycle basis. Each of SDRAM interface and flash memory interface has a separate chip select. When a bus cycle falls into the SDRAM's address space, the chip select of the SDRAM is asserted and the shared pins generate the SDRAM-related timing control signals to perform the cycle. Similarly for flash memory, if the cycle is its address space, its chip select is asserted and the shared pins generate the signals to perform the cycle. Note that it is possible that while ARM is accessing the flash memory (for example, to fetch an instruction), there may be concurrent DMA activities on the SDRAM. In such case, flash memory and the DMA will have to arbitrate for the control for the memory interface.

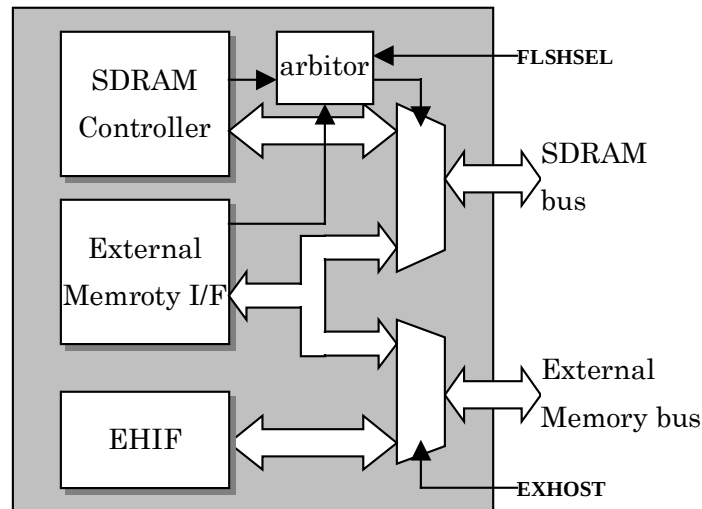


Figure 57 Pin sharing

Table 34 shows DM320 pins that are shared between SDRAM and the External Bus (EXBUS). All the logic for multiplexing SDRAM interface signals and the EXBUS interface signals to the I/O pad is done by internal register and FLHSSEL/EXHOST pins. Table 36 shows the pin list which shared between SDRAM bus and external memory bus.

There are two types of usage for memory resource in DM320. The first type is the usage for the SDRAM memory, which is the same as DSC25. The masters of the SDRAM memory is the SDRAM controller (SDRC). The second type is the usage of SDRAM interface pins but not only the SDRAM memory. SDRC or External Memory Interface (EMIF) are the master that uses SDRAM bus pins and they are arbitrated. With this, a scheme can be implemented to allow the pins to be shared by the SDRAM external and EXBUS interface.

Pin sharing is required to support the optional media storage card hot plug within the constraint of pin count. The basic idea of hot plug is to have two separate buses for the media storage card (for example Compact Flash Card) and the flash memory so that plugs and unplug of the media card does not interfere with the normal operation of flash memory.

Another pin sharing method is required to support external HOST CPU connection with external memory bus. The external CPU can be a bus master by releasing all the external memory bus to it.

Table 34 Shared Memory Configuration

	FLHSSEL*	BUSSEL*	SDRAM	Flash ROM	CFC/SSFDC/SRAM
MODE1	Low	0	SDRAM bus	Ex. Memory bus	Ex. Memory bus
MODE2	High	0	SDRAM bus	SDRAM bus	Ex. Memory bus

MODE3	High	1	SDRAM bus	SDRAM bus	SDRAM bus
-------	------	---	-----------	-----------	-----------

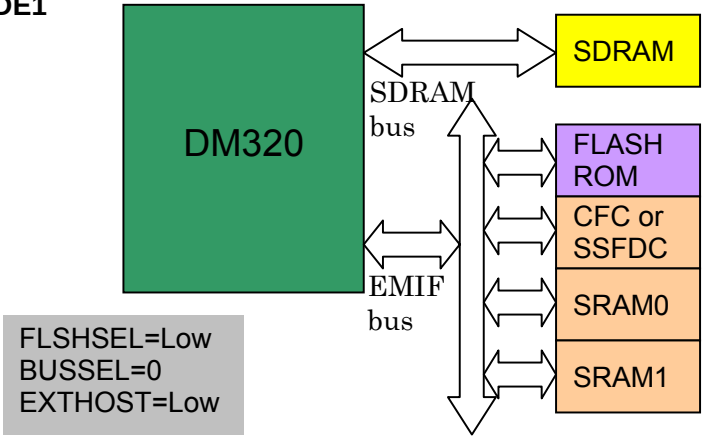
*FLSHSEL: pin name BUSSEL: CPU registers in EMIF module

Table 35 External HOST CPU Configuration

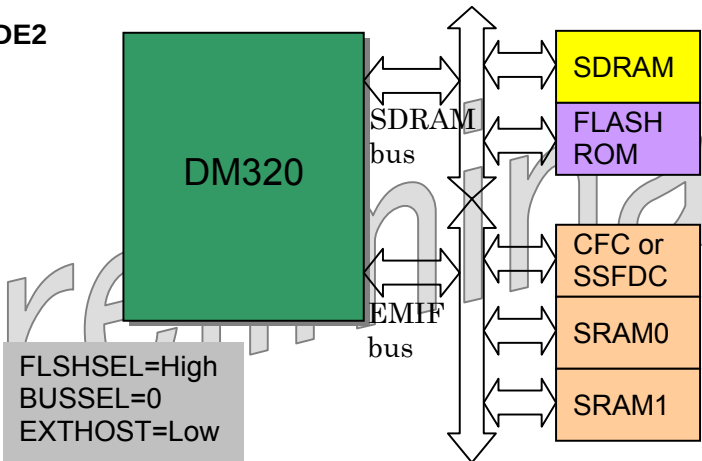
EXTHOST*	External Host CPU
Low	Not available
High	Available

*EXTHOST: pin name

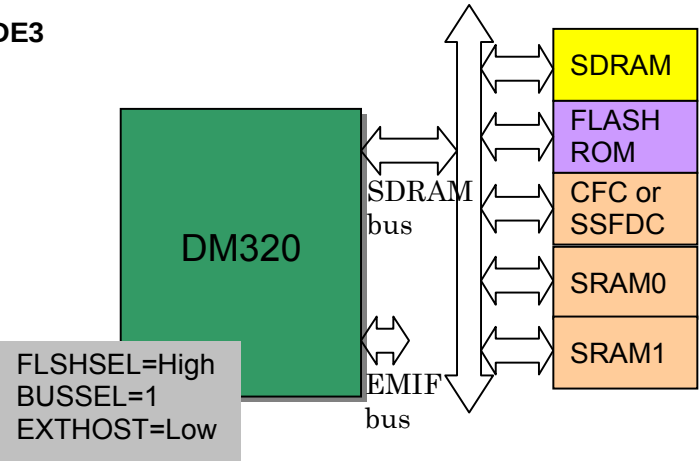
MODE1



MODE2



MODE3



MODE3+External HOST CPU

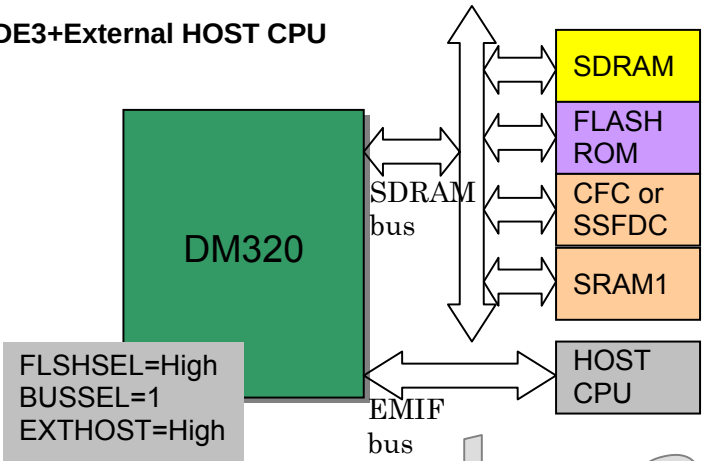


Figure 58 Memory connection on DM320

Table 36 Memory Bus Connection

Pin name	External Memory bus
SDR_DQ23	ARM_A22
SDR_DQ22	ARM_A21
SDR_DQ21	ARM_A20
SDR_DQ20	ARM_A19
SDR_DQ19	ARM_A18
SDR_DQ18	ARM_A17
SDR_DQ17	ARM_A16
SDR_DQ16	ARM_A15
SDR_A14	ARM_A14
SDR_A13	ARM_A13
SDR_A12	ARM_A12
SDR_A11	ARM_A11
SDR_A10	ARM_A10
SDR_A9	ARM_A9
SDR_A8	ARM_A8

SDR_A7	ARM_A7
SDR_A6	ARM_A6
SDR_A5	ARM_A5
SDR_A4	ARM_A4
SDR_A3	ARM_A3
SDR_A2	ARM_A2
SDR_A1	ARM_A1
SDR_A0	ARM_A0
SDR_DQ15	ARM_D15
SDR_DQ14	ARM_D14
SDR_DQ13	ARM_D13
SDR_DQ12	ARM_D12
SDR_DQ11	ARM_D11
SDR_DQ10	ARM_D10
SDR_DQ9	ARM_D9
SDR_DQ8	ARM_D8
SDR_DQ7	ARM_D7
SDR_DQ6	ARM_D6
SDR_DQ5	ARM_D5
SDR_DQ4	ARM_D4
SDR_DQ3	ARM_D3
SDR_DQ2	ARM_D2
SDR_DQ1	ARM_D1
SDR_DQ0	ARM_D0
SDR_RAS	EM_OE
SDR_CAS	EM_WE
SDR_DQ26	EM_RDWR
SDR_DQ25	EM_BEH
SDR_DQ24	EM_BEL

Preliminary

8 Communication Interface

8.1 USB 2.0 Full Speed OTG

DM320 incorporates a USB 2.0 (Full Speed) compliant OTG controller.

8.1.1 Features

- ❑ Compliant with USB Specification Revision 2.0
- ❑ Support for Full speed device and mini host
- ❑ Built-in USB Xceiver
- ❑ Built-in 4K RAM for Endpoint FIFO
- ❑ Dynamic FIFO sizing
- ❑ DMA transferring capability
- ❑ Handles session negotiations (external components required)

Preliminary

8.1.2 USB Functional Description

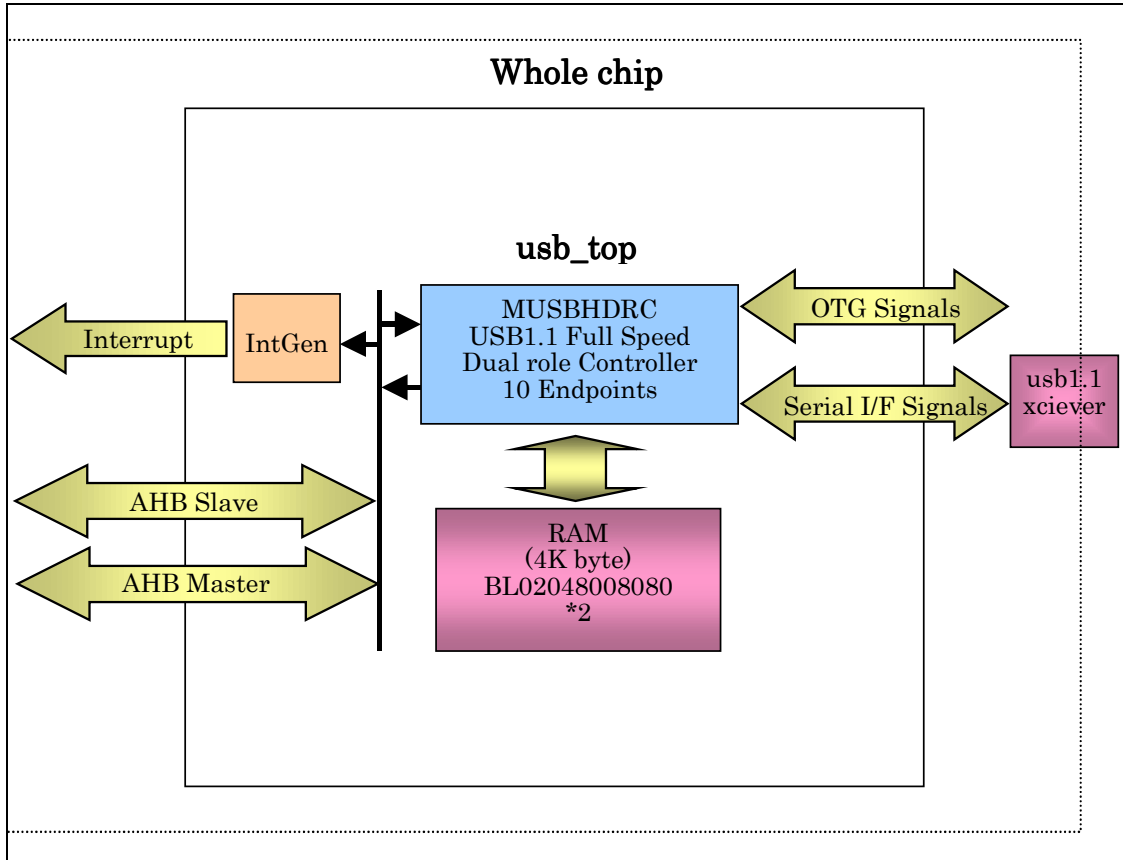


Figure 59 Block Diagram of USB

8.1.3 USB Device Configuration

The USB controller supports flexible end point configuration.

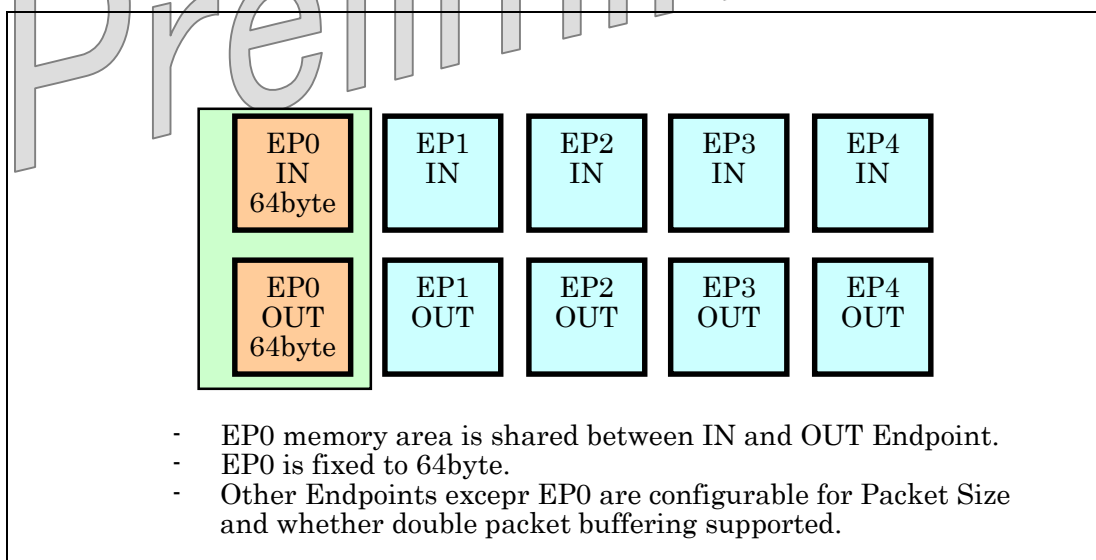


Figure 60 Device Configuration

8.1.4 USB Power Management

When USB bus is in suspend, all the clocks used in USB module except a clock used for resume detection can be cut-off by register for power saving. The clocks will be automatically activated as soon as the resume signal is detected.

If USB is not used, user can completely disable all the USB clocks by a register in CLK Controller module. In that case, more power saving can be achieved by connecting VDD pins for USB I/O to GND.

8.1.5 DMA Data Transfer

Internal DMA controller can be used to transfer data between SDRAM and USB endpoints without ARM intervention.

A DMA transmission from/to one endpoint FIFO is allowed in one time. User has to choose one endpoint as DMA target endpoint preceding DMA transfer. User should also specify SDRAM base address and transfer length. Then writing '1' to DON bit in DMATRG register starts DMA transmission. After the transmission of the specified amount of data is completed, ARM is notified the DMA completion by an interrupt.

For OUT endpoints, data stored in OUT FIFO can be transferred to SDRAM. If no data is received in FIFO, setting DON register does not start DMA. Also the number of data in OUT FIFO is less than DMA transfer length specified by user, the actual DMA transfer length is automatically clipped to actual length.

For IN endpoints, data to be sent to HOST can be transferred from SDRAM. If there is no space for the IN FIFO, setting DON register has no effect. Also the room for the FIFO is less than the specified DMA length, the actual DMA transfer is clipped by the actual room length.

User can also change DMA Endean. Options are LSB byte first, MSB byte first or byte swap in half-word.

8.2 VLYNQ

VLYNQ is a high speed serial link that gives the DM320 the flexibility to extend its capabilities. VLYNQ's throughput (up to 400 Mbits per second) and low pin count will enable new interfaces (IEEE1394, PCI, 802.11, etc.) to be seamlessly added. The DM320 will implement the full nibble wide version of VLYNQ and DMA access to and from SDRAM to provide the greatest flexibility and highest throughput.

VLYNQ is an extension of the internal VBUSP so external VLYNQ peripherals appear to the ARM in the same way as integrated peripherals. This will allow popular VLYNQ peripherals to be integrated in the future with minimal impact on firmware.

8.3 SD/MMC Interface

As shown in Figure 61, the MMC passes data between the ARM and the DMA controller on one side and a memory card or cards on the other side. The ARM or the DMA controller can read from or write to the registers in the MMC Controller. As necessary, the ARM and/or the DMA controller can store or retrieve data in memory.

The MMC Controller initiates transfers between itself and the memory card(s). The communication follows either the controller's native MMC protocol or a protocol that is based on the SPI standard. The native protocol can use one bi-directional data line (for MMC) or four parallel data lines (for SD cards). The SPI protocol uses two serial data lines: one for storing data to the card and one for retrieving data from the card.

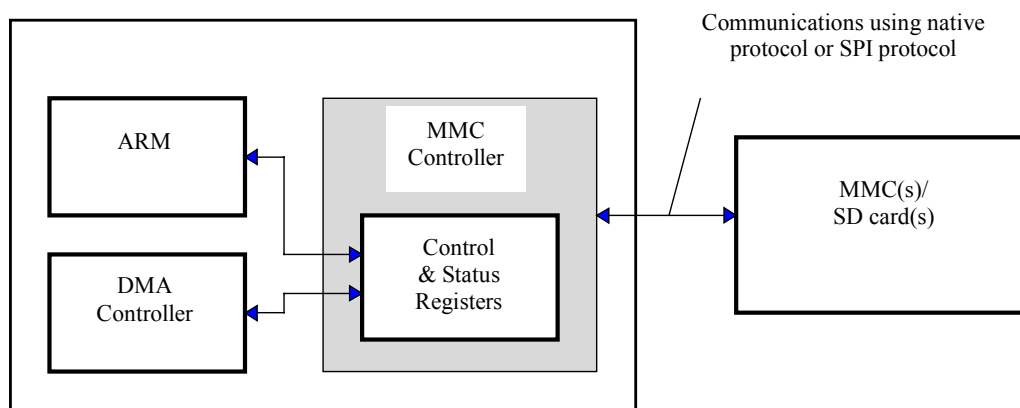


Figure 61 Role of the MMC Controller

The MMC Controller includes the following features:

- ❑ Support for a Multimedia Card (an MMC) or a Secure Digital Memory Card (an SD card)
- ❑ The capability to use either the MMC/SD protocol or the SPI protocol
- ❑ Software-oriented implementation for future extensions
- ❑ A programmable frequency for the clock that controls the timing of transfers between the MMC Controller and the memory card.
- ❑ Support for DMA transfer between MMC Controller and SDRAM

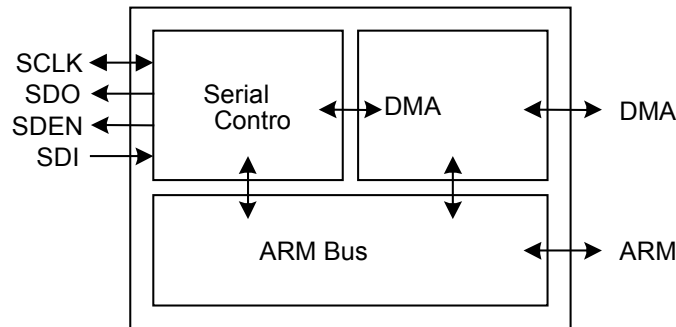
8.4 Memory Stick Interface

- ❑ V1.31 (4 bit data) support
- ❑ Supports MS PRO V1.0
- ❑ Supports DMA transfer to and from SDRAM

8.5 Serial Port

The DM320 has a two-channel clock synchronized serial port interface (SPI). The transfer unit is 8 bits each time, and it is a simple three-line serial interface consisting of a clock line, output data line, and input data line. Serial port 0 has a DMA transfer function with SDRAM for transfer of large quantities of data. DM320 SPI has master mode and slave mode and which is selectable. The SPI can supply the serial clock when it is used in a master mode. The SPI also can be synchronized by the external serial clock input when it is used in a slave mode.

Both of the serial ports and the DMA channel are controlled by ARM registers.



8.6 UART

DM320 supports a two-channel two-line (RX, TX) start-stop synchronization serial port (UART). A 32-byte FIFO is built in for each the transmitting side and receiving side, and interrupts can be generated due to causes such as errors or various states of FIFO. Data is 8 or 7 bits. Even, odd or no parity bit can be selected, and 1 or 2 stop bits can be selected. Transmission/reception timing uses the ARM or X'tal clock. The maximum baud rate achievable is a 1/16 of the ARM or X'tal clock (in bps).

The UART module detects parity errors and overrun errors. Interrupt to ARM is generated when any of the following events occur:

- ☐ Amount of data in receiving-side FIFO exceeds set value
- ☐ Amount of data in transmitting-side FIFO exceeds set value
- ☐ Word received has an error
- ☐ Change occurs in DSR or CTS input(one channel only)
- ☐ The timeout period is reached

ARM controls the UART module by accessing the following internal registers.

- * Data transmitting-side/receiving-side registers (FIFO)
- * Bit rate register
- * Mode register
- * Receiving-side FIFO control register
- * Transmitting-side FIFO control register
- * Line control register

* Status register

8.7 I²C Interface

The DM320 includes an Inter Integrated Circuit(I²C) serial us interface that can act as single master. Both 100kbps and 400kbps I²C-bus system is implemented. The interface uses 7bit/10bit addressing. The DM320 initiates and terminates transfers and generates clock signals.

8.8 GIO

General-purpose IOs(GIOs) consists of 41 pins. GIOs serve two purpose. One is to provide users a general-purpose programmable I/O which gives a convenient way of performing simple control to external devices. The other is to provide additional dedicated interface signals to external devices. With the additional signals, DM320 is able to either increase the performance or the functionality of the overall system. Since the GIOs can not perform both functions at the same time, it is up to the users to trade off between general-purpose programming IOs and dedicated feature and performance enhanced functions.

8.8.1 Parallel IO mode

Each GIO port can be set to input or output. If a GIO port is set to input, the ARM can read the status of that GIO port by GIO port registers. Also can control the polarity of input signal. If a GIO port is set to input and set to be inverted, then the reverse polarity will be read from the GIO port registers.

In the GIO module, output is controlled using a set register and a clear register. When a GIO port is set to output, 1 is output when 1 is written in the relevant bit in the set register, and 0 is output when 1 is written in the relevant bit of the clear register. It is possible to control only the bits you wish to control, without affecting the output status of other GIO ports.

8.8.2 External Interrupt Settings

A GIO port can be used as an external ARM interrupt port. GIO0 through GIO15 can be used as external interrupt ports. A maximum of 16 external interrupts can be specified by settings in registers in the GIO module. These interrupt signals are supplied to the interrupt controller. Either rise or fall can be selected as the interrupt trigger, but the pulse width must be at least two clocks of the DM320 internal ARM clock.

8.8.3 GIO Functional Settings

GIO ports (GIO0-GIO33) can be used as dedicated function ports rather than general-purpose input/output ports. Dedicated ports can be selected by setting the proper bits in the GIO registers. A table of GIO ports and signals that can be selected is shown in Table 37.

After a reset all GIO pins are defined as non-inverted, general-purpose input.

Table 37 GIO port selection output

Pin name	FSEL bits	pin attr	bits state "00"	pin attr	bits state "01"	pin attr	bits state "10"	pin attr	bits state "11"	memo
GIO0	[1:0]	I/O	GIO	I/O	GIO		reserved		reserved	
GIO1	[1:0]	I/O	GIO	I/O	GIO		reserved		reserved	
GIO2	[1:0]	I/O	GIO	I/O	GIO		reserved		reserved	
GIO3	[1:0]	I/O	GIO	I/O	GIO		reserved		reserved	
GIO4	[1:0]	I/O	GIO	I/O	GIO		reserved		reserved	
GIO5	[1:0]	I/O	GIO	I/O	GIO		reserved		reserved	
GIO6	[1:0]	I/O	GIO	I/O	GIO		reserved		reserved	
GIO7	[1:0]	I/O	GIO	I/O	GIO		reserved		reserved	

Pin name	FSEL bits	pin attr	bits state "00"	pin attr	bits state "01"	pin attr	bits state "10"	pin attr	bits state "11"	memo
GIO8	[1:0]	I/O	GIO	I/O	GIO		reserved		reserved	
GIO9	[1:0]	I/O	GIO	O	DSP_XF		reserved		reserved	
GIO10	[3:2]	I/O	GIO	O	DSP_BDX1		reserved		reserved	
GIO11	[5:4]	I/O	GIO	I/O	DSP_BCLKX1		reserved		reserved	
GIO12	[5:4]	I/O	GIO	I/O	DSP_BFSX1		reserved		reserved	
GIO13	[5:4]	I/O	GIO	I	DSP_BDR1		reserved		reserved	
GIO14	[5:4]	I/O	GIO	I/O	DSP_BCLKR1		reserved		reserved	
GIO15	[5:4]	I/O	GIO	I/O	DSP_BFSR1		reserved		reserved	
GIO16	[7:6]	I/O	GIO	I	DSP_BCLKS1		reserved	O	CLKOUT0	General clock out0(12MHz)
GIO17	[9:8]	I/O	GIO	O	VBUSCHARGE	I	VLINQ_RXD0	I/O	GIO	
GIO18	[9:8]	I/O	GIO	O	VBUSEN	I	VLINQ_RXD1	I/O	GIO	
GIO19	[9:8]	I/O	GIO	I	VBUSVAL	I	VLINQ_RXD2	I/O	GIO	
GIO20	[9:8]	I/O	GIO	I	VBUSSES	I	VLINQ_RXD3	I/O	GIO	
GIO21	[9:8]	I/O	GIO	I	VBUSLO	O	VLINQ_TXD0	I/O	GIO	
GIO22	[9:8]	I/O	GIO	I	CID	O	VLINQ_TXD1	I/O	GIO	
GIO23	[9:8]	I/O	GIO	O	PUCON	O	VLINQ_TXD2	I/O	GIO	
GIO24	[9:8]	I/O	GIO	O	PDCON	O	VLINQ_TXD3	I/O	GIO	
GIO25	[9:8]	I/O	GIO	O	IDEN	I/O	VLINQ_SCRUN	O	CLKOUT1A	General clock out1A
GIO26	[9:8]	I/O	GIO	I/O	GIO	I/O	VLINQ_CLOCK	O	CLKOUT2A	General clock out2A
GIO27	[11:10]	I/O	GIO	I	UART_RXD1		reserved		reserved	
GIO28	[11:10]	I/O	GIO	O	UART_TXD1		reserved		reserved	
GIO29	[13:12]	I/O	GIO	I/O	GIO	I/O	SIF_EN		reserved	
GIO30	[13:12]	I/O	GIO	O	I2C_CLK	I/O	SIF_CLK		reserved	
GIO31	[13:12]	I/O	GIO	I/O	I2C_DATA	I	SIF_DI		reserved	
GIO32	[13:12]	I/O	GIO	I/O	GIO	O	SIF_DO		reserved	
GIO33	[15:14]	I/O	GIO	O	PWM0		reserved	O	CLKOUT1B	General clock out1B
GIO34	[17:16]	I/O	GIO	O	PWM1	O	R2	O	CLKOUT2B	General clock out2B
GIO35	[19:18]	I/O	GIO	O	TMRC0		reserved		reserved	
GIO36	[21:20]	I/O	GIO	O	TMRC1		reserved		reserved	
GIO37	[23:22]	I/O	GIO	O	MIRQ	O	B2	O	MIRST	
GIO38	[25:24]	I/O	GIO	I	EXT_LINE_ID		reserved		reserved	
GIO39	[27:26]	I/O	GIO	O	FIELD_VENC		reserved		reserved	FIELD_VENC (LCD_OE)
GIO40	[29:28]	I/O	GIO	I	C_WEN		reserved		reserved	

8.9 Card Detect Function

Card Detect has two functions.

1. Provide an indication that the card status (in or out) changed during stand-by mode.
2. Causes external interrupt if a card is plugged in or out when DM320 is active.

CARD_DET signal is inputted from GIO8, High level input when Card is plugged in. Low level input when Card is plugged out.

9 External CPU Interface

This module allows a host device to access AIM, DSP Internal memory and SDRAM connected to DM320.

By setting the pin EXTHST to high level, a host device can access some of the memory areas of DM320. When in external CPU mode, ARM_A22-ARM_A0, EM_CE0, EM_WE and EM_OE become input pins. The data bus is fixed at 16-bits and uses ARM_D15-ARM_D0. Access from a host device can be performed dynamically by enabling EM_WAIT output for the host device to monitor the status of the DM320. DM320 can interrupt the host device through GIO25, after GIO25 has been configured to function as the MIRQ (interrupt) signal.

9.1 Features

- ❑ The data bus is fixed at 16-bits and uses ARM_D15-ARM_D0. Both read and write are half-word (16 bit) access.
- ❑ Access from a host device can be performed dynamically by enabling EM_WAIT output for the host device to monitor the status of the DM320 ('0':Busy, '1':ready). WAITEN register of the External CPU Interface must be set in advance to enable the EM_WAIT output.
- ❑ The access time seen from the host device varies depending on ARM, DSP and SDRAM clock frequency settings.
- ❑ 32byte Burst/Continuous Burst access to SDRAM is supported via sixteen 16bit data registers.
- ❑ Host device can issue an interrupt signal to the ARM by setting the register INT.
- ❑ If a read is performed continuously after read, EM_OE or EM_CE0 must once return to high level. RTRG register must be set in case EM_OE becomes high level and EM_CE0 stays low.

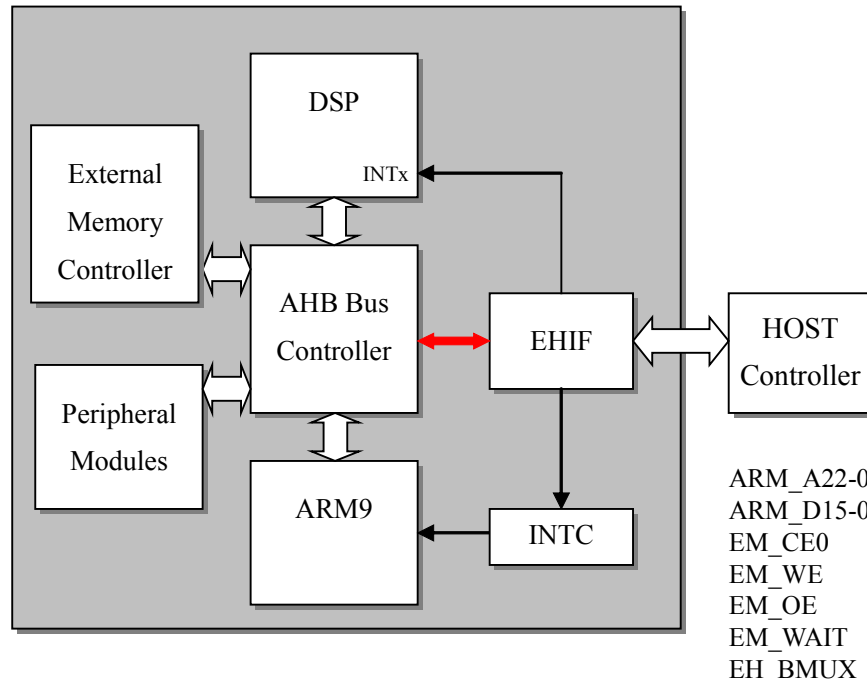


Figure 62 Host Interface mode

9.2 Memory Space

The memory areas that can be accessed from an external host controller are ARM internal memory (16 Kbytes), SDRAM (16,128Kbytes), DSP internal memory (128 Kbytes) and Peripheral modules (4K bytes). However, the memory map addresses differ from the usual ARM memory map. Table 38 shows the memory map of DM320 as seen from the external CPU.

Table 38 Memory Space

Access Type	Memory Area	Ext Host Memory Map		ARM Memory Map		size(byte)
		Start address	End address	Start Address	End Address	
Direct	EHIF registers	0x00:0000	0x00:007F	N/A	N/A	128
	Peripherals	0x00:1000	0x00:1FFF	0x0003:0000	0x0003:0FFF	4k
	ARM TCM	0x00:8000	0x00:DFFF	0x0002:0000	0x0002:5FFF	24 K
	DSP Internal Memory	0x02:0000	0x03:FFFF	0x0004:0000	0x0005:FFFF	128 K
	SDRAM	0x04:0000	0xFF:FFFF	0x0090:0000	0x018B:FFFF	16,128 K
Register	All (Burst transfer)	--	--	0x0000:0000	0x1FFF:FFFF	512M

ARM/DSP internal memory access on the DM320 are in shared-access mode, meaning both the DM320 and the host CPU can access memory. Asynchronous host accesses are resynchronized internally, and in the event that the ARM9 in DM320 and the host CPU request access to the same memory block at the same time, the host CPU has access priority. But please note that the host CPU doesn't suspend an access by the ARM9 in the middle of the access although the host CPU has higher priority than the ARM9. At that time, the access by the host CPU waits for completion of the access by ARM9. The priority takes effect only when the both start to access to the same memory block at the same time. The access priority to the external SDRAM is configured by priority registers in SDRAM controller. The EM_WAIT pin can provide handshaking to the host during memory access.

9.3 Bus Multiplex Access Mode

The EHIF supports address/data bus multiplex access. This mode can be used by setting EH_BMUX pin to high.

In this mode, both address and data are transferred via ARM_D15-ARM_D0. ARM_A22 pin is used to indicate which address or data is transferred in the access. The address are 23-bit, so two address access need to be done consecutively. ARM_A21-ARM_A0 are not used in this mode.

Memory map is not changed even in bus multiplex mode.

The following steps show the procedure to transmit or receive the data in this mode.

1. Send the start address to or from which the data is written or read.

To transfer the address, perform two write accesses consecutively whichever you will write data to DM320 or read data from DM320. The address is transferred via ARM_D bus, and the MSBs(bit22-16) must be transmitted the first and the LSBs(bit15-0) must be transmitted the next. During these accesses, ARM_A22 need to be high to indicate that the address is transferred. ARM_D15-ARM_D7 are ignored when you write the MSBs of the address.

2. Transfer data.

Write or read 16-bit data via ARM_D15-0. ARM22 need to be low to indicate that the data is transferred. After the access, target address is linearly incremented automatically. So you can consecutively transfer bytes of data to sequential address without sending the address through ARM_D bus between each byte of data. Please note that If the address reaches to the maxim value(0x3FFFFFF), the address is reset to 0x000000 for the next 16-bit transfer.

3. Back to 1 when you want to change the address.

9.4 Access Time

The External Host I/F timings that are referenced to internal ARM clock (ARM_CLK) frequency. The access time seen from the host CPU varies depending on ARM, DSP and SDRAM clock frequency settings. Moreover wait cycles will be inserted if there is other access by other AHB bus master to same memory block. The number of cycles of the ARM clock is shown below.

EHIF regist ers	Write	2 cycles + α
	Read	2 cycles + α
TCM	Write	3 cycles + α
	Read	4 cycles + α
SDR AM	Write	(3 + n) cycles + α
	Read	(3 + n) cycles + α

α : Input and/or Output delay(ns)

n: Peripheral wait cycles

If a read is performed continuously after read, EM_OE must be high when RTRG(EHIFCTL[4]) is '1', and EM_CS0 must be high when RTRG(EHIFCTL[4]) is '0'.

If a write is performed continuously after write, EM_WE must be high.

Access from a host CPU to the DM320 can be performed dynamically by using the DM320's EM_WAIT output. The host CPU can monitor the status of the EM_WAIT signal ('0':Busy, '1':ready) and will wait to transfer data until the DM320 is not busy. To enable the EM_WAIT output signal for dynamic access, set the bit WAITEN to 1 in the EHIFCTL (0x01) register. Please note that EM_WAIT output is not asserted when an access takes only two cycles

even if WAITEN bit is 1. Also the host CPU can monitor the status of the AHBBSY register in the EHIFSTS(0x18) register.

9.5 SDRAM access

Single/32byte Burst/Continuous Burst access to SDRAM is supported. Sixteen 16bit data registers are used for Burst/Continuous Burst access. When in Continuous Burst transfer mode, data of the specified length in 32 byte unit are transferred to and from SDRAM. There are 2 modes for Continuous Burst access.

mode1 : Address decode enabled. Register, AIM, HPIB and SDRAM single accesses are allowed during Continuous Burst access. There are some registers to monitor the status of EHIF during Continuous Burst access which are:

Burst transfer wait flag : To check the Burst transfer status every 16 accesses. This is for a host device which does not have a WAIT input.

SDRAM access wait flag : To check the access status between EHIF and SDRC before you start access to AIM/HPIB/SDRAM(single) during Continuous Burst transfer, or before you end the Continuous Burst transfer.

Mode2 : No address decodes. No other access is allowed during DMA.

Preliminary

10 ARM-DSP Communication

This section describes the hardware and software interface between the ARM subsystem and the DSP subsystem. The Host Port Interface Bridge (HPIB) facilitates communication between the ARM and DSP.

10.1 Introduction

The DSP Controller in the DM320 controls data transfer and interrupts between the host and DSP, via the Host Port Interface Bridge (HPIB). DM320 has the Texas Instruments TMS320VC5409-equivalent DSP built into it. This DSP has an Host Port Interface (HPI), which allows communication between a host CPU, such as the ARM7 core used in the DM320.

As shown in Figure 63, the ARM and DSP can communicate through the HPI. The ARM (or external host) can access the DSP memory through a 128-Kbyte window into the DSP memory map. A software designer can define a communication protocol and data structures understood by the ARM and DSP to pass command requests, acknowledgements, and datagram between the ARM and DSP. HPIB module also has DMA controller inside, it enables data transfer from/to DSP internal memory to/from external memory (SDRAM, Flash ROM etc.) through memory traffic controller (MTC) module. This DMA controller is managed by DSP through XIO bus of DSP.

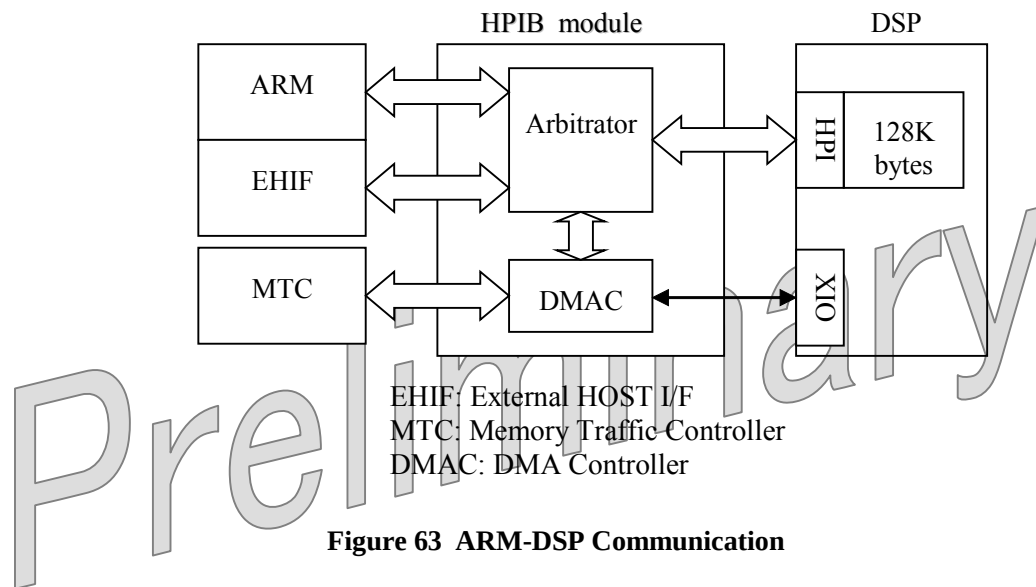


Figure 63 ARM-DSP Communication

10.2 HPI-16 mode

Normally, the HPI in the TMS320VC5409 DSP can be used in two modes, HPI-8 mode and HPI-16 mode. However the TMS320VC5409-equivalent DSP built into the DM320 can be used only in non-multiplexed HPI16 mode. In non-multiplexed HPI-16 mode, the ARM can read and write the DSP memory through a 16-bit bus. However, in this mode handshaking is not possible like it is in HPI-8 mode. Although handshaking is not possible, the ARM can still use the INT0 interrupt to interrupt the DSP, and the DSP can use an I/O port mapped to DSP address 0xFFFF to interrupt the ARM. For additional information on non-multiplexed HPI16 mode, refer to *TMS320C54x DSP Reference Set, Volume 5: Enhanced Peripherals*.

10.3 Memory Access

The DSP internal memory (32-Kwords of DARAM, 32-Kwords of SARAM) can be read and written by the ARM or an external host through the HPI-Bridge. Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM(data) and 0xC000-0xFFFFh of DSP SARAM(PAGE1:program) correspond to addresses 0004:0000-0005:FFFF in the ARM memory map. Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM(data) and 0xC000-0xFFFFh of DSP SARAM(PAGE1:program) correspond to addresses 0002:0000-0003:FFFF in the external host memory map. The memory maps of the ARM and external HOST are shown in Table 39 and Table 40, respectively. When the endianness of the ARM and DSP do not match, EXCHG in HPIBCTL (0x00) can be used to swap the upper and lower 8-bits of data transferred between the ARM and DSP.

Table 39 ARM Memory Map

ARM address(byte)	DSP address(word)	DSP memory	Size(byte)
0x40000-0x4FFFF	0x0-0x7FFF	Data & Program(Page0)	64k
0x50000-0x57FFF	0x8000-0xBFFF	Data	32k
0x58000-0x5FFFF	0xC000-0xFFFF	Program(Page1)	32k

Table 40 External HOST Memory Map

ARM address(byte)	DSP address(word)	DSP memory	Size(byte)
0x20000-0x2FFFF	0x0000-0x7FFF	Data & Program(Page0)	64k
0x30000-0x37FFF	0x8000-0xBFFF	Data	32k
0x38000-0x3FFFF	0xC000-0xFFFF	Program(Page1)	32k

10.4 HPI Interrupt

The sections that follow explain how the host and the DSP interrupt each other.

10.4.1 Host Device Using DSPINT0 to Interrupt the DSP

The host (ARM) can interrupt the DSP by writing a 0 to the CPU register in DSP controller(DSPC) module.

10.4.2 DSP Using Bit 0 of 0xFFFF to Interrupt the Host Device

The DSP can interrupt the ARM through the memory mapped I/O port, 0xFFFF. The DSP's HINT pin is not used to interrupt the ARM, since the HPI is operating in a mode that cannot control the HINT pin (non-multiplexed HPI16 mode). The memory map of the DSP's I/O space is shown in [エー! 参照元が見つかりません。](#), and as shown in the table, address 0xFFFF is used to interrupt the ARM from the DSP. This interrupt signal is connected to the ARM's interrupt controller and corresponds to INT11 in the interrupt controller.

10.5 Boot loader

10.5.1 ARM boot sequence

Upon power up and reset, the ARM program counter is reset to 0xFFFF:0000, which is mapped to 0x0010:0000 (start of the CS0 address space). The ARM should then execute a boot sequence from CS0 region to start the DM320 system.

The ARM boot sequence should initialize the different peripheral registers, such as the register for the clock controller module, the SDRAM controller, etc.

Also, the ARM boot phase should reset the DSP via the DSP Controller register bit. When done, the ARM should take the DSP out of reset. At this stage the DSP begins execution at 0xFF80, a hard coded address in ROM, which places the DSP into IDLE 1 mode (a power down mode). This gives the ARM the opportunity to download the DSP

code by using the HPI-16 interface. After it completes downloading the DSP image, the ARM can send an interrupt to the DSP, which wakes it up from IDLE 1 mode and jump to address 0x7F80 where it starts running its application code loaded by the ARM.

10.5.2 DSP Boot Sequence

On the 'C54x DSP, the reset vector is located at address 0x0FF80. Within the DM320, this memory location has been ROM'ed with a small boot loader. The figure55 describes this process. It is achieved in 6 steps:

The DSP goes out of reset and load its program counter (PC) register with 0x0FF80

The ROM code found at this location branches the DSP to address 0x0F800 where the initialization routine resides.

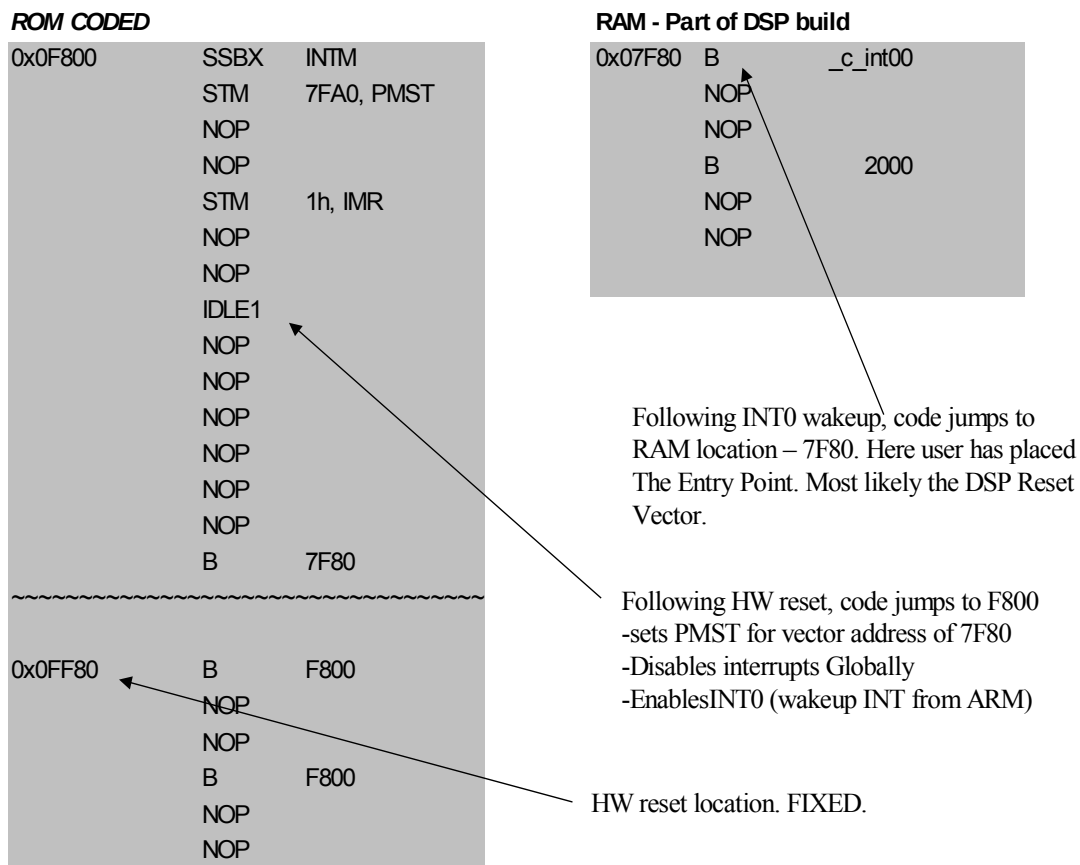
At address 0x0F800, the DSP status register PMST is initialized to mode the vector table to 0x07F80, the interrupts is disabled except for INT0 and finally, the DSP is sent to a power down mode (Idle 1).

While in this mode, the ARM loads the DSP internal 64k words of Program/Data memory with the DSP code. The code being loading from the arm side is not predefined; refer to the DM320 library and application note for more information.

When the ARM is done downloading the DSP code, it wakes it up from IDLE mode by asserting INT0.

The DSP then branches to address 0x7F80 where the new interrupt vector table is located. The ARM should have loaded this location with at least a branch to the start of your code.

Figure 64 Host Interface mode



Preliminary

11 Power Saving Modes

The DM320 has been designed as a low power device, however, in some cases, it could be necessary to reduce even more the power consumption.

The following describes different modes that the DSP or ARM cores can be placed into to save power.

11.1 DSP Idle modes

The C54x DSP embedded in the DM320 has power-down modes in which it enters a dormant state and dissipates less power than normal operation while maintaining the CPU contents. This allows operations to continue unaltered when the power-down mode is terminated.

You can invoke one of the power-down modes either by executing the IDLE 1 IDLE 2 instructions.

11.1.1 IDLE1 Mode

The IDLE1 mode halts all CPU activities except the system clock. Because the system clock remains applied to the peripheral modules, the peripheral circuits continue operating and the CLKOUT pin remains active. Thus, peripherals such as McBSP and timers can take the CPU out of its power-down state.

Use the IDLE 1 instruction to enter the IDLE1 mode. To terminate IDLE1, use a wake-up interrupt. If INTM = 0 when the wake-up interrupt takes place, the C54x DSP enters the ISR when IDLE1 is terminated. If INTM = 1, the C54x DSP continues with the instruction following the IDLE 1 instruction. All wake-up interrupts must be set to enable the corresponding bits in the IMR register regardless of the INTM value. The only exceptions are the non-maskable interrupts, RS and NMI.

11.1.2 IDLE2 Mode

The IDLE2 mode halts the on-chip peripherals as well as the CPU. Because the on-chip peripherals are stopped in this mode, they cannot be used to generate the interrupt to wake up the DSP as with IDLE1. However, power is significantly reduced because the device is completely stopped.

Use the IDLE 2 instruction to enter the IDLE2 mode. To terminate IDLE2, activate any of the external interrupt signal (RS, NMI, and INT0) with a 10-ns minimum pulse. If INTM = 0 when the wake-up interrupt takes place, the C54x DSP enters the ISR when IDLE2 is terminated. If INTM = 1, the C54x DSP continues with the instruction following IDLE 2 instruction. All wake-up interrupts must be set to enable the corresponding bits in the IMR register regardless of the INTM value. Reset all peripherals when IDLE2 terminates, especially the McBSP when the CLKS pin is used.

When RS is the wake-up interrupt in IDLE2, a 10-ns minimum pulse of RS can activate the reset sequence.

11.2 ARM Power Saving Modes

The ARM9 subsystem used in the DM320 features several power-saving modes to lengthen the operating time on batteries powered systems. The content of the CPU and the registers settings are maintained when the device is in one of these modes.

Depending on the power down mode chosen, there is different ways to enter and exit it. Each one is described below.

11.2.1 Sleep Mode:

Clock supply to each module can be stopped by setting the module input clock off. This is done by setting the proper bits in the MODx control registers. If clock supply to the ARM core, ARM internal memory and bus controller is stopped, DM320 is completely stopped. Therefore, these clocks must be stopped last.

Even if all clocks in DM320 are stopped, RAM contents are saved as long as power is not lost, and it is not necessary to reload ARM and DSP programs. If you wish to save SDRAM contents, the SDRAM must be set to self-refresh mode before stopping the SDRAM clock.

Recovery from sleep is performed by GIO0. You need to apply a pulse signal on the GIO0 pin (active low) at least the width of two clocks of SYSCLK or X'tal clock. Since the PLLs still operate, the full recovery is quick.

11.2.2 Sleep by sleep mode register setting

Clock supply to all modules and the built-in PLLs can be stopped by setting the sleep mode register in the CLOCK controller. Recovery from sleep mode is performed by GIO0, but since it takes 8192 clock cycles until the internal PLLs are stable, this recover time is slow.

Recovery using GIO0 is performed, but the recovery times are progressively slower for the three methods Sleep Mode:

When you write '1' into the sleep mode bit of the clock controller register (SLPMD bit of CLKC) the DSC enters into sleep mode. When in this mode, all the internal PLLs and internal clocks are stopped. The SLPMD is cleared by pulling the GIO0 pin to the ground for at least the width of the input clock.

11.2.3 Power-down mode

When the device is placed in a sleep mode by pulling the PWDN pin low, the oscillation of the external oscillator cell stops when entering the power down mode.

If X'tal (MXI, MXO) is used as the clock source to ARM, a more uniform power-down can be implemented by stopping its oscillation. The PWRDN terminal is used in power down of the oscillator. When this terminal is HIGH, oscillation is stopped. To recover, first set the PWRDN terminal to LOW, and wait for stable oscillation (about same time as power reset). After that, recovery using GIO0 is performed. Compared to the power down mode, the recovery time with respect to the crystal oscillation stabilizing time is longer.

Stopping of 48 MHz oscillators (M48XI, M48XO) can also be performed by setting registers in the CLOCK controller.

11.2.4 Wake up Method

It returns from above-mentioned powering down * mode, state of one time device it is necessary to make sleep * mode. The difference of powering down * mode and sleep * mode is whether the PWDN terminal '1' or '0' just. First, the PWDN terminal is reset to '0', oscillation of the input clock is made to put in stable state. Because (the same time as power on * reset) this state already has become sleep * mode, when low * pulse is inputted from the GIO0 terminal as a external interrupt, the edge is detected in inside, the sleep * mode * bit is cleared, returns to normal state. Because as for on & off of the clock immediately before entering into sleep * mode, you follow setting, the ARM and the other things, as for the module whose operation is necessary the occasion where it returns unless it sets to on by all means, after the returning being not to operate note is necessary.

11.2.5 PLL Bypass Mode

The BPSMD (the CLKC [13]) by the fact that it sets to '1', the PLL becomes bypass * mode. Bypass * mode, discharge doing the VCO of all PLL, it points to the state of the input clock = output clock of the PLL. (It does not influence the M/N) it is mode just in order to make the electric power consumption of the PLL minimum.

11.3 DAC Power

If you want to tune off the power to unused DACs, you must do as follows:

The following conditions must always be met:

- Connect VSSA and VSS3V to ground
- Connect VDD3V to power (3.3V)

The voltage of pins IREF, DAOUT, BIAS, and VREF_RGB must satisfy following formula:

$$\text{Voltage (VDDA) } \geq \text{Voltage (IREF,DAOUT,BIAS,VREF_RGB)}$$

Thus, IREF, BIAS, and VREF_RGB must be LOW.

Because of terminal resistance connected to VSSA, DAOUT goes to GND. DAOUT is an output pin, so you (the user) don't have to worry about DAOUT.

Preliminary

12 PIN Assignment

pad count	Pin Name	dir	MUX'ed 1	MUX'ed 2	MUX'ed 3	MUX'ed 4	MUX'ed 5	MUX'ed 6
1	IREF	in						
2	AOUT	out						
3	VSSA	gnd						
4	VDDAHV	power						
5	BIAS	in						
6	VREFIN	in						
7	VSS3V	gnd						
8	VDD3VHV	power						
9	M48XI	bidi						
10	M48XO	out						
11	MXI	bidi						
12	MXO	out						
13	VDD1	power						
14	VSS1	gnd						
15	VDD2	power						
16	VSS2	gnd						
			(CCD data)	(YCbCr 16bit)	(YCbCr 8bit)	(ETM9)		
17	PCLK	in						
18	VD	bidi						
19	HD	bidi						
20	FIELD_CCD	bidi						
21	CI7	bidi		Cb7,Cr7	Y7,Cb7,Cr7	TRCE_SYNC		
22	CI6	bidi		Cb6,Cr6	Y6,Cb6,Cr6	TRACE_CLK		
23	CI5	in	C_DATA13	Cb5,Cr5	Y5,Cb5,Cr5			
24	CI4	in	C_DATA12	Cb4,Cr4	Y4,Cb4,Cr4			
25	CI3	in	C_DATA11	Cb3,Cr3	Y3,Cb3,Cr3			
26	CI2	in	C_DATA10	Cb2,Cr2	Y2,Cb2,Cr2			
27	CI1	in	C_DATA9	Cb1,Cr1	Y1,Cb1,Cr1			
28	CI0	in	C_DATA8	Cb0,Cr0	Y0,Cb0,Cr0			
29	YI7	in	C_DATA7	Y7	Y7,Cb7,Cr7			
30	YI6	in	C_DATA6	Y6	Y6,Cb6,Cr6			
31	YI5	in	C_DATA5	Y5	Y5,Cb5,Cr5			
32	YI4	in	C_DATA4	Y4	Y4,Cb4,Cr4			
33	YI3	bidi	C_DATA3	Y3	Y3,Cb3,Cr3	TRACE_PKT7		
34	YI2	bidi	C_DATA2	Y2	Y2,Cb2,Cr2	TRC_PEST2		
35	YI1	bidi	C_DATA1	Y1	Y1,Cb1,Cr1	TRC_PESTT1		
36	YI0	bidi	C_DATA0	Y0	Y0,Cb0,Cr0	TRC_PEST0		
			(SDRAM)	(ext Flash)	(CFC)	SM,Nand,xD	(Ext Host)	

37	SDR_CLK	bidi	SDR_CLK				
38	SDR_CKE	out	SDR_CKE				
39	SDR_CS	out	SDR_CS				
40	SDR_WE	out	SDR_WE				
41	SDR_DQM3	out	SDR_DQM3				
42	SDR_DQM2	out	SDR_DQM2				
43	SDR_DQM1	out	SDR_DQM1				
44	SDR_DQM0	out	SDR_DQM0				
45	SDR_RAS	out	SDR_RAS				
46	SDR_CAS	out	SDR_CAS				
47:61	SDR_A[14:0]	out	SDR_A[14:0]				
62:93	SDR_D[31:0]	bidi	SDR_D[31:0]				
94	FLASH_CE	out		FLASH_CE			
95	CF_CE1	out			CF_CE1	SM_CE	
96	CF_CE2	out			CF_CE2	SM_ALE	
97	EM_OE	bidi		EM_OE			EH_OE
98	CF_OE	out			CF_OE	SM_CLE	
99	EM_WE	bidi		EM_WE	CF_WE		EH_WE
100	CF_IORD	out			CF_IOR	SM_RE	
101	CF_IOWR	out			CF_IOW	SM_WE	
102	CF_IOIS16	in			CF_IOIS16		
103	EM_WAIT	bidi		EM_WAIT			EH_WAIT
104	CF_WAIT	in			CF_WAIT		
105:120	ARM_D[15:0]	bidi		EM_D[15:0]	CF_D[15:0]	SM_D[15:0]	EH_D[15:0]
121:143	ARM_A[22:0]	bidi		DM_A[22:0]	CF_REG CF_A[10:0]		EH_A[22:0]
144	EM_BEH	out		EM_BEH			
145	EM_BEL	out		EM_BEL			
146	EM_CE0	bidi		EM_CE0			EH_CS
147	EM_CE1	out		EM_CE1			
148	CF_RDY	in			CF_RDY	SM_RYBY	
149	BUSWIDTH	in					
150	FLASHSEL	in					
151	EXTHOST	in					
152	EH_BMUX	in					
153	GIO8	bidi	General I/O	Ext INT (Card Det)			
			(MS Pro)	(SD)	(MMC)	(SPI mode)	
154	SD_CLK	bidi	CLK	CLK	CLK	CLK	
155	SD_CMD	bidi	Enable	CMD	CMD	data to the card	
156	SD_DATA3	bidi	data line 3	data line 3		card select line	
157	SD_DATA2	bidi	data line 2	data line 2			
158	SD_DATA1	bidi	data line 1	data line 1			
159	SD_DATA0	bidi	data line 0	data line 0	data line 0	data	

			(Digital LCD YUV 16bits)	(Std RGB)	EP LCD	(CASIO LCD)	(Micro Display)	(S LO
160	HSYNC	bidi	HSYNC	HSYNC	HSYNC	HSYNC		CL1
161	VSYNC	bidi	VSYNC	VSYNC	VSYNC	VSYNC	FRM_START	FLM
162	VCLK	out	VCLK	DCLK	DCLK	DCLK	CLK	CL2
163	YOUT7	bidi	YOUT7	DATA7	DATA5	DATA5	VIDEO_IN7	DAT
164	YOUT6	bidi	YOUT6	DATA6	DATA4	DATA4	VIDEO_IN6	DAT
165	YOUT5	bidi	YOUT5	DATA5	DATA3	DATA3	VIDEO_IN5	DAT
166	YOUT4	bidi	YOUT4	DATA4	DATA2	DATA2	VIDEO_IN4	DAT
167	YOUT3	bidi	YOUT3	DATA3	DATA1	DATA1	VIDEO_IN3	DAT
168	YOUT2	bidi	YOUT2	DATA2	DATA0	DATA0	VIDEO_IN2	DAT
169	YOUT1	bidi	YOUT1	DATA1	GCP	STBYB	VIDEO_IN1	DAT
170	YOUT0	bidi	YOUT0	DATA0	LP	RIT	VIDEO_IN0	DAT
171	COUT7	bidi	COUT7	LCD_AC	RES	CP		
172	COUT6	bidi	COUT6	LCD_OE	XINH	STH	DATA_VALID	
173	COUT5	bidi	COUT5	BRIGHT	YSCL	STB		
174	COUT4	bidi	COUT4	COMP_SYNC	YSCLD	GCPK	DISP_CLK	
175	COUT3	bidi	COUT3	PWM_OUT	FRYS	GRES		
176	COUT2	bidi	COUT2		FRYP	POL		
177	COUT1	bidi	COUT1		FRX		BALANCE	
178	COUT0	bidi	COUT0		DY	GSRT	FULL	
179	GIO0	bidi	General I/O	Ext INT (WakeUp)				
180	GIO1	bidi	General I/O	Ext INT				
181	GIO2	bidi	General I/O	Ext INT				
182	GIO3	bidi	General I/O	Ext INT				
183	GIO4	bidi	General I/O	Ext INT				
184	GIO5	bidi	General I/O	Ext INT				
185	GIO6	bidi	General I/O	Ext INT				
186	GIO7	bidi	General I/O	Ext INT				
						(ETM9)		
187	DSP_BDX0	out				TRACE_PKT6		
188	DSP_BCLKX0	bidi				TRACE_PKT5		
189	DSP_BFSX0	bidi				TRACE_PKT4		
190	DSP_BDR0	bidi				TRACE_PKT3		
191	DSP_BCLKR0	bidi				TRACE_PKT2		
192	DSP_BFSR0	bidi				TRACE_PKT1		
193	DSP_BCLKS0	bidi				TRACE_PKT0		
194	GIO9	bidi	General I/O	Ext INT	DSP_XF			
195	GIO10	bidi	General I/O	Ext INT	DSP_BDX1			
196	GIO11	bidi	General I/O	Ext INT	DSP_BCLKX1			
197	GIO12	bidi	General I/O	Ext INT	DSP_BFSX1			
198	GIO13	bidi	General I/O	Ext INT	DSP_BDR1			
199	GIO14	bidi	General I/O	Ext INT	DSP_BCLKR1			
200	GIO15	bidi	General I/O	Ext INT	DSP_BFSR1			

201	GIO16	bidi	General I/O		DSP_BCLKS1		Gen clk out0	
					USB1.1OTG	(VLYNQ)		
202	USB_DP	bidi			?			
203	USB_DM	bidi			?			
204	USB_VDD	power						
205	GIO17	bidi	General I/O		VBCHARGE	VLYNQ_RXD0		
206	GIO18	bidi	General I/O		VBUSEN	VLYNQ_RXD1		
207	GIO19	bidi	General I/O		VBUSVAL	VLYNQ_RXD2		
208	GIO20	bidi	General I/O		VBUSSES	VLYNQ_RXD3		
209	GIO21	bidi	General I/O		VBUSLO	VLYNQ_TXD0		
210	GIO22	bidi	General I/O		CID	VLYNQ_TXD1		
211	GIO23	bidi	General I/O		PUCON	VLYNQ_TXD2		
212	GIO24	bidi	General I/O		PDCON	VLYNQ_TXD3		
213	GIO25	bidi	General I/O		IDEN	VLYNQ_SCRN	Gen clk out1A	
214	GIO26	bidi	General I/O			VLYNQ_CLK	Gen clk out2A	
215	RXD0	in		UART_RXD0	AIMSIF DIN			
216	TXD0	out		UART_TXD0	AIMSID DOUT			
217	SIF_EN	bidi	SIF w/DMA EN		AIMSIF EN			
218	SIF_CLK	bidi	SIF w/DMA CLK		AIMSIF CLK			
219	SIF_DI	in	SIF w/DMA DI					
220	SIF_DO	out	SIF w/DMA DO					
221	GIO27	bidi	General I/O	UART_RXD1				
222	GIO28	bidi	General I/O	UART_TXD1				
223	GIO29	bidi	General I/O		SIF EN			
224	GIO30	bidi	General I/O	I2C_CLK	SIF CLK			
225	GIO31	bidi	General I/O	I2C_DATA	SIF DI			
226	GIO32	bidi	General I/O		SIF DO			
227	GIO33	bidi	General I/O	PWM0			Gen clk out1B	
228	GIO34	bidi	General I/O	PWM1			Gen clk out2B	
229	GIO35	bidi	General I/O		(CCD shut)			
230	GIO36	bidi	General I/O		(CCD strobe)			
231	GIO37	bidi	General I/O			MIRQ	MRST	
232	BTSEL0	in						
233	BTSEL1	in						
234	SYSCLK	in						

235	RSTN	in						
		in						
236	PWRDN							
237	TCK	in						
			JTAG that	JTAG that	JTAG that			
238	TDI	in	is	is	is			
			used at	used at	used at			
239	TMS	in	TOP	ARM	DSP			
240	TRST	in						
241	TDO	out						
242	SCANEN	in						
243	CLKSEL	in						
244	TEST2	in						
245	TEST1	in						
246	TEST0	in						
				EXT_LINE_ID (VENC UNIPAC LCD)				
247	GIO38	bidi	General I/O					
248	GIO39	bidi	General I/O	FIELD_VENC				
249	GIO40	bidi	General I/O	C_WEN				
250	RTCK	bidi				RTCK	TEST4	
251	EMU0	bidi						
252	EMU1	bidi						

Preliminary

Pin-out Summary

Preliminary